



**ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA
INFORMÁTICA**

TRABAJO FIN DE GRADO

**Desarrollo de un generador configurable de
modelos de características UVL integrado en
UVLHub**

Realizado por

Jiménez Ortega, Antonio

Para la obtención del título de

Grado en Ingeniería Informática - Ingeniería del Software

Dirigido por

Benavides Cuevas, David

Romero Organvández, David

En el departamento de

Lenguajes y Sistemas Informáticos

Convocatoria de octubre, curso 2025/26

DECLARACIÓN DE AUTORÍA

D. **Jiménez Ortega, Antonio**, autor del Trabajo Fin de Grado titulado «Desarrollo de un generador configurable de modelos de características UVL integrado en UVLHub», presentado en Sevilla, octubre 2026 para la obtención del título de Grado en Ingeniería Informática - Ingeniería del Software, declara bajo su responsabilidad que:

- El trabajo es original y ha sido realizado por el autor en colaboración con sus tutores académicos.
- Se han citado adecuadamente todas las fuentes consultadas, tanto bibliográficas como digitales.
- No se ha plagiado contenido de otros autores ni se han utilizado materiales sin la correspondiente autorización.
- Autoriza a la Universidad a utilizar este trabajo con fines académicos y de difusión, citando siempre su autoría.

Mairena del Alcor, a 29 de septiembre de 2026

Fdo.: Jiménez Ortega, Antonio

AGRADECIMIENTOS

Quiero expresar mi profundo agradecimiento a mis tutores, David Benavides y David Romero, por la orientación, supervisión y apoyo proporcionados durante el desarrollo de este Trabajo de Fin de Grado. Sus indicaciones y experiencia han sido fundamentales para comprender el contexto del proyecto, tomar las decisiones funcionales y técnicas adecuadas, y afrontar los distintos retos que han ido apareciendo durante su realización.

También quiero agradecer al grupo de DiversoLab y a todas las personas que han contribuido al desarrollo del entorno tecnológico sobre el que se ha construido este trabajo por facilitar un marco en el que aplicar los conocimientos adquiridos durante mis años en el grado.

Finalmente, dedico este trabajo a mi familia y a todo mi entorno cercano. Su acompañamiento y confianza han sido indudablemente esenciales para completar este proyecto y afrontar el esfuerzo que supone un trabajo de esta magnitud. Abuela, dondequiera que estés, ya estoy un paso más cerca de convertirme en ingeniero.

Mairena del Alcor, agosto 2026

ACKNOWLEDGEMENTS

I would like to express my deep gratitude to my mentors, David Benavides and David Romero, for the guidance, supervision, and support they provided throughout the development of this Bachelor's Thesis. Their advice and experience have been essential in helping me understand the context of the project, make the appropriate functional and technical decisions, and face the various challenges that arose during its completion.

I would also like to thank the DiversoLab team and everyone who contributed to the development of the technological environment on which this project was built for providing a framework in which to apply the knowledge I acquired during my undergraduate studies.

Finally, I dedicate this project to my family and all those close to me. Their support and trust have undoubtedly been essential to completing this project and tackling the effort required for a project of this magnitude. Grandma, wherever you are, I'm one step closer to becoming an engineer.

Mairena del Alcor, August 2026

RESUMEN

Este Trabajo de Fin de Grado aborda el desarrollo y la integración de un generador configurable de modelos de características en UVLHub, una plataforma orientada a la gestión y manipulación de modelos escritos en Universal Variability Language (UVL). El trabajo responde a la necesidad de disponer de modelos UVL expresivos y configurables con los que evaluar herramientas de análisis de variabilidad software, incluidos modelos con atributos y restricciones complejas. Para ello, se ha desarrollado un componente generador independiente siguiendo el sistema de extensiones de Flamapy. Este componente separa la configuración del generador de las operaciones que construyen los modelos. Además, se ha adaptado el módulo de UVLHub para delegar en él la lógica de generación, lo que facilita su reutilización y mantenimiento. La solución se ha implementado con Python y Flask. El entorno de desarrollo local se ha definido mediante Docker Compose y se ha desplegado una instancia de revisión en Render. También se ha adaptado la infraestructura de pruebas del módulo. La opción de generación con comprobación SAT limita la generación al nivel booleano y verifica la satisfacibilidad de los modelos generados. La solución se ha comprobado mediante 264 pruebas automatizadas superadas, incluidas pruebas de interfaz con pytest y Selenium. La principal aportación es una base modular para generar modelos UVL destinada a la evaluación y el desarrollo de herramientas de análisis de variabilidad.

Palabras clave: SPL, UVL, UVLHub, Flamapy, modelos de características, generación automática de modelos, Python, Docker, pruebas automatizadas.

ABSTRACT

This Final Degree Project addresses the development and integration of a configurable feature model generator into UVLHub, a platform for managing and manipulating models written in the Universal Variability Language (UVL). The work responds to the need for expressive and configurable UVL models with which to evaluate software variability analysis tools, including models with attributes and complex constraints. To this end, an independent generator component was developed following Flamapy's plugin architecture. The component separates the generator configuration from the operations that build the models. The UVLHub generator module was also adapted to delegate its generation logic to this component, facilitating reuse and maintenance. The solution was implemented with Python and Flask. The local development environment was defined with Docker Compose, and a review instance was deployed on Render. The module's testing infrastructure was also adapted. The SAT-checked generation option limits generation to the Boolean level and checks the satisfiability of the generated models. The solution was verified by 264 passing automated tests, including interface tests using pytest and Selenium. The main contribution is a modular foundation for generating UVL models to support the evaluation and development of variability analysis tools.

Keywords: SPL, UVL, UVLHub, Flamapy, feature models, automatic model generation, Python, Docker, automated testing.

ÍNDICE GENERAL

Declaración de autoría	ii
Agradecimientos	iv
Acknowledgements	v
Resumen	vii
Abstract	viii
Índice de tablas	xvii
Índice de figuras	xx
Lista de acrónimos	xxii
1. Introducción	1
1.1. Contexto	1
1.2. Problema	2
1.3. Motivación	3
1.4. Punto de partida y contribución propia	3
1.5. Objetivos	6
1.6. Alcance	9
1.7. Estructura de la memoria	10
2. Estado del arte	12
2.1. Ingeniería de Líneas de Productos Software (SPL)	12
2.1.1. Líneas de Productos Software	12
2.1.2. Modelos de características	14
2.1.3. Análisis automático y motores de razonamiento	15
2.2. Universal Variability Language (UVL)	18
2.2.1. Estructura básica de un modelo UVL	19
2.2.2. Relaciones y <i>Boolean level</i>	19
2.2.3. Niveles de expresividad avanzados	20
2.3. Ecosistema de herramientas y aplicación en variabilidad software	23
2.3.1. Flamapy como <i>framework</i> de análisis de variabilidad	23
2.3.2. UVLHub como plataforma de gestión de modelos UVL	24
2.3.3. Generación automática de modelos de características	26

2.4.	Trabajos relacionados	27
2.4.1.	Generating Feature Models with UVL's Full Expressiveness . .	27
2.4.2.	S.P.L.O.T.	28
2.4.3.	FeatureIDE	28
2.4.4.	BeTTy	28
2.4.5.	ETHOM	29
2.4.6.	Comparativa de trabajos relacionados	29
3.	Planificación del proyecto	32
3.1.	Acta de constitución del proyecto	32
3.1.1.	Interesados del proyecto	33
3.2.	Metodología	34
3.2.1.	Metodología de desarrollo	34
3.2.2.	Metodología de validación y evaluación	37
3.3.	Estructura de descomposición del trabajo (EDT)	38
3.4.	Cronograma e hitos (línea base)	39
3.5.	Plan de comunicaciones y seguimiento	43
3.6.	Gestión de riesgos	44
3.6.1.	Seguimiento de riesgos al cierre	46
3.7.	Control de cambios	48
3.8.	Plan de calidad	49
3.9.	Presupuesto	50
3.10.	Cierre del proyecto y lecciones aprendidas	53
4.	Análisis de requisitos	55
4.1.	Descripción del sistema	55
4.2.	Requisitos de información	56
4.3.	Requisitos funcionales	63
4.4.	Requisitos no funcionales	66
4.5.	Reglas de negocio	67
4.6.	Restricciones del proyecto	71
4.7.	Casos de uso	72
4.7.1.	Diagrama general de casos de uso	72
4.7.2.	Especificación de casos de uso principales	73
4.8.	Matriz de trazabilidad	78
4.8.1.	Requisitos funcionales $\leftrightarrow$ casos de uso	78
4.8.2.	Requisitos $\leftrightarrow$ pruebas	79
5.	Diseño de la solución	84
5.1.	Modelo conceptual del dominio	84

5.1.1.	Estructura del modelo	84
5.1.2.	Restricciones del dominio	87
5.1.3.	Espacio de soluciones y configuraciones	88
5.2.	Arquitectura del sistema	88
5.2.1.	Visión general	88
5.2.2.	Diagrama de componentes	90
5.2.3.	Descripción secuencial del flujo de generación	90
5.3.	Patrones de diseño aplicados	95
5.3.1.	Patrón de interacción estilo asistente (<i>wizard</i>)	95
5.4.	Diseño de la interfaz de usuario	95
5.4.1.	Mapa de navegación	96
5.4.2.	Prototipo visual inicial	97
5.4.3.	Decisiones de diseño visual	97
5.5.	Flujo de datos principal	98
5.5.1.	Estructura de la representación intermedia	101
5.5.2.	Mapeo entre niveles	101
6.	Implementación	103
6.1.	Análisis de tecnologías	103
6.2.	Tecnologías utilizadas	105
6.3.	Módulo de validación y lógica principal	110
6.3.1.	Validación sintáctica y comprobaciones básicas	110
6.3.2.	Validación semántica y restricciones de dominio	112
6.4.	Módulo de generación de artefactos	114
6.4.1.	Lógica principal de generación de modelos	114
6.4.2.	Síntesis del artefacto final generado	120
6.5.	Rutas HTTP y API del generador	121
6.5.1.	Endpoints implementados	121
6.5.2.	Descarga de modelos UVL generados	122
6.6.	Integración continua	122
7.	Pruebas y validación	125
7.1.	Estrategia de pruebas	125
7.2.	Pruebas unitarias	126
7.2.1.	<code>test_engine.py</code> del paquete <code>fm_generator</code>	126
7.2.2.	<code>test_validators.py</code>	127
7.2.3.	<code>test_wizard_builders.py</code>	127
7.2.4.	<code>test_wizard_persisters.py</code>	127
7.2.5.	<code>test_wizard_state.py</code>	127
7.3.	Pruebas de integración	127

7.3.1. <code>test_params_contract.py</code> del paquete <code>fm_generator</code>	128
7.3.2. <code>test_wizard_flow.py</code>	128
7.3.3. <code>test_wizard_outputs.py</code>	128
7.4. Pruebas de extremo a extremo	129
7.5. Cobertura y métricas de calidad	130
8. Resultados y discusión	133
8.1. Diseño de la evaluación	133
8.2. Resultados	133
8.2.1. Resultados sobre el objetivo O1	133
8.2.2. Resultados sobre el objetivo O2	136
8.2.3. Resultados sobre el objetivo O3	138
8.2.4. Resultados sobre el objetivo O4	139
8.2.5. Resultados sobre el objetivo O5	141
8.2.6. Resultados sobre los objetivos formativos O6–O10	141
8.3. Comparación con trabajos previos	143
8.4. Discusión	144
9. Conclusiones y trabajo futuro	145
9.1. Conclusiones	145
9.2. Limitaciones	145
9.3. Trabajo futuro	146
Bibliografía	149
A. Modelos UVL generados y configuraciones	152
A.1. Modelos ilustrativos por nivel de expresividad	152
A.1.1. Nivel booleano y cardinalidad de grupo	152
A.1.2. Nivel aritmético, cardinalidad de grupo, cardinalidad de característica y funciones de agregación	154
A.1.3. Nivel de tipos y restricciones sobre cadenas	157
B. Interfaz del generador de modelos UVL	160
B.1. Recorrido por el asistente	160
B.2. Prototipos visuales iniciales	171
B.3. Selector visual de distribuciones	175
C. Manual de instalación y despliegue	176
C.1. Requisitos previos	176
C.2. Disponibilidad del código	176
C.3. Instalación de dependencias y ejecución	177

C.4. Despliegue en producción	178
C.5. Verificación de la instalación	178
C.5.1. Ejecución de las pruebas automatizadas	179
C.5.2. Medición de cobertura	179
D. Manual de usuario	181
D.1. Acceso al sistema	181
D.2. Funcionalidad principal	182
D.3. Mensajes de error y resolución de incidencias	182
D.4. Contacto y soporte	183

ÍNDICE DE TABLAS

1.1. Procedencia de los principales elementos relacionados con el desarrollo.	5
1.2. Criterios de éxito de los objetivos y capítulos donde se verifican.	9
2.1. Comparativa entre SAT y BDD como técnicas de análisis automático de modelos de características.	18
2.2. Comparación de los trabajos relacionados según la expresividad, el tamaño documentado, la satisfacibilidad y la validez sintáctica de la salida. “No documentado” indica que la fuente consultada no proporciona esa evidencia.	30
3.1. Acta de constitución del proyecto.	33
3.2. Registro de interesados del proyecto.	34
3.3. Trazabilidad de los repositorios y recursos utilizados durante el desarrollo.	36
3.4. Estructura de descomposición del trabajo (EDT).	39
3.5. Cronograma de ejecución e hitos del proyecto.	40
3.6. Fases de desarrollo y esfuerzo asociado: línea base frente a valores reales.	41
3.7. Fases de redacción de la memoria (estimadas vs. reales).	42
3.8. Resumen global del proyecto (estimadas vs. reales).	43
3.9. Plan de comunicaciones y seguimiento del proyecto.	44
3.10. Riesgos identificados, exposición y planes de mitigación.	45
3.11. Seguimiento de riesgos al cierre: materialización, respuesta y residuales.	48
3.12. Registro de control de cambios sobre la línea base.	49
3.13. Desembolsos de capital registrados durante el proyecto.	51
3.14. Desglose de horas y costes de personal.	52
3.15. Pagos registrados por herramientas y servicios de desarrollo.	52
3.16. Costes directos cuantificados para el proyecto.	53
3.17. Lecciones aprendidas al cierre del proyecto.	54
4.1. Atributos del requisito de información RI-01.	57
4.2. Atributos del requisito de información RI-02.	58
4.3. Atributos del requisito de información RI-03.	58
4.4. Atributos del requisito de información RI-04.	59
4.5. Atributos del requisito de información RI-05.	60
4.6. Atributos del requisito de información RI-06.	62
4.7. Atributos del requisito de información RI-07.	63
4.8. Requisitos funcionales del sistema.	65

4.9. Requisitos no funcionales del sistema.	67
4.10. Reglas de negocio del dominio.	71
4.11. Restricciones del proyecto.	72
4.12. Especificación del caso de uso CU-01.	74
4.13. Especificación del caso de uso CU-02.	75
4.14. Especificación del caso de uso CU-03.	76
4.15. Especificación del caso de uso CU-04.	77
4.16. Especificación del caso de uso CU-05.	78
4.17. Matriz de trazabilidad entre requisitos funcionales y casos de uso. . .	78
4.18. Matriz de trazabilidad entre requisitos funcionales y no funcionales y sus verificaciones.	82
6.1. Bloques de configuración de FmgeneratorModel . Elaboración propia.	112
7.1. Casos superados y ejemplos por fichero de las dos suites de pruebas. .	130
7.2. Cobertura de sentencias del código de producción del generador. . .	131
7.3. Resumen de la suite automatizada de pruebas.	132
8.1. Validación sintáctica de los modelos generados para O1.	135
8.2. Comprobación SAT de la configuración booleana evaluada para O1. .	135
8.3. Pruebas técnicas del efecto de los parámetros de configuración. . . .	137
8.4. Pruebas técnicas de integración con Flamapy y UVLHub.	138
8.5. Perfiles de tamaño utilizados en la evaluación temporal.	139
8.6. Tiempo de preparación del entorno de ejecución.	140
8.7. Tiempo de generación de modelos UVL según el tamaño del modelo y el volumen del lote. Cada resultado corresponde a tres repeticiones. .	140
8.8. Evidencias asociadas a los objetivos formativos. Elaboración propia. .	142
C.1. Repositorios, commits de referencia y licencias declaradas.	177

ÍNDICE DE FIGURAS

2.1.	Representación de una línea de productos software basada en activos comunes. Extraído de [8].	13
2.2.	Ejemplo de modelo de características. Extraído de [3].	14
2.3.	Ejemplo gráfico de modelo de características. Elaboración propia. . .	15
2.4.	Representación de una misma fórmula mediante CNF y un BDD reducido. Elaboración propia.	17
2.5.	Arquitectura general de UVLHub y sus integraciones principales. Extraído de [17].	25
2.6.	Estructura de módulos de UVLHub. Extraído de [17].	26
4.1.	Diagrama general de casos de uso del sistema. Elaboración propia. . .	73
5.1.	Modelado conceptual del sistema. Elaboración propia.	85
5.2.	Diagrama de componentes del sistema. Elaboración propia.	90
5.3.	Diagrama de secuencia del flujo de configuración. Elaboración propia.	92
5.4.	Diagrama de secuencia de la generación ordinaria de modelos UVL en el navegador mediante Pyodide. Elaboración propia.	93
5.5.	Diagrama de secuencia de la generación de modelos booleanos satisficibles en el servidor mediante PySAT. Elaboración propia.	94
5.6.	Mapa de navegación del generador UVL. Elaboración propia.	97
5.7.	Flujo de datos de la generación ordinaria en Pyodide y de la generación satisficible mediante SAT en el servidor. Elaboración propia.	100
6.1.	Estructura del módulo generator de UVLHub. Elaboración propia. . .	106
6.2.	Estructura de la herramienta generadora. Elaboración propia.	107
6.3.	Plantillas HTML utilizadas para los pasos del asistente. Elaboración propia.	107
6.4.	Scripts utilizados para preparar y ejecutar el generador mediante Pyodide. Elaboración propia.	108
6.5.	Estructura del directorio docker de UVLHub. Elaboración propia. . .	108
6.6.	Suite de pruebas del módulo generator. Elaboración propia.	109
6.7.	Suite de pruebas de <i>fm_generator</i> . Elaboración propia.	109
6.8.	<i>Workflows</i> visibles en GitHub Actions del repositorio <i>upstream</i> de UVLHub. Captura facilitada por el autor.	123
8.1.	Configuración usada para verificar O1. Captura realizada por el autor.	134

B.1. Interfaz del paso Batch. Captura realizada por el autor.	160
B.2. Interfaz del paso Language levels. Captura realizada por el autor. . .	161
B.3. Configuración del árbol de características, primera vista. Captura realizada por el autor.	162
B.4. Configuración del árbol de características, segunda vista. Captura realizada por el autor.	163
B.5. Configuración de restricciones, primera vista. Captura realizada por el autor.	164
B.6. Configuración de restricciones, segunda vista. Captura realizada por el autor.	165
B.7. Configuración de atributos, primera vista. Captura realizada por el autor.	166
B.8. Configuración de atributos, segunda vista. Captura realizada por el autor.	167
B.9. Opciones de salida del generador. Captura realizada por el autor. . .	168
B.10. Salida del generador, segunda vista. Captura realizada por el autor. .	169
B.11. Salida del generador, tercera vista. Captura realizada por el autor. . .	170
B.12. Mapa de navegación del generador UVL. Elaboración propia.	170
B.13. Boceto inicial del paso 1. Elaboración propia.	171
B.14. Boceto inicial del paso 2. Elaboración propia.	172
B.15. Boceto inicial del paso 3. Elaboración propia.	173
B.16. Boceto inicial del paso 4. Elaboración propia.	174
B.17. Boceto inicial del paso 5. Elaboración propia.	175
B.18. Selector visual de distribuciones. Captura realizada por el autor. . . .	175
C.1. Salida de consola tras el arranque correcto del contenedor web. Elabo- ración propia.	179
D.1. Pantalla inicial de UVLHub. Captura realizada por el autor.	181
D.2. Mensaje mostrado cuando no se obtiene un modelo confirmado como satisfacible tras 20 intentos. Captura realizada por el autor.	183

LISTA DE ACRÓNIMOS

API	Interfaz de programación de aplicaciones (<i>Application Programming Interface</i>)
AST	Árbol de sintaxis abstracta (<i>Abstract Syntax Tree</i>)
BDD	Diagramas de decisión binaria (<i>Binary Decision Diagrams</i>)
CapEx	Gastos de capital (<i>Capital Expenditures</i>)
CI/CD	Integración continua y entrega o despliegue continuo (<i>Continuous Integration / Continuous Delivery or Deployment</i>)
CLI	Interfaz de línea de comandos (<i>Command-Line Interface</i>)
CNF	Forma normal conjuntiva (<i>Conjunctive Normal Form</i>)
CU	Caso de uso
DOI	Identificador de objeto digital (<i>Digital Object Identifier</i>)
ECTS	Sistema Europeo de Transferencia y Acumulación de Créditos (<i>European Credit Transfer and Accumulation System</i>)
EDT	Estructura de descomposición del trabajo
E2E	Extremo a extremo (<i>End-to-End</i>)
JSON	<i>JavaScript Object Notation</i>
LLM	Gran Modelo de Lenguaje (<i>Large Language Model</i>)
OpEx	Gastos operativos (<i>Operational Expenditures</i>)
PMI	Instituto de Gestión de Proyectos (<i>Project Management Institute</i>)
REST	<i>Representational State Transfer</i>
RF	Requisito funcional
RN	Requisito de negocio
RNF	Requisito no funcional
SAT	Problema de satisfacibilidad booleana (<i>Boolean Satisfiability Problem</i>)

SMT	Satisfacibilidad módulo teorías (<i>Satisfiability Modulo Theories</i>)
SPL	<i>Software Product Lines</i>
TASOVA	Red en nuevas Tendencias en Arquitectura Software y Variabilidad
UVL	<i>Universal Variability Language</i>
WASM	<i>WebAssembly</i>

1 INTRODUCCIÓN

1.1 Contexto

El desarrollo de software actual debe dar respuesta a sistemas cada vez más configurables, capaces de adaptarse a distintas necesidades, plataformas y contextos de uso. Dentro de este escenario, las Líneas de Productos Software (Software Product Lines, SPL) [1] ofrecen un enfoque para gestionar familias de productos relacionados a partir de los elementos que comparten y de aquellos que pueden variar entre unas configuraciones y otras. Esto permite tratar la variabilidad de forma explícita y facilita tareas como la configuración de productos, la reutilización de componentes o el análisis automático de las distintas combinaciones posibles.

Una de las formas más extendidas de representar esta variabilidad son los modelos de características (Feature Models) [2]. Estos modelos describen las características que puede incluir una familia de productos y las relaciones que existen entre ellas, permitiendo establecer qué elementos son obligatorios u opcionales, cuáles pueden combinarse y qué restricciones deben respetarse para obtener una configuración válida.

A medida que los modelos han ganado en complejidad, también ha surgido la necesidad de disponer de formatos que permitan representarlos de manera común. Universal Variability Language (UVL) [3] nace con este propósito y proporciona un lenguaje para describir modelos de variabilidad con diferentes grados de expresividad. Además de las relaciones habituales entre características, UVL permite incorporar construcciones como atributos, características tipadas y restricciones aritméticas o sobre cadenas, ampliando así los escenarios que pueden representarse y analizarse.

En torno a estos modelos se han desarrollado distintas herramientas destinadas a facilitar su procesamiento y análisis. Entre ellas se encuentra Flamapy [4], un *framework* desarrollado en Python que proporciona soporte para el análisis automático de modelos de características. Su arquitectura extensible permite incorporar nuevos formatos, operaciones y técnicas de análisis sin necesidad de modificar el núcleo de la herramienta, lo que facilita su evolución y su uso en diferentes líneas de investigación.

El presente Trabajo Fin de Grado se desarrolla dentro de este ecosistema y se enmarca en la actividad investigadora de DiversoLab y en TASOVA Plus (Red en nuevas Tendencias en Arquitectura Software y Variabilidad) [5]. Este entorno promueve el desarrollo de herramientas relacionadas con la ingeniería de líneas de productos software, la arquitectura software y el análisis de variabilidad, siguiendo además principios asociados al software libre y a la ciencia abierta (Open Science).

Otro de los componentes de este ecosistema es UVLHub [6], una plataforma orientada a la publicación, gestión y exploración de modelos expresados en UVL. La plataforma ofrece un espacio en el que investigadores y desarrolladores pueden compartir modelos y reutilizarlos en distintos experimentos o herramientas. Flamapy, UVL y UVLHub conforman así buena parte del entorno tecnológico sobre el que se desarrolla este trabajo.

1.2 Problema

La evolución de los lenguajes de modelado y de los motores de razonamiento ha permitido trabajar con formas de variabilidad cada vez más complejas. Sin embargo, disponer de herramientas capaces de procesar esa complejidad no resulta suficiente por sí solo: para poder evaluarlas también es necesario contar con modelos que hagan uso de esas capacidades.

Buena parte de los modelos de características disponibles se concentra en construcciones relativamente básicas [7]. Estos modelos siguen siendo útiles para numerosos análisis, pero resultan menos adecuados cuando se pretende experimentar con elementos más avanzados del lenguaje o estudiar cómo se comportan los motores de razonamiento ante distintos grados de complejidad. Se produce así una diferencia entre la expresividad que pueden ofrecer los lenguajes y herramientas actuales y la variedad de modelos disponibles para ponerlos a prueba.

Es por ello que se identifican de forma explícita las siguientes limitaciones:

1. **Expresividad limitada de los modelos de características disponibles:** Muchos de estos modelos se basan principalmente en relaciones estructurales básicas entre características. Aunque estos modelos permiten realizar análisis fundamentales, no siempre permiten representar escenarios más complejos donde intervienen elementos avanzados del lenguaje, lo que reduce la capacidad de explorar nuevas técnicas de razonamiento y análisis.
2. **Complicaciones a la hora de evaluar y mejorar motores de razonamiento avanzados sobre modelos más complejos:** Esto es consecuencia directa del problema anteriormente descrito. Sin estos modelos más avanzados, la capacidad de validar nuevas aproximaciones se ve limitada, por lo que es más complicado estudiar hasta qué punto estos motores pueden aprovechar construcciones más avanzadas del lenguaje de modelado.
3. **Necesidad de ampliar el conjunto de modelos disponibles para investigación en variabilidad software:** La investigación en este ámbito requiere disponer de modelos que permitan experimentar con diferentes escenarios, niveles de complejidad y características del lenguaje. Sin una base amplia y diversa de

modelos, resulta mucho más difícil comparar técnicas, evaluar resultados y avanzar en el desarrollo de herramientas más potentes.

1.3 Motivación

La disponibilidad de modelos de características con distintos niveles de expresividad resulta fundamental para continuar avanzando en el desarrollo y evaluación de técnicas de análisis automático de variabilidad software. Como se ha comentado anteriormente, la evolución de los lenguajes de modelado permite representar escenarios cada vez más complejos, pero esta capacidad debe estar acompañada de modelos que permitan estudiar y validar el comportamiento de las herramientas encargadas de procesarlos.

Abordar esta necesidad dentro de un entorno de investigación permite trabajar sobre problemas que aparecen de forma directa durante el desarrollo de nuevas técnicas y herramientas. En este sentido, el contexto proporcionado por TASOVA Plus y DiversoLab ofrece un marco adecuado para contribuir al avance de soluciones relacionadas con la ingeniería de líneas de productos software, la arquitectura software y el análisis de variabilidad, lo que además permite seguir principios asociados al software libre y la reutilización del conocimiento científico.

Desde una perspectiva académica y profesional, este Trabajo Fin de Grado representa también una oportunidad para aplicar los conocimientos adquiridos durante el grado en un proyecto software real. Durante su desarrollo, se ha trabajado sobre un ecosistema existente, lo que ha permitido abordar tareas habituales de ingeniería del software: comprender código previo, integrar componentes, documentar las decisiones adoptadas y verificar los cambios mediante pruebas automatizadas.

Además, la participación en un proyecto vinculado a investigación permite adquirir experiencia en un contexto diferente al de los desarrollos software tradicionales. La necesidad de analizar información técnica, consultar documentación especializada y tomar decisiones fundamentadas ha favorecido el desarrollo de habilidades relacionadas con la búsqueda autónoma de conocimiento y la resolución de problemas abiertos.

1.4 Punto de partida y contribución propia

Este Trabajo Fin de Grado se desarrolla sobre un conjunto de componentes y avances previos que forman parte del ecosistema Flamapy-UVLHub. Por este motivo, resulta necesario delimitar qué elementos constituyen la base sobre la que se construye el trabajo y cuáles corresponden a las contribuciones realizadas durante el desarrollo del presente proyecto.

En primer lugar, el trabajo de Sundermann et al. [7] constituye una referencia para este proyecto. Sus autores presentan UVLGenerator, un prototipo implementado para generar modelos con diferentes niveles de expresividad de UVL. El presente TFG toma como referencia su enfoque de generación, pero desarrolla una implementación propia en Python, organizada mediante un modelo de configuración y una operación compatibles con Flamapy. La implementación de la lógica de generación constituye la principal aportación técnica de este trabajo.

Además, el proyecto parte de la infraestructura existente del ecosistema Flamapy-UVLHub. UVLHub ya proporcionaba una plataforma para la gestión y manipulación de modelos UVL, así como un módulo **generator** sobre el que se ha realizado la integración del componente desarrollado.

La Tabla 1.1 distingue la procedencia de los componentes utilizados y las aportaciones realizadas durante este trabajo.

Elemento	Procedencia
Enfoque de generación de modelos UVL con expresividad avanzada	Propuesta y prototipo UVLGenerator presentados por Sundermann et al. [7]. Constituyen una referencia conceptual y técnica para este trabajo.
Ecosistema Flamapy y plataforma UVLHub	Infraestructura previa sobre la que se desarrolla el trabajo. En Flamapy participan diferentes grupos de investigación, entre ellos los pertenecientes a las Universidades de Sevilla, Málaga, Graz y la UNED. En cuanto a UVLHub, ha sido desarrollado principalmente por DiversoLab.
Generador de modelos de características compatible con Flamapy	Implementación propia desarrollada en este TFG en Python. Su aportación principal es la lógica de generación, organizada mediante un modelo de configuración y una operación, que permite producir jerarquías, atributos y restricciones a partir de los parámetros definidos por el usuario.
Integración del generador dentro del módulo <i>generator</i> de UVLHub	Desarrollo realizado durante este TFG, adaptando la aplicación existente para utilizar el nuevo componente generador independiente.
Nueva interfaz del asistente del generador	Aportación realizada por David Romero dentro del módulo <i>generator</i> de UVLHub. Incluye la ampliación del asistente de cinco a seis pasos, el panel lateral de seguimiento de la configuración actual y el componente <i>slider</i> para facilitar la definición de distribuciones.

Elemento	Procedencia
Refactorización de determinados elementos de UVLHub	Trabajo realizado por David Romero dentro del proyecto global, incluyendo la reorganización de algunos ficheros del módulo generador como <code>routes.py</code> , <code>wizard.py</code> y otros elementos relacionados. Posteriormente, durante este TFG, se han realizado nuevas modificaciones arquitectónicas necesarias para integrar el generador.
Batería de pruebas unitarias del módulo generador	La base de la suite fue desarrollada mayoritariamente por David Romero. En particular, los ficheros <code>test_engine.py</code> , <code>test_validators.py</code> , <code>test_wizard_outputs.py</code> y <code>test_wizard_flow.py</code> proceden de ese trabajo previo. Durante este TFG se desarrollaron las pruebas <code>test_selenium.py</code> , <code>test_wizard_persisters.py</code> , <code>test_wizard_state.py</code> , <code>test_wizard_builders.py</code> y <code>test_params_contract.py</code> , además de adaptar pruebas existentes a la nueva arquitectura del generador.
Generador basado en modelos de lenguaje de gran tamaño (<i>large language models</i> , LLM)	Funcionalidad desarrollada por David Romero dentro del ecosistema UVLHub. Comparte el módulo de la aplicación con el generador configurable, pero su desarrollo y evaluación quedan fuera del alcance de este Trabajo Fin de Grado.

Tabla 1.1: Procedencia de los principales elementos relacionados con el desarrollo.

Por tanto, la contribución principal de este Trabajo Fin de Grado se centra en la implementación de la lógica de funcionamiento de un generador configurable de modelos de características compatible con Flamapy y en su integración en UVLHub. En concreto, las principales aportaciones realizadas durante este trabajo son:

- La implementación de la lógica de generación de modelos de características a partir de la propuesta conceptual de Sundermann et al., incluyendo la construcción de jerarquías, atributos y restricciones según los parámetros y niveles de expresividad seleccionados.
- La definición del modelo de configuración y de la operación de generación compatibles con Flamapy, de forma que la lógica desarrollada pueda utilizarse como un componente independiente de UVLHub.

- La integración del nuevo componente con el módulo *generator* de UVLHub y la adaptación de la infraestructura existente para trabajar con esta nueva arquitectura.
- La validación del funcionamiento del sistema mediante pruebas automatizadas adaptadas a la implementación.

1.5 Objetivos

El objetivo principal de este trabajo es dar apoyo a la mejora de los motores de razonamiento existentes para el análisis de modelos de características mediante la obtención de una cantidad significativa y variada de modelos con expresividad avanzada. Para contribuir a este propósito, se plantea desarrollar una herramienta configurable para la generación automática y aleatoria de modelos de características, compatible con Flamapy y capaz de producir modelos UVL con diferentes niveles de expresividad. [7].

De forma más específica, se pueden desglosar los objetivos de este trabajo de la siguiente forma:

- 01.** Desarrollar un generador automático de modelos de características capaz de producir modelos UVL con diferentes niveles de complejidad y expresividad, ampliando la variedad de modelos disponibles para el análisis de variabilidad.
- 02.** Desarrollar un sistema de configuración flexible que permita modificar los parámetros principales del proceso de generación y adaptar los modelos obtenidos a distintos escenarios.
- 03.** Desarrollar una herramienta compatible con el ecosistema Flamapy e integrarla en UVLHub como solución para la generación configurable de modelos de características.
- 04.** Evaluar el comportamiento temporal del generador bajo escenarios de generación con un volumen elevado de modelos, analizando el coste asociado a la inicialización del entorno y al proceso de generación.
- 05.** Validar el funcionamiento del generador mediante una batería de pruebas automatizadas que permita comprobar la corrección de los modelos producidos y detectar posibles errores durante el proceso de generación.

De forma complementaria, el trabajo persigue una serie de objetivos formativos relacionados con la aplicación de conocimientos técnicos y metodológicos durante el desarrollo. Su cumplimiento se evaluará de forma cualitativa a partir de las evidencias

documentadas en la memoria, incluyendo las decisiones de diseño, la integración realizada, las pruebas desarrolladas y la planificación seguida.

- O6.** Reutilizar el conocimiento científico en el desarrollo de herramientas software (objetivo heredado de la red TASOVA Plus)
- O7.** Adquirir experiencia en el desarrollo y mantenimiento de proyectos software reales mediante la participación en un entorno de investigación y código abierto.
- O8.** Profundizar en el uso de tecnologías empleadas en el proyecto, especialmente Python, Flask y Docker, así como en la aplicación de pruebas automatizadas durante el desarrollo de un proyecto software integrado.
- O9.** Mejorar la capacidad de búsqueda, análisis y selección de información técnica relevante para resolver problemas dentro de un contexto de investigación.
- O10.** Aplicar buenas prácticas de planificación, gestión y documentación durante el desarrollo individual de un proyecto software de mayor duración y complejidad.

En la Tabla 1.2 se pueden conocer los criterios de éxito y los capítulos donde se verifican.

Obj.	Criterio de éxito (medible)	Cómo se verifica	Cap.
O1	El generador permite producir modelos UVL automáticamente incorporando diferentes niveles de complejidad y expresividad del lenguaje, ampliando la variedad de modelos disponibles para el análisis de variabilidad.	Generación de modelos con distintas configuraciones y lectura de los archivos producidos mediante UVLReader para comprobar su validez sintáctica. Comprobación SAT posterior de los diez modelos booleanos generados con la opción <i>Ensure satisfiable</i> desactivada.	6, 7, 8

Obj.	Criterio de éxito (medible)	Cómo se verifica	Cap.
O2	El generador dispone de un sistema de configuración que permite modificar los parámetros principales del proceso de generación y adaptar los modelos obtenidos a distintos escenarios.	Revisión de las opciones de configuración disponibles y ejecución de pruebas con diferentes configuraciones para comprobar su efecto sobre los modelos generados.	6, 7, 8
O3	La solución desarrollada proporciona una herramienta compatible con el ecosistema Flamapy y puede ser utilizada desde UVLHub como herramienta de generación de modelos de características.	Verificación de la compatibilidad con los componentes empleados de Flamapy y comprobación del flujo completo de generación desde UVLHub.	6, 7, 8
O4	El comportamiento temporal del generador queda caracterizado mediante mediciones de la carga inicial del entorno y de la generación de lotes de 100, 500 y 1000 modelos bajo tres perfiles de tamaño.	Medición repetida de la carga de Pyodide y de los tiempos de generación, calculando la media y la desviación típica para cada escenario.	8
O5	El generador cuenta con una batería de pruebas automatizadas que permita validar su correcto funcionamiento y detectar errores en los modelos producidos.	Ejecución de las pruebas automatizadas del componente generador y del módulo de UVLHub, incluyendo las pruebas de lectura de los modelos generados y de integración con Flamapy.	7
O6	El desarrollo reutiliza conocimiento científico relacionado con la generación de modelos de características y lo aplica en una herramienta software concreta.	Análisis del trabajo de Sundermann et al. y trazabilidad de su enfoque conceptual en el diseño e implementación del generador.	1, 2, 5, 6

Obj.	Criterio de éxito (medible)	Cómo se verifica	Cap.
O7	El desarrollo proporciona experiencia en la evolución de un proyecto software real, integrado en un ecosistema de investigación y código abierto.	Análisis de la integración del componente en Flamapy y UVLHub, del uso de Docker y de las pruebas ejecutadas sobre ambos proyectos.	3, 5, 6, 7, 8
O8	El desarrollo aplica Python, Flask, Docker y pruebas automatizadas en la construcción e integración del generador.	Revisión de la implementación del componente, de su integración en UVLHub y de la ejecución de las suites de pruebas.	5, 6, 7
O9	Las decisiones técnicas del proyecto se apoyan en la búsqueda, análisis y selección de información técnica y científica relevante.	Revisión del estado del arte, de las fuentes consultadas y de las decisiones justificadas en el diseño e implementación.	2, 5, 6
O10	El desarrollo individual del proyecto aplica prácticas de planificación, gestión de riesgos y documentación técnica.	Revisión de la planificación, de la gestión de riesgos, de la documentación elaborada y de la reflexión final sobre el desarrollo.	3, 9

Tabla 1.2: Criterios de éxito de los objetivos y capítulos donde se verifican.

1.6 Alcance

Este TFG comprende el desarrollo de un generador automático de modelos de características integrado dentro del ecosistema de herramientas asociado a Flamapy y orientado principalmente a su utilización desde UVLHub. La solución desarrollada permite generar modelos expresados en UVL mediante una configuración definida por el usuario, incorporando diferentes niveles de complejidad y expresividad con el objetivo de ampliar la disponibilidad de modelos para tareas de análisis e investigación en variabilidad software.

Dentro del alcance del proyecto se incluye el diseño e implementación del generador como herramienta compatible con Flamapy mediante un modelo de configuración y

una operación de generación compatibles con el *framework*. Asimismo, se contempla su integración dentro del módulo *generator* de UVLHub, permitiendo que los usuarios de la plataforma puedan interactuar con la funcionalidad desarrollada mediante una interfaz adaptada al flujo existente de la aplicación. Esta integración incluye la definición de los mecanismos necesarios para configurar el proceso de generación, la incorporación de soporte para diferentes construcciones del lenguaje UVL y la validación del funcionamiento del generador mediante pruebas automatizadas destinadas a comprobar la corrección de los modelos producidos.

En cuanto a lo que queda **fuera** del alcance del proyecto, se consideran los siguientes aspectos:

- La creación de una aplicación web independiente para el generador. Aunque la integración realizada en UVLHub incorpora una interfaz que permite configurar y ejecutar el proceso de generación, el generador como componente reutilizable no incluye una interfaz propia. En caso de utilizarse fuera de UVLHub, sería necesario desarrollar una capa de interacción adaptada al contexto concreto de uso.
- La modificación o ampliación de otros módulos de UVLHub distintos al módulo *generator*, ya que la intervención realizada se limita a la incorporación del generador dentro del módulo correspondiente.
- El desarrollo de nuevos motores de razonamiento o algoritmos de análisis de variabilidad. El objetivo del generador es proporcionar modelos con diferentes características y niveles de expresividad que puedan servir como base experimental para este tipo de herramientas, pero esto es algo que se pretende conseguir indirectamente con su uso.
- El desarrollo y la evaluación del generador basado en modelos de lenguaje de gran tamaño (LLM) existente en UVLHub. Esta funcionalidad pertenece a un desarrollo previo y no forma parte del generador configurable implementado en este TFG.

1.7 Estructura de la memoria

El resto de la memoria se organiza como sigue:

- El **Capítulo 2** presenta el **estado del arte** relacionado con la ingeniería de líneas de productos software, los modelos de características, el lenguaje UVL y las herramientas que forman parte del ecosistema en el que se desarrolla el trabajo. Además, se revisan trabajos relacionados con el objetivo de identificar las necesidades que justifican la propuesta desarrollada.

- El **Capítulo 3** describe la **planificación y gestión del proyecto**, incluyendo la metodología seguida durante el desarrollo, la organización de las tareas, la gestión de riesgos y los aspectos relacionados con la evolución del trabajo a lo largo del tiempo.
- El **Capítulo 4** define los **requisitos** que debe cumplir la solución desarrollada. En este capítulo se establecen las funcionalidades esperadas del generador, sus restricciones y la relación existente entre los requisitos y las pruebas destinadas a validarlos.
- El **Capítulo 5** presenta el **diseño de la solución** propuesta, describiendo la arquitectura general del sistema, la organización de sus componentes y las decisiones tomadas para integrar el generador dentro del ecosistema de Flamapy y UVLHub.
- El **Capítulo 6** detalla la **implementación** de la solución, incluyendo las tecnologías utilizadas, la estructura de la herramienta desarrollada, los mecanismos empleados para la generación de modelos y la integración realizada con la plataforma UVLHub.
- El **Capítulo 7** describe la estrategia de **pruebas** aplicada para verificar el correcto funcionamiento del sistema. En él se presentan las pruebas unitarias, de integración y de extremo a extremo (E2E) utilizadas para validar la generación de modelos y la integración de la funcionalidad desarrollada.
- El **Capítulo 8** analiza los **resultados** obtenidos durante la evaluación del trabajo. Se contrastan los resultados con los criterios de éxito definidos para los objetivos y se discuten las aportaciones realizadas frente al contexto presentado en capítulos anteriores.
- El **Capítulo 9** recoge las **conclusiones** del trabajo, valorando el grado de cumplimiento de los objetivos planteados, las limitaciones encontradas y las posibles líneas de trabajo futuro derivadas de la solución desarrollada.

Definidos el contexto, el problema, los objetivos y el alcance del trabajo, el siguiente capítulo establece los fundamentos teóricos y tecnológicos necesarios para comprender el entorno de variabilidad software sobre el que se desarrolla la propuesta.

2 ESTADO DEL ARTE

2.1 Ingeniería de Líneas de Productos Software (SPL)

2.1.1 Líneas de Productos Software

A lo largo de la historia, el desarrollo de software se ha basado tradicionalmente en la construcción de sistemas individuales orientados a resolver necesidades concretas, donde cada nuevo producto software suele desarrollarse como un proyecto independiente, incluso cuando comparte una parte significativa de sus funcionalidades, arquitectura o componentes con otros sistemas existentes. Esta estrategia puede resultar adecuada cuando el número de productos es reducido; sin embargo, hoy en día esto presenta limitaciones importantes para la gran mayoría de organizaciones, ya que surge de forma paralela la necesidad de adaptar un determinado sistema base a una gran cantidad de clientes con problemas ligeros o enormemente diferentes.

Debido a este hecho surge la Ingeniería de Líneas de Productos Software. Clements y Northrop definen las SPL de la siguiente forma: «*a set of software-intensive systems that share a common, managed set of features satisfying the specific needs of a particular market segment or mission and that are developed from a common set of core assets in a prescribed way*» [8]. Gracias a ellas es posible identificar, gestionar y reutilizar aquellos elementos comunes entre diferentes productos, a la vez que se mantienen explícitamente aquellos aspectos que pueden variar entre las distintas configuraciones posibles, haciendo así factible y mucho más viable la gestión de dichos elementos en lugar de construir cada producto desde cero.

La Figura 2.1 muestra una forma de entender este concepto.

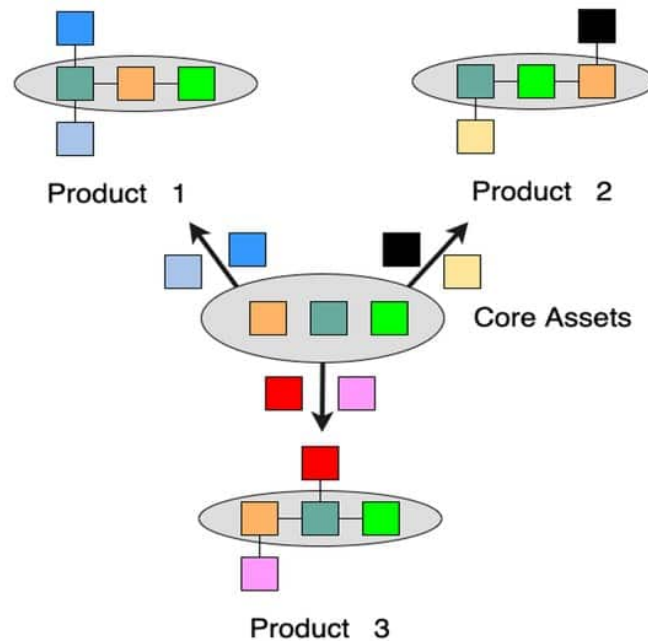


Figura 2.1: Representación de una línea de productos software basada en activos comunes. Extraído de [8].

La existencia de elementos comunes y diferencias entre los productos introduce el concepto de variabilidad, definido por van Gurp, Bosch y Svahnberg como «*Variability is the ability to change or customize a system*» [9]. Esta debe ser identificada, representada y gestionada durante el proceso de desarrollo, ya que permite definir qué elementos pueden formar parte de cada producto y bajo qué condiciones pueden combinarse.

La gestión de esta variabilidad requiere distinguir entre dos actividades complementarias. Por una parte, la ingeniería de dominio (*domain engineering*) se encarga de analizar un determinado dominio, identificar sus elementos comunes y variables, y construir los activos reutilizables que formarán la base de la línea de productos. Por otra parte, la ingeniería de aplicación (*application engineering*) utiliza dichos activos para derivar productos concretos mediante la selección de las características necesarias y la resolución de las variabilidades definidas.

No obstante, este paradigma introduce consigo retos relacionados con la representación y gestión de la complejidad. A medida que aumenta el número de características variables y configuraciones posibles, resulta necesario disponer de mecanismos que permitan gestionar esta variabilidad de forma íntegra y precisa, lo cual da surgimiento a técnicas como los modelos de características.

2.1.2 Modelos de características

Los modelos de características (*Feature Models*) son considerados uno de los mecanismos más utilizados para representar la variabilidad dentro de las líneas de productos software. Originalmente, fueron introducidos como parte del método FODA (*Feature-Oriented Domain Analysis*) por Kang et al., con el objetivo de describir las características comunes y variables existentes dentro de una familia de sistemas software [10]. Benavides et al. los describen de la siguiente forma: «*a feature model represents the information of all possible products of a software product line in terms of features and relationships among them*» [11]. Estos modelos permiten representar el espacio de configuraciones de una línea de productos mediante las características que pueden formar parte de cada producto y las relaciones existentes entre ellas.

Para entenderlo mejor, estos modelos se pueden representar mediante una estructura jerárquica en forma de árbol, donde cada nodo representa una característica del sistema. Las relaciones existentes entre características permiten establecer dependencias entre ellas, indicando qué elementos son obligatorios, opcionales o alternativos dentro de una configuración concreta. Además, pueden incorporar restricciones adicionales que expresan condiciones que no pueden representarse únicamente mediante la jerarquía del modelo. Véase la Figura 2.2.

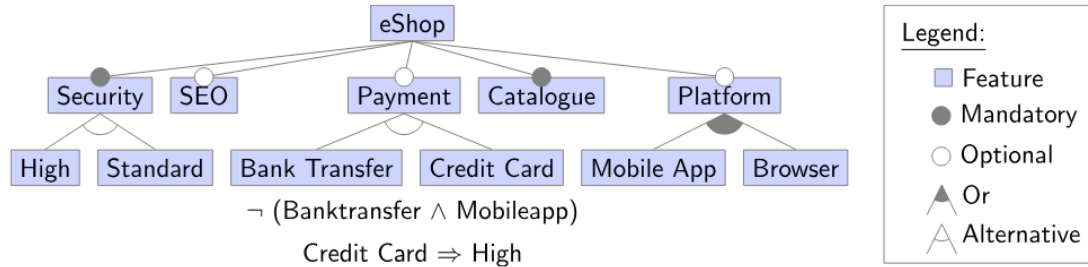


Figura 2.2: Ejemplo de modelo de características. Extraído de [3].

Aunque los modelos de características tradicionales permiten representar una gran cantidad de escenarios de variabilidad, inicialmente estaban orientados principalmente a expresar relaciones estructurales entre características. Con el aumento de la complejidad de los sistemas software surgió la necesidad de incorporar mecanismos adicionales, como atributos, restricciones aritméticas o diferentes tipos de dominios, que permitan representar escenarios más cercanos a los sistemas reales. A su vez, al incrementar la expresividad de los modelos, también se incrementa la complejidad de los mismos, lo cual hace necesaria la existencia de herramientas capaces de procesar estos modelos automáticamente y verificar propiedades sobre las configuraciones posibles.

2.1.3 Análisis automático y motores de razonamiento

Una vez definidos los modelos de características como mecanismo para representar la variabilidad de una SPL, surge la necesidad de disponer de técnicas capaces de analizar automáticamente la información contenida en dichos modelos. El análisis automático permite obtener información sobre las configuraciones representadas, comprobar propiedades del modelo y realizar operaciones de razonamiento sin necesidad de explorar manualmente cada una de las posibles variantes.

Los motores de razonamiento (*reasoning engines*) se encargan de analizar automáticamente los modelos de variabilidad. Entre otras operaciones, permiten comprobar si un modelo es satisfacible, detectar inconsistencias e identificar características que no pueden formar parte de ninguna configuración válida.

Por ejemplo, puede considerarse un modelo de características cuya raíz es *Primer Plato*. Las características *Pasta* y *Arroz* forman una relación alternativa. Esto significa que, en cada configuración válida, debe seleccionarse exactamente una de las características que forman parte de dicho grupo: puede elegirse *Pasta* o *Arroz*, pero no ambas a la vez. Por su parte, *Salsa* se define como una característica opcional de *Primer Plato*, por lo que puede seleccionarse o no junto con la alternativa elegida. La Figura 2.3 muestra esta estructura.

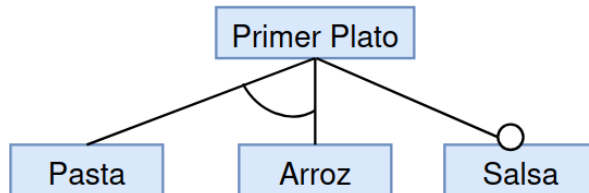


Figura 2.3: Ejemplo gráfico de modelo de características. Elaboración propia.

Si se añade como restricción global la expresión **Pasta & Arroz**, el modelo resulta insatisfacible, ya que la restricción exige seleccionar simultáneamente dos características que la relación alternativa impide seleccionar a la vez. La expresión puede escribirse como una restricción válida; sin embargo, no existe ninguna configuración que satisfaga al mismo tiempo la jerarquía del modelo y dicha restricción.

Una de las aproximaciones más utilizadas para realizar análisis automático sobre modelos de características consiste en transformar dichos modelos en fórmulas proposicionales que puedan ser procesadas mediante solucionadores SAT (*Boolean Satisfiability Problem*). El problema SAT consiste en determinar si existe una asignación de valores para un conjunto de variables booleanas que haga verdadera una fórmula lógica dada. En el contexto de las líneas de productos software, las características y restricciones de un modelo pueden representarse mediante variables booleanas y expresiones lógicas equivalentes, permitiendo utilizar solucionadores SAT para

responder consultas relacionadas con la validez de configuraciones o la existencia de productos derivados [11].

Para aplicar esta técnica, el modelo se traduce normalmente a una fórmula lógica equivalente que conserva las restricciones definidas entre sus elementos. Por ejemplo, una relación obligatoria entre dos características puede expresarse mediante una implicación lógica, mientras que las restricciones de exclusión entre características pueden representarse mediante cláusulas que impidan determinadas combinaciones. Una vez realizada esta transformación, el solucionador SAT puede determinar si existe una configuración válida o identificar aquellas asignaciones que cumplen las restricciones del modelo.

Además de la codificación SAT, los Diagramas de Decisión Binarios (*Binary Decision Diagrams*, BDD) permiten representar una función booleana mediante un grafo dirigido acíclico. Cada nodo interno corresponde a una variable de decisión y cada nodo terminal representa uno de los valores de la función. Esta representación permite codificar de forma compacta el espacio de configuraciones válidas y realizar distintas operaciones de análisis sobre él [12].

Para ilustrar ambas representaciones sobre un mismo ejemplo, considérese la fórmula $\varphi = (A \Rightarrow B) \wedge \neg(A \wedge C)$. La primera restricción establece que la selección de A requiere la de B , mientras que la segunda impide seleccionar simultáneamente A y C . Su forma normal conjuntiva es $\varphi \equiv (\neg A \vee B) \wedge (\neg A \vee \neg C)$.

La Figura 2.4 muestra la representación de esta misma fórmula mediante su CNF (Forma Normal Conjuntiva) para su análisis con SAT y mediante un BDD reducido. En el BDD, cada arista etiquetada con 0 o 1 representa la asignación de los valores falso y verdadero, respectivamente, a la variable del nodo de origen, mientras que los terminales 0 y 1 indican que la fórmula resulta falsa o verdadera. De esta forma, ambas partes de la figura utilizan exactamente las mismas restricciones y variables.

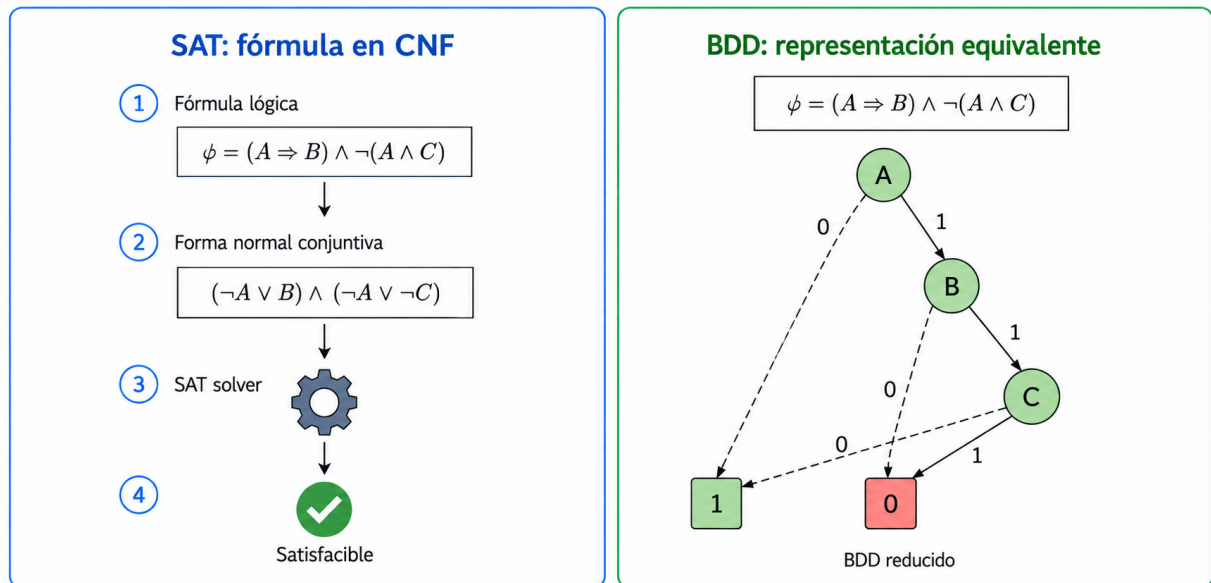


Figura 2.4: Representación de una misma fórmula mediante CNF y un BDD reducido. Elaboración propia.

La Tabla 2.1 compara SAT y BDD como técnicas de análisis automático. SAT permite resolver consultas de satisfacibilidad, mientras que un BDD representa la función booleana del modelo y puede reutilizarse para distintas operaciones.

Criterio	SAT	BDD
Representación del problema	Transformación del modelo de características a una fórmula lógica que debe ser evaluada por un solucionador SAT.	Construcción de una estructura gráfica que representa las posibles asignaciones de la función booleana.
Tipo de análisis	Orientado principalmente a responder consultas concretas sobre la existencia de configuraciones válidas.	Permite realizar análisis globales sobre el espacio completo de configuraciones representado.
Resultado obtenido	Determina si existe una asignación que satisface las restricciones y, en algunos casos, proporciona una configuración válida.	Proporciona una representación compacta del conjunto de configuraciones válidas.
Ventaja principal	Gran eficiencia para comprobar propiedades concretas del modelo y resolver problemas de satisfacibilidad.	Permite realizar múltiples operaciones de análisis sobre el modelo una vez construido el BDD.
Limitación principal	Requiere resolver consultas individuales y no mantiene necesariamente una representación completa del espacio de configuraciones.	La construcción del diagrama puede resultar costosa para modelos con elevada complejidad.

Tabla 2.1: Comparativa entre SAT y BDD como técnicas de análisis automático de modelos de características.

2.2 Universal Variability Language (UVL)

Universal Variability Language (UVL) es un lenguaje textual para representar modelos de características o *feature models*, así como sus restricciones de variabilidad [3, 13]. En este trabajo se utiliza como formato de salida del generador, por lo que esta sección se centra en las construcciones que el sistema puede producir y que posteriormente son procesadas por las herramientas del ecosistema Flamapy.

El Apéndice A.1.1 incluye ejemplos más extensos de modelo UVL completo. Los modelos presentados en esta sección también se han construido como archivos UVL completos y se han validado mediante el *parser* utilizado por Flamapy.

2.2.1 Estructura básica de un modelo UVL

Un archivo UVL generado por el sistema se compone principalmente de los bloques **include**, **features** y **constraints**. El bloque **include**, situado al comienzo del archivo cuando es necesario, declara los *minor levels* de UVL utilizados por el modelo. El bloque **features** define el árbol de características o *feature tree*, mientras que **constraints** recoge las restricciones que relacionan elementos de distintas ramas de dicho árbol.

Cada modelo tiene una única característica raíz o *root feature*. A partir de ella, las características se organizan mediante indentación y se agrupan según la relación de selección que exista entre una característica padre y sus características hijas. Los nombres que contienen espacios o caracteres especiales se escriben entre comillas dobles, y no deben confundirse con valores de tipo cadena de atributos o características de este tipo, que se escriben con comillas simples.

2.2.2 Relaciones y *Boolean level*

El *Boolean level* constituye la base de los modelos generados. Permite definir las relaciones estructurales de un *feature tree* mediante los grupos *mandatory*, *optional*, *alternative* y *or*.

Una característica *mandatory* debe seleccionarse siempre que se haya seleccionado su característica padre. En cambio, una característica *optional* puede seleccionarse o no cuando la característica padre forma parte de la configuración. Un grupo *alternative* exige seleccionar exactamente una de sus características hijas, mientras que un grupo *or* exige seleccionar al menos una de ellas y permite seleccionar varias simultáneamente.

Además de estas relaciones jerárquicas, UVL permite definir restricciones entre características pertenecientes a partes distintas del árbol. Estas restricciones se denominan *cross-tree constraints* y pueden emplear operadores de negación (!), conjunción (&), disyunción (|), implicación (=>) y equivalencia (<=>).

El generador también puede activar el *minor level* de *group cardinality*. Esta construcción limita el número de características hijas seleccionables dentro de un grupo mediante una expresión de la forma [mínimo..máximo]. Por ejemplo, una cardinalidad [0..2] permite seleccionar desde ninguna hasta dos características del grupo.

El Listado 2.1 muestra un modelo completo del *Boolean level*. La característica **Accessories** utiliza una *group cardinality* de **[0..2]**, por lo que pueden seleccionarse como máximo dos correas. La primera restricción entre árboles (*cross-tree constraint*) exige que el modelo incluya conexión mediante Wi-Fi o Bluetooth, mientras que la segunda establece que la resistencia al agua requiere disponer de pantalla.

```

1 include
2   Boolean.group-cardinality
3
4 features
5   Smartwatch
6     optional
7       GPS
8       Display
9       "Wi-Fi"
10      Bluetooth
11      "Water Resistance"
12      Accessories
13        [0..2]
14          "Leather Strap"
15          "Metal Strap"
16          "Silicone Strap"
17
18 constraints
19   "Wi-Fi" | Bluetooth
20   "Water Resistance" => Display

```

Código 2.1: Modelo UVL completo del *Boolean level* con *group cardinality*.

2.2.3 Niveles de expresividad avanzados

Los niveles de UVL permiten incorporar construcciones adicionales sobre el núcleo booleano. El *Arithmetic level* añade *attributes* numéricos y operaciones sobre sus valores, mientras que el *Type level* permite declarar características con valores de tipo **Integer**, **Real** o **String**. Cada nivel incluye las capacidades de los niveles anteriores.

2.2.3.1. *Arithmetic level*, *attributes* y niveles menores

Un *attribute* asocia un valor a una característica. En UVL se declara entre llaves después del nombre de dicha característica. Por ejemplo, **Standard {Capacity 300}** asigna el valor 300 al *attribute* **Capacity** de la característica **Standard**. Estos valores pueden utilizarse posteriormente en las restricciones del modelo.

El *Arithmetic level* permite aplicar operadores como $+$, $-$, $*$, $/$, $==$, $!=$, $<$ o $>=$ sobre *attributes* numéricos. El generador puede incorporar además los *minor levels* de *feature cardinality* y *aggregate functions*.

La *feature cardinality*, declarada mediante `Arithmetic.feature-cardinality`, permite indicar cuántas instancias de una característica pueden seleccionarse. Por su parte, las *aggregate functions* permiten operar sobre los valores de un mismo *attribute* presentes en las características seleccionadas. La función `sum()` calcula la suma de dichos valores y `avg()` calcula su media; por tanto, no cuentan características, sino que agregan los valores de sus *attributes*.

El Listado 2.2 combina estas construcciones en un modelo completo. **Storage** es un *attribute* asociado a los reproductores multimedia, mientras que **Capacity** se asocia a las opciones de batería. La primera restricción exige que la suma de los valores de almacenamiento sea 12 y la segunda establece que la capacidad media de las baterías disponibles debe ser, como mínimo, 300.

```

1 include
2   Arithmetic.feature-cardinality
3   Arithmetic.aggregate-function
4
5 features
6   Smartwatch
7     mandatory
8       Connectivity
9       or
10        Bluetooth
11        "Wi-Fi"
12   Battery
13     alternative
14       Standard {Capacity 300}
15       "Long Life" {Capacity 450}
16   optional
17     "Music Player" {Storage 2}
18     "Film Player" {Storage 10}
19     "Watch Faces" cardinality [5..20]
20
21 constraints
22   ("Music Player".Storage + "Film Player".Storage) == 12
23   avg(Capacity) >= 300

```

Código 2.2: Modelo UVL completo con *attributes*, *feature cardinality* y *aggregate functions*.

2.2.3.2. *Type level y string constraints*

El *Type level* amplía el modelo mediante características cuyo valor pertenece a un tipo determinado. UVL permite declarar características de tipo **Integer**, **Real** y **String**; el tipo se escribe antes del nombre de la característica. Estas declaraciones no deben confundirse con los *attributes*: una característica tipada representa directamente un valor configurable, mientras que un *attribute* añade información a una característica del modelo.

El *minor level* de *string constraints*, activado mediante **Type.string-constraints**, permite expresar restricciones sobre valores de tipo cadena. En particular, la función **len()** obtiene la longitud de una cadena, por lo que puede utilizarse para imponer un número concreto de caracteres o establecer límites sobre dicho valor.

El Listado 2.3 muestra un modelo completo con características de tipo **String** y **Real**. La restricción **len("Serial Number") == 10** exige que el valor del número de serie tenga exactamente diez caracteres. Las demás restricciones establecen que el peso sea mayor que 30 y que el color no pueda tomar el valor **'Pink'**.

```
1 include
2     Type.string-constraints
3
4 features
5     Smartwatch
6         mandatory
7             String Color
8             Real Weight
9             String "Serial Number"
10
11 constraints
12     len("Serial Number") == 10
13     Weight > 30.0
14     Color != 'Pink'
```

Código 2.3: Modelo UVL completo con características del *Type level* y *string constraints*.

Los tres listados anteriores se han validado mediante **UVLReader** de Flamapy antes de incorporarlos al documento. Esta comprobación confirma que **UVLReader** puede leer los archivos presentados, pero no demuestra por sí sola que los modelos sean satisfacibles; la evaluación de esa propiedad se aborda de forma independiente en el Capítulo 8.

2.3 Ecosistema de herramientas y aplicación en variabilidad software

A continuación se presentan las herramientas empleadas para gestionar y analizar modelos de características más allá de su simple representación, las cuales proporcionan capacidades para la creación, gestión y análisis automático de modelos, integrando diferentes técnicas de razonamiento y facilitando su adaptación a nuevos escenarios. Dentro de este ecosistema destaca Flamapy como *framework* orientado al análisis automático de modelos de características mediante una arquitectura extensible. Asimismo, destaca UVLHub como plataforma destinada a la gestión y compartición de modelos definidos en UVL.

2.3.1 Flamapy como *framework* de análisis de variabilidad

Flamapy es un *framework* desarrollado en Python orientado al análisis automático de modelos de características dentro del ámbito de la ingeniería de líneas de productos software [4, 14]. Su objetivo principal es proporcionar un entorno común para trabajar con modelos de variabilidad, facilitando tanto su procesamiento como la aplicación de diferentes técnicas de análisis automático. Para ello, Flamapy puede entenderse desde dos perspectivas complementarias: como una herramienta que permite ejecutar análisis sobre modelos de características de forma directa, y como un *framework* extensible que proporciona una arquitectura modular para incorporar nuevos modelos, operaciones y técnicas de análisis.

Desde la perspectiva de herramienta, Flamapy proporciona diferentes mecanismos para facilitar su utilización en distintos escenarios. Entre ellos se encuentra una interfaz de línea de comandos (CLI, *Command-Line Interface*) que permite ejecutar análisis directamente sobre modelos de variabilidad, así como una fachada de Python (*Python Facade*) que simplifica su integración dentro de otros desarrollos software. Además, ofrece interfaces adicionales como servicios REST (*Representational State Transfer*) mediante Swagger y soporte para ejecución en entornos web mediante WebAssembly (WASM) [15], permitiendo utilizar sus capacidades de análisis desde diferentes aplicaciones y contextos.

Como *framework*, Flamapy sigue una arquitectura modular basada en *plugins*, los cuales permiten extender sus capacidades sin modificar el núcleo principal del sistema. Esta arquitectura hace mucho más sencilla la incorporación de nuevos lenguajes de modelado, metamodelos o técnicas de análisis mediante componentes independientes que pueden integrarse dentro del ecosistema existente. Gracias a este enfoque, es posible adaptar Flamapy a diferentes necesidades relacionadas con la ingeniería de variabilidad, incorporando soporte para nuevos tipos de modelos o herramientas de razonamiento.

Otro de los elementos principales que ofrece Flamapy son las operaciones (*operations*), que representan las diferentes tareas de análisis que pueden ejecutarse sobre modelos de características. Cada operación define una funcionalidad concreta, como la comprobación de satisfacibilidad, la obtención de configuraciones válidas o el cálculo de propiedades del modelo, independientemente del formato en el que este haya sido definido. Esta separación permite reutilizar las mismas operaciones sobre diferentes tipos de modelos siempre que exista una representación compatible dentro del *framework*.

Por otra parte, las transformaciones (*transformations*) permiten convertir modelos entre diferentes representaciones dentro de Flamapy. Estas transformaciones actúan como mecanismo de conexión entre los distintos metamodelos soportados, permitiendo que los modelos puedan ser procesados por diferentes componentes del *framework* (o incluso no del *framework*) independientemente de su representación original. Por ejemplo, el componente *UVLWriter*, que es el que se ha utilizado para descargar los modelos creados por el generador diseñado, permite transformar un modelo de características almacenado internamente en Flamapy a su representación textual en formato UVL. Otras de las transformaciones existentes, por nombrar algunas, son *AFM Reader*, *AFM Writer*, *XML Reader*, *JSON Writer*, *SPLIT Writer* o *FM to BDD*. Todo esto facilita la interoperabilidad entre lenguajes de variabilidad, metamodelos y herramientas de análisis.

2.3.2 UVLHub como plataforma de gestión de modelos UVL

UVLHub es una plataforma orientada a la gestión de modelos de características definidos mediante el lenguaje UVL (*Universal Variability Language*) dentro del ámbito de la ingeniería de líneas de productos software [6, 16]. Tiene como objetivo proporcionar un repositorio especializado que permita almacenar, organizar, compartir y analizar modelos de variabilidad, facilitando su reutilización en entornos de investigación y desarrollo software, así como la colaboración entre usuarios.

La razón principal de su existencia se debe a la complicada localización de los modelos de características empleados por los usuarios, ya que estos los almacenaban en diferentes repositorios, páginas personales o proyectos independientes, utilizando además diferentes formatos de representación, dificultando su búsqueda, reutilización y análisis. UVLHub sigue los principios de ciencia abierta (*Open Science*), como accesibilidad, transparencia y colaboración, con la finalidad de compartir estos modelos de forma homogénea, haciendo más sencilla la colaboración entre usuarios.

Una de las características más relevantes de UVLHub es su capacidad de integración con otras herramientas y servicios relacionados con la gestión y análisis de modelos de variabilidad. Por una parte, la plataforma se integra con Zenodo, un repositorio abierto de investigación que permite almacenar los conjuntos de datos de forma

permanente mediante la generación de identificadores DOI (*Digital Object Identifier*), garantizando que los modelos publicados puedan seguir estando disponibles aunque la plataforma dejase de mantenerse en el futuro. Por otra parte, UVLHub incorpora las capacidades de análisis automático proporcionadas por Flamapy, permitiendo obtener información relevante sobre los modelos almacenados, como el número de características, la validez del modelo o el número de configuraciones posibles que pueden derivarse de ellos.

En la Figura 2.5 se puede ver una vista general sobre la arquitectura del sistema.

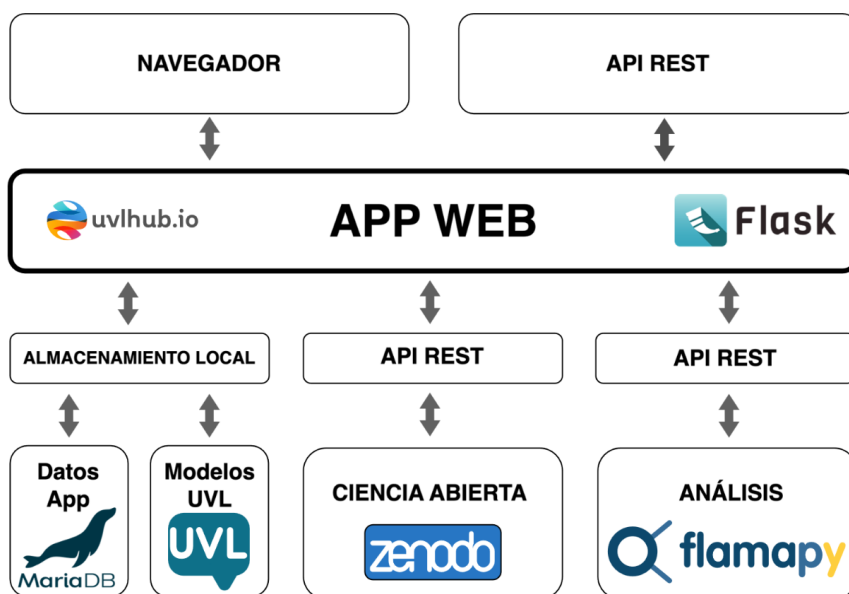


Figura 2.5: Arquitectura general de UVLHub y sus integraciones principales.
Extraído de [17].

Además de sus integraciones externas, UVLHub organiza sus funcionalidades (*features*) en módulos orientados a cubrir las principales necesidades relacionadas con la gestión de modelos UVL. Algunas de las más relevantes son:

- **dataset**: representa el núcleo principal de UVLHub, permitiendo gestionar el ciclo de vida de los conjuntos de modelos UVL. Esta funcionalidad permite realizar operaciones como la subida de datasets, su almacenamiento junto con los metadatos asociados, la descarga de los modelos y la resolución de identificadores DOI generados mediante la integración con Zenodo.
- **featuremodel**: permite gestionar la información asociada a los modelos de características almacenados dentro de un dataset. Esta funcionalidad define elementos como el modelo de características, sus metadatos y las métricas asociadas, estableciendo la relación entre los datasets y los archivos UVL almacenados.

- **explore**: proporciona mecanismos para explorar los datasets disponibles dentro de la plataforma mediante diferentes criterios de búsqueda, permitiendo localizar modelos de características publicados previamente.
- **flamapy**: integra las capacidades de análisis automático proporcionadas por Flamapy dentro de UVLHub. Esta integración permite realizar operaciones sobre los modelos UVL, como la validación del modelo o transformaciones hacia otros formatos soportados.
- **zenodo**: permite la comunicación con la interfaz de programación de aplicaciones (API, *Application Programming Interface*) REST de Zenodo para realizar la publicación y almacenamiento permanente de los datasets, proporcionando identificadores DOI asociados a los recursos compartidos.

La Figura 2.6 representa los módulos actuales de UVLHub en conjunto y cómo se complementan entre sí:

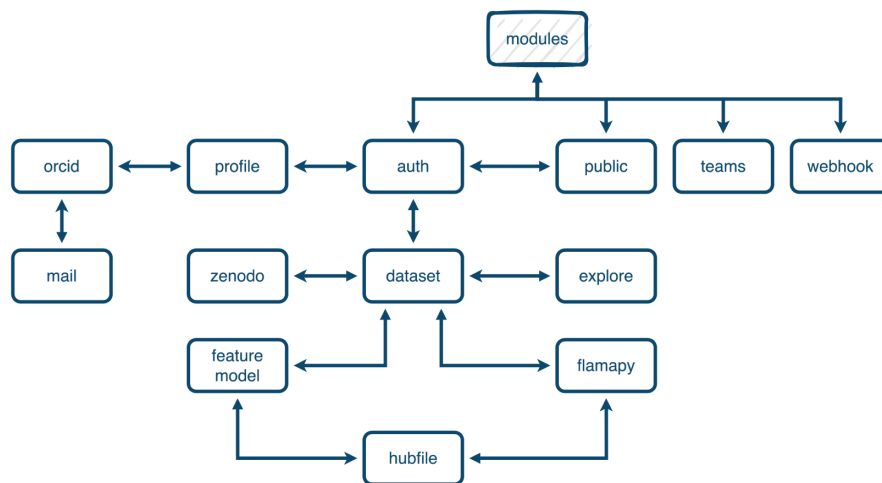


Figura 2.6: Estructura de módulos de UVLHub. Extraído de [17].

Estos módulos son originales de UVLHub. A ellos hay que añadirle también el módulo **generator**, del cual también se disponía antes de este trabajo.

2.3.3 Generación automática de modelos de características

Los modelos de características permiten representar la variabilidad existente dentro de una línea de productos software. Sin embargo, a medida que aumenta el número de características, restricciones y posibles configuraciones, la creación manual de estos modelos puede convertirse en una tarea costosa y difícil de gestionar.

Por este motivo, la generación automática de modelos de características se ha utilizado como mecanismo para producir modelos sintéticos destinados, entre otros usos, a la evaluación y prueba de herramientas de análisis de variabilidad [18, 19].

Estas técnicas permiten obtener conjuntos de modelos con diferentes tamaños, estructuras y grados de complejidad, facilitando la realización de experimentos sobre distintos escenarios de evaluación.

2.4 Trabajos relacionados

Dentro del ámbito de la ingeniería de líneas de productos software existen diferentes propuestas orientadas a la generación automática de modelos de características. Estas aproximaciones se emplean principalmente para crear modelos sintéticos destinados a evaluar herramientas de análisis de variabilidad, realizar experimentos controlados o estudiar la complejidad de técnicas de razonamiento.

La comparación se organiza según cuatro criterios: las construcciones de UVL que puede generar cada propuesta, el mayor tamaño de modelo documentado, la existencia de una garantía de satisfacibilidad y la validez sintáctica de la salida. Los dos últimos criterios permiten distinguir entre producir un modelo con una determinada estructura, garantizar que posee una configuración válida y comprobar que el artefacto resultante puede ser interpretado por un parser UVL. Además de Sundermann et al. [7], se consideran S.P.L.O.T., FeatureIDE, BeTTY y ETHOM, por representar enfoques con objetivos distintos dentro de la generación y evaluación de modelos de características.

2.4.1 Generating Feature Models with UVL's Full Expressiveness

El trabajo de Sundermann et al. [7] constituye la base conceptual del presente TFG, tal como se ha expuesto en la Sección 1.4. Los autores presentan un generador de modelos UVL que cubre todos los niveles de expresividad del lenguaje, incluyendo atributos, expresiones aritméticas, tipos, cadenas y funciones agregadas.

La propuesta permite configurar el número de modelos y distintos parámetros estructurales de la generación. También incorpora de forma opcional una garantía de satisfacibilidad módulo teorías (SMT, *Satisfiability Modulo Theories*), aplicable a las construcciones que soporta. El artículo muestra configuraciones de ejemplo con entre 800 y 1000 características, pero no establece un tamaño máximo evaluado ni informa de una validación de la salida mediante un parser UVL externo.

Por tanto, este TFG no pretende superar a dicho trabajo en expresividad del lenguaje. Su aportación se sitúa en la implementación e integración de un componente compatible con Flamapy dentro de UVLHub, así como en la validación de los modelos generados mediante **UVLReader**. La garantía de satisfacibilidad implementada en este trabajo se limita actualmente a modelos booleanos, por lo que constituye una limitación frente al enfoque SMT de Sundermann et al.

2.4.2 S.P.L.O.T.

S.P.L.O.T. (*Software Product Lines Online Tools*) es una plataforma web orientada al desarrollo y análisis de líneas de productos software que incluye un generador de modelos de características sintéticos [20]. Sus modelos se basan en estructuras jerárquicas y restricciones lógicas del nivel booleano.

En consecuencia, no contempla atributos tipados, expresiones aritméticas, restricciones sobre cadenas ni otras construcciones avanzadas de UVL. La documentación consultada no establece un tamaño máximo de modelo ni una opción para garantizar la satisfacibilidad de todas las salidas generadas. Además, su formato de salida no es UVL, por lo que no se documenta una validación sintáctica mediante un parser de dicho lenguaje.

2.4.3 FeatureIDE

FeatureIDE es un *framework* extensible para el desarrollo software basado en características, que proporciona herramientas para crear, editar y analizar modelos de características [21]. También incorpora un generador de modelos sintéticos integrado en su propio ecosistema.

En el contexto de los generadores comparados por Sundermann et al. [7], la generación de FeatureIDE se limita principalmente a construcciones booleanas. Por ello, no proporciona generación de atributos ni de restricciones propias de los niveles avanzados de UVL. No se ha identificado un tamaño máximo documentado, una garantía de satisfacibilidad de la salida ni una validación sintáctica mediante un parser UVL, puesto que los modelos se generan en el formato propio de la herramienta.

2.4.4 BeTTy

BeTTy (*Benchmarking and Testing on the Automated Analysis of Feature Models*) es un *framework* orientado al *benchmarking* y a la evaluación de herramientas de análisis automático de modelos de características [22]. Permite configurar parámetros de la generación para crear escenarios experimentales con diferente complejidad estructural y distinto número de restricciones.

BeTTy permite generar tanto modelos de características tradicionales como modelos con atributos. Además, documenta una opción para solicitar modelos satisfacibles en su generador de modelos clásico. Estas capacidades no equivalen a generar archivos UVL que incorporen todos los niveles del lenguaje: la documentación consultada no acredita la producción de características tipadas y restricciones sobre cadenas conforme a UVL, ni una validación de la salida mediante un parser de dicho lenguaje. Su aportación se centra en la configuración de modelos destinados a la evaluación de herramientas de análisis.

2.4.5 ETHOM

ETHOM (*Evolutionary algoRiTHm for Optimized feature Models*) es una propuesta basada en algoritmos evolutivos para generar modelos de características con propiedades que resultan exigentes para las herramientas de análisis [23]. A diferencia de los generadores aleatorios, formula la obtención del modelo como un problema de optimización y persigue, por ejemplo, maximizar el tiempo de análisis o el consumo de memoria. Su prototipo se desarrolló para integrarse en el entorno de BeTTY, por lo que aquí se estudia como un enfoque de generación evolutiva dentro de ese ecosistema [23].

ETHOM se incluye en esta revisión porque comparte la finalidad general de crear casos de evaluación para herramientas de variabilidad. Sin embargo, su objetivo no es generar modelos UVL con expresividad avanzada, sino modelos de características tradicionales difíciles de analizar. Por ello, los criterios de construcciones UVL y validez sintáctica UVL no son aplicables directamente; la propuesta tampoco documenta un límite de tamaño ni una garantía general de satisfacibilidad equivalente a la considerada en este trabajo.

2.4.6 Comparativa de trabajos relacionados

La Tabla 2.2 reúne las capacidades documentadas de cada propuesta y los resultados comprobados para el generador de este TFG.

Trabajo	Construcciones UVL	Tamaño, satisfacibilidad y salida
S.P.L.O.T. [20]	Booleanas.	Tamaño y satisfacibilidad no documentados. Formato propio, sin validación UVL documentada.
FeatureIDE [21]	Booleanas en su generador.	Tamaño y satisfacibilidad no documentados. Formato propio, sin validación UVL documentada.
BeTTY [22]	Modelos booleanos y modelos con atributos; cobertura completa de los niveles UVL no documentada.	Tamaño máximo no establecido en la fuente comparada. Opción documentada para solicitar modelos satisfacibles. Formatos distintos de UVL, sin validación UVL documentada.
ETHOM [23]	Modelos tradicionales; no orientado a UVL.	Tamaño y satisfacibilidad no documentados. La validez sintáctica UVL no es aplicable porque no genera salida UVL.
Sundermann et al. [7]	Todos los niveles de UVL.	Ejemplo de 800–1000 características, sin máximo declarado. Satisfacibilidad opcional mediante SMT. Genera UVL, sin validación externa mediante parser documentada.
Este TFG	Niveles booleano, aritmético y de tipos, con las construcciones y niveles menores implementados en el generador.	100 características por modelo evaluadas. Satisfacibilidad parcial: solo modelos booleanos mediante PySAT. Los 30 modelos evaluados fueron aceptados por <code>UVLReader</code> .

Tabla 2.2: Comparación de los trabajos relacionados según la expresividad, el tamaño documentado, la satisfacibilidad y la validez sintáctica de la salida. “No documentado” indica que la fuente consultada no proporciona esa evidencia.

Esta tabla muestra que S.P.L.O.T. y FeatureIDE se orientan principalmente a construcciones booleanas, mientras que BeTTY también permite generar modelos con atributos, aunque la tabla no documenta cobertura completa de los niveles de UVL. ETHOM ofrece un enfoque diferente, pues optimiza la dificultad de los modelos generados, aunque no persigue producir artefactos UVL ni ampliar la expresividad del lenguaje.

La propuesta de Sundermann et al. es la referencia más próxima, al cubrir todos los niveles de UVL y ofrecer una garantía opcional de satisfacibilidad. Frente a ella, este TFG aporta la integración del generador en Flamapy y UVLHub y

una comprobación explícita de la validez sintáctica de los modelos generados con **UVLReader**. No obstante, no se afirma una ventaja en tamaño máximo evaluado ni en garantía de satisfacibilidad, ya que esta última se limita a la configuración booleana actual.

El presente capítulo ha establecido los fundamentos teóricos y tecnológicos necesarios para comprender el contexto del trabajo, así como las principales soluciones existentes relacionadas con la generación automática de modelos de características. A partir de este análisis se define el marco sobre el que se desarrolla la propuesta presentada en este TFG. El siguiente capítulo describe la planificación seguida durante el proyecto, incluyendo la metodología, las fases de desarrollo, los recursos empleados y la gestión de riesgos.

3 PLANIFICACIÓN DEL PROYECTO

La planificación del proyecto toma como referencia las prácticas de dirección de proyectos del Project Management Institute (PMI). Se han adaptado al alcance de este TFG instrumentos como el acta de constitución, la estructura de descomposición del trabajo (EDT) y el seguimiento de plazos, riesgos y cambios [24].

3.1 Acta de constitución del proyecto

La Tabla 3.1 recoge el propósito, el alcance, las restricciones y el criterio de cierre definidos para el proyecto.

Elemento	Descripción
Propósito	Desarrollar una herramienta que permita generar automáticamente modelos de características en formato UVL para facilitar la creación de modelos de variabilidad software, proporcionando a investigadores y desarrolladores un mecanismo configurable para definir las características, restricciones y atributos de los modelos generados.
Objetivos	Los objetivos del proyecto son los definidos en la Sección 1.5. De forma general, el proyecto busca diseñar e implementar un generador automático de modelos de características, integrarlo en UVLHub y proporcionar soporte para diferentes niveles de expresividad del lenguaje UVL.
Alcance	El proyecto comprende el diseño e implementación del generador automático de modelos de características, su integración dentro de la plataforma UVLHub y la incorporación progresiva de soporte para modelos booleanos, aritméticos y con tipos definidos por UVL. Quedan fuera del alcance la modificación del funcionamiento interno de Flamapy y el desarrollo de nuevos algoritmos de análisis o razonamiento.

Acta de constitución del proyecto (continuación)

Elemento	Descripción
Supuestos	Se asume la disponibilidad del ecosistema software necesario para el desarrollo, incluyendo acceso al código fuente de UVLHub y Flamapy, así como las herramientas necesarias para la generación y análisis de modelos de variabilidad. También se considera disponible el entorno de desarrollo necesario para implementar, probar y validar la solución.
Restricciones	El proyecto está condicionado por las tecnologías y arquitectura existentes en UVLHub y Flamapy, especialmente el lenguaje Python (su <i>framework</i> Flask) y el estándar UVL. Además, debe desarrollarse antes del 7 de octubre, la fecha límite para la entrega de la memoria.
Entregables principales	Los principales entregables del proyecto son el código fuente del generador automático integrado en UVLHub y la memoria del TFG.
Criterio de cierre	El proyecto se considerará finalizado cuando el generador produzca modelos válidos para los niveles de expresividad incluidos en el alcance, se encuentre integrado en UVLHub y se hayan ejecutado pruebas de la generación y de su integración con el asistente existente.

Tabla 3.1: Acta de constitución del proyecto.

3.1.1 Interesados del proyecto

La Tabla 3.2 identifica a los participantes en el proyecto y resume su papel en el desarrollo o la evaluación del TFG.

Interesado	Rol	Interés / expectativa	Implicación
Autor del proyecto	Responsable del desarrollo	Completar el Trabajo Fin de Grado mediante el desarrollo de una solución funcional, documentada y alineada con los objetivos establecidos.	Diaria

Interesado	Rol	Interés / expectativa	Implicación
Tutores del TFG	Supervisión y validación académica	Garantizar el correcto desarrollo del proyecto, proporcionando orientación técnica y revisando los avances y entregables realizados.	Periódica
Grupo de investigación / ecosistema UVLHub-Flamapy	Entorno receptor de la solución	Disponer de una herramienta que permita generar modelos de características de forma automática y ampliar las capacidades existentes del ecosistema de variabilidad software.	Puntual
Tribunal del TFG	Evaluación final del proyecto	Valorar la calidad de la memoria, la defensa realizada y los resultados obtenidos durante el desarrollo del trabajo.	Al cierre

Tabla 3.2: Registro de interesados del proyecto.

3.2 Metodología

3.2.1 Metodología de desarrollo

Para el desarrollo del proyecto se ha seguido una metodología incremental e iterativa, adaptada a la naturaleza evolutiva del generador automático de modelos de características. Debido al carácter investigador y evolutivo del proyecto, la especificación inicial se refinó progresivamente durante el desarrollo conforme se analizaban las necesidades del generador y su integración dentro del ecosistema existente.

En una primera etapa se desarrolló una versión inicial del generador orientada a la creación de modelos de características booleanos, junto con su integración dentro de UVLHub mediante la interfaz gráfica proporcionada para configurar el proceso de generación. Una vez obtenida esta primera versión funcional, se realizaron ciclos sucesivos de implementación, validación y corrección que permitieron mejorar la estabilidad del sistema e incorporar nuevas funcionalidades pertinentes a los siguientes niveles de expresividad que se pretendió conseguir. Esta forma de hacerlo permitió validar cada nueva incorporación antes de continuar con las siguientes etapas y reducir el riesgo de introducir errores en funcionalidades previamente implementadas que se pudieran arrastrar a las siguientes.

Alternativas consideradas

No se optó por metodologías secuenciales como el modelo en cascada, ya que la naturaleza del proyecto requería realizar ajustes continuos conforme avanzaba el desarrollo y se identificaban nuevas necesidades relacionadas con la generación de modelos de características. Tampoco se aplicó una metodología Scrum completa, puesto que el desarrollo no se organizó mediante sprints, roles o hitos propios de este marco de trabajo.

Control de versiones

El control de versiones del proyecto se ha realizado mediante Git, utilizando GitHub como plataforma de almacenamiento y seguimiento de la evolución del código. Debido a la evolución del proyecto y a la necesidad de integrar el generador dentro de diferentes versiones de UVLHub, el desarrollo quedó distribuido entre varios repositorios y una solicitud de incorporación de cambios (*pull request*) en el repositorio oficial del proyecto.

Por este motivo, el repositorio `antjimort/uvlhub` no representa por sí solo todo el historial del Trabajo Fin de Grado. Dicho repositorio recoge principalmente la fase final de integración, refactorización, corrección y validación realizada entre mayo y septiembre de 2026. La actividad desarrollada entre mayo y julio corresponde a la fase principal de estabilización, mientras que entre el 14 y el 18 de septiembre se realizaron correcciones adicionales derivadas de la revisión técnica de los tutores. Véase la Tabla 3.3 sobre la trazabilidad de los repositorios usados durante el desarrollo.

Recurso	Periodo relevante	Papel dentro del desarrollo
<code>fm_generator</code>	15/05/2025– 21/04/2026	Repositorio inicial del componente generador. Contiene 38 commits realizados por el autor relacionados con la implementación de niveles UVL, atributos, restricciones, cardinalidades y opciones de configuración. Los commits anteriores a mayo de 2025 no se consideran parte de la contribución del TFG.
<code>uvlhub_v2</code>	14/02/2026– 03/04/2026	Repositorio transitorio utilizado durante la migración de los cambios previos a una nueva versión de UVLHub. En él se realizaron ajustes de migración, persistencia de parámetros del asistente y avances en los niveles booleano y aritmético del generador.
PR #117	06/04/2026– 19/04/2026	Solicitud de incorporación de cambios en el repositorio oficial de UVLHub, titulada <i>feat: boolean level ready</i> . La PR contiene 46 commits en total, de los cuales 45 fueron realizados por el autor, y fue fusionada en la rama <code>generator</code> el 19 de abril de 2026.
<code>uvlhub</code>	05/2026– 09/2026	Repositorio utilizado durante la fase final de integración, estabilización y consolidación técnica. Incluye refactorizaciones de la arquitectura, correcciones de compatibilidad, adaptación de pruebas, automatización de despliegue y correcciones finales relacionadas con la validación de modelos UVL, PySAT y la integración del generador como <i>plugin</i> de Flamapy.

Tabla 3.3: Trazabilidad de los repositorios y recursos utilizados durante el desarrollo.

Los repositorios y la solicitud de incorporación de cambios anteriores no deben interpretarse como desarrollos independientes que puedan sumarse entre sí, ya que parte de los commits realizados en el repositorio `fm_generator` fueron posteriormente incorporados en el PR #117. La Tabla 3.3 permite identificar el origen de cada fase del desarrollo sin duplicar la dedicación contabilizada en el cronograma.

La convención *Conventional Commits* [25] se adoptó progresivamente durante el proyecto. Por este motivo, algunos commits iniciales no siguen una nomenclatura

completamente normalizada. Sin embargo, en las fases posteriores se utilizaron prefijos como *feat* para la incorporación de nuevas funcionalidades, *fix* para la corrección de errores, *refactor* para reorganizaciones internas del código, *test* para cambios en las pruebas, *chore* para tareas de mantenimiento, *style* para ajustes de estilo y *ci* para modificaciones de integración continua.

Esta organización permite mantener la trazabilidad de la evolución completa del generador, desde la implementación inicial como componente independiente hasta su integración y estabilización dentro de UVLHub.

Herramientas de seguimiento

El seguimiento del desarrollo se ha realizado principalmente mediante el propio historial de GitHub, utilizando los commits realizados como registro de la evolución del proyecto y de las decisiones técnicas tomadas durante la implementación.

Además, las reuniones periódicas con los tutores han permitido revisar el estado del desarrollo para analizar las deficiencias encontradas con el propósito de corregirlas y mejorar, por tanto, la calidad del generador. Los detalles de estas reuniones se detallarán en el siguiente apartado.

3.2.2 Metodología de validación y evaluación

La validación del proyecto se ha realizado mediante un proceso iterativo durante el desarrollo, combinando pruebas funcionales, pruebas automatizadas y revisiones periódicas con los tutores del proyecto.

Tras la implementación de cada nueva funcionalidad, se realizaron comprobaciones manuales para verificar que el generador producía modelos de características correctos según los parámetros configurados, incluyendo los diferentes niveles de expresividad soportados por UVL, así como la correcta interacción del usuario con el asistente de generación de UVLHub. Además, se ejecutaron pruebas automatizadas para detectar errores y regresiones, incorporando casos que verificasen el comportamiento esperado o reprodujesen los fallos relevantes detectados durante el desarrollo.

La evaluación del cumplimiento del proyecto se ha realizado siguiendo los criterios de éxito definidos en la Tabla 1.2. Estos criterios permiten comprobar de forma objetiva aspectos como la generación automática de modelos UVL, la configuración del proceso de generación, el soporte de construcciones avanzadas del lenguaje, la integración dentro del ecosistema Flamapy y UVLHub, y la existencia de una batería de pruebas automatizadas.

Además de la validación técnica continua, se realizaron revisiones del estado del proyecto junto con los tutores. Cuando se alcanzó aproximadamente el 80 % de desarrollo, se realizó una primera revisión en la que se concluyó que la solución era

funcionalmente válida, pero presentaba ciertas deficiencias técnicas que debían ser corregidas antes de considerarla finalizada. Entre ellas se identificaron problemas relacionados con la integración del generador dentro de UVLHub, la pérdida de algunas funcionalidades existentes durante determinadas modificaciones (que posteriormente se recuperaron sin problema), la reducción de la cobertura de pruebas, la presencia de elementos no deseados en el repositorio y aspectos de calidad del código (documentación y limpieza de la implementación).

Tras aplicar las primeras correcciones, se realizaron nuevas revisiones con los tutores. La versión principal obtuvo una valoración positiva en julio de 2026. Una revisión técnica posterior identificó ajustes adicionales relacionados con la integración con Flamapy, la validez de los modelos generados y las pruebas; estas observaciones dieron lugar a las correcciones efectuadas durante septiembre de 2026.

3.3 Estructura de descomposición del trabajo (EDT)

La Tabla 3.4 presenta los paquetes de trabajo y sus entregables verificables.

ID	Paquete de trabajo	Entregable verificable
1	Gestión y planificación del proyecto	Planificación inicial del proyecto, estudio de la solución propuesta por los investigadores a partir del trabajo de referencia y seguimiento del desarrollo realizado.
1.1	Planificación inicial y estudio de la solución	Acta de constitución, definición del alcance, planificación temporal del proyecto y análisis de la solución propuesta en el artículo de referencia.
1.2	Seguimiento y control	Reuniones de seguimiento, revisión del estado del desarrollo y gestión de los cambios realizados durante el proyecto.
2	Análisis y diseño de la solución	Diseño de la arquitectura del generador y definición de los componentes necesarios para su implementación.
2.1	Análisis de requisitos	Catálogo de requisitos funcionales y técnicos del generador automático de modelos.
2.2	Diseño del generador y su integración	Arquitectura definida para la generación de modelos, configuración del sistema e integración con UVLHub.
3	Desarrollo del generador automático	Implementación del generador capaz de producir modelos de características en formato UVL.

ID	Paquete de trabajo	Entregable verificable
3.1	Implementación de niveles de expresividad	Soporte implementado para generación de modelos booleanos, aritméticos y con tipos definidos por UVL.
3.2	Configuración e integración con el asistente	Sistema de configuración del generador y adaptación de su integración al asistente existente de UVLHub.
4	Pruebas y validación	Suite de pruebas automatizadas y validación funcional del generador integrado.
5	Documentación y defensa	Memoria final del Trabajo Fin de Grado y material necesario para la presentación de defensa.

Tabla 3.4: Estructura de descomposición del trabajo (EDT).

3.4 Cronograma e hitos (línea base)

Teniendo como fecha de inicio del proyecto el 12 de abril de 2025, el cronograma real quedó dividido en varias etapas de desarrollo, integración y validación. La Tabla 3.5 recoge los periodos de actividad identificables a partir del historial de los repositorios y de los hitos de integración del proyecto.

Periodo real	Fin prevista	EDT	Actividad principal	Hito o resultado
04/2025	04/2025	1.1	Análisis inicial y planificación del proyecto.	Planificación inicial y estudio de viabilidad del generador.
04/2025	04/2025	2.1–2.2	Análisis de requisitos y diseño de la solución.	Requisitos definidos y prototipo inicial preparado para validar la integración.
05–07/2025	05/2026	3	Primera iteración del generador y de su integración inicial con UVLHub.	Implementación de restricciones booleanas, cardinalidades, atributos manuales, configuración inicial del asistente e integración preliminar del componente.
08/2025–01/2026	—	—	Periodo de inactividad del proyecto.	No se realizaron actividades ni se establecieron hitos durante este intervalo.
02–04/2026	05/2026	3	Reanudación del desarrollo, migración a UVLHub 2.0 e incorporación de nuevos niveles de expresividad.	Migración funcional, soporte del nivel booleano, funciones agregadas e integración mediante el PR #117, fusionado el 19 de abril de 2026.
05–07/2026	05/2026	3–4	Integración final, refactorización, adaptación de pruebas y validación del sistema.	Generador estabilizado, pruebas adaptadas a la nueva arquitectura y validación final de los tutores.
14–26/09/2026	—	3–4	Correcciones técnicas y ampliación de las comprobaciones posteriores a la fase principal de validación.	Corrección de la generación de tipos y expresiones, validación de modelos UVL con <code>UWLReader</code> y <code>PySAT</code> , integración del generador como <i>plugin</i> de <code>Flamapy</code> , reorganización de las pruebas unitarias y limpieza de elementos no deseados del repositorio.
08–10/2026	—	5	Redacción, revisión y preparación de la memoria y sus apéndices.	Documento preparado para su entrega y defensa.

La distribución del esfuerzo se mantiene agrupada por paquetes de trabajo, ya que las horas reales se registraron por fases generales y no de forma independiente para cada una de las iteraciones temporales anteriores. Esta decisión evita asignar artificialmente horas concretas a periodos para los que no existe un registro horario detallado.

Las fases de desarrollo y su esfuerzo asociado se recogen en la Tabla 3.6.

#	Fase	EDT	Obj.	Est. (h)	Real (h)	Hito / entregable
1	Análisis inicial y planificación del proyecto	1.1	O1	12	12	Prototipo inicial y planificación del desarrollo validada.
2	Análisis y diseño de la solución	2.1–2.2	O1–O2	25	25	Requisitos definidos, análisis de la solución propuesta y prototipo inicial desarrollado para validar la viabilidad de la integración.
3	Desarrollo e integración del generador automático	3	O1–O4	593	703	Generador implementado e integrado dentro de UVLHub con soporte para los niveles de expresividad definidos y corrección de errores detectados durante el desarrollo.
4	Pruebas y validación de la solución	4	O1, O4 y O5	120	170	Suite de pruebas automatizadas, validaciones funcionales y revisiones finales del sistema completadas.
Total				750	910	

Tabla 3.6: Fases de desarrollo y esfuerzo asociado: línea base frente a valores reales.

La fase principal de desarrollo e integración concluyó en julio de 2026. Posteriormente, durante septiembre, se realizaron correcciones y pruebas adicionales derivadas de la revisión técnica de los tutores. Estas actividades continuaron, al menos, hasta el

26 de septiembre de 2026, fecha de la ejecución completa de las pruebas del módulo generador documentada en este trabajo.

La línea base preveía finalizar las fases principales en mayo de 2026. La finalización de la fase principal en julio supuso una desviación de dos meses. La continuación de las correcciones técnicas durante septiembre amplió a cuatro meses la desviación respecto a esa previsión.

En términos de esfuerzo, las 910 horas reales de desarrollo superaron en 160 horas las 750 horas inicialmente estimadas, lo que representa una desviación del 21,3 %. El esfuerzo adicional se concentró en las fases de desarrollo e integración del generador y de pruebas y validación, debido a los problemas de compatibilidad con dependencias externas, la necesidad de realizar refactorizaciones, la adaptación de las pruebas y las correcciones finales realizadas en septiembre tras la revisión de los tutores.

De las 910 horas de desarrollo, 400 corresponden al contrato de apoyo técnico asociado a TASOVA Plus y las 510 restantes son atribuibles al desarrollo del TFG. Junto con las 70 horas dedicadas a la redacción, maquetación y revisión de la memoria, las cuales se pueden ver en la Tabla 3.7, la dedicación atribuible al TFG asciende a 580 horas.

#	Capítulos	EDT	Est.	Real	Hito / Entregable
1	Caps. 1–3: introducción, estado del arte y planificación	5	30 h	33 h	Borrador de contexto y plan aprobado por los tutores.
2	Caps. 4–7: núcleo técnico (requisitos, diseño, implementación y pruebas)	5	25 h	24 h	Núcleo técnico redactado y trazable con el repositorio.
3	Caps. 8–9: resultados y conclusiones	5	15 h	13 h	Memoria completa lista para revisión final y depósito.
Total			70 h	70 h	

Tabla 3.7: Fases de redacción de la memoria (estimadas vs. reales).

Las 70 horas reales indicadas en la Tabla 3.7 corresponden al trabajo completo de elaboración de la memoria, incluyendo la redacción y revisión de los capítulos, la organización de referencias, la preparación de tablas y figuras, la maquetación en L^AT_EX, la revisión de referencias cruzadas y la preparación de los apéndices.

La distribución de 33, 24 y 13 horas corresponde a bloques de capítulos y no a una estimación basada únicamente en su extensión. Las tareas técnicas necesarias para obtener resultados, ejecutar pruebas y recopilar evidencias se contabilizaron

dentro de las fases de desarrollo y validación, evitando sumar dos veces el mismo esfuerzo.

En la Tabla 3.8 se puede encontrar el resumen global en horas del proyecto.

# Fase	Est.	Real
Desarrollo (Tabla 3.6)	750 h	910 h
Memoria (Tabla 3.7)	70 h	70 h
Total	820 h	980 h

Tabla 3.8: Resumen global del proyecto (estimadas vs. reales).

El total de 980 horas representa la dedicación real acumulada durante el proyecto, pero no constituye íntegramente la carga académica del TFG. De este total, 400 horas corresponden al contrato de apoyo técnico asociado a TASOVA Plus y no forman parte de la dedicación académica del TFG. Las 580 horas restantes sí son atribuibles al TFG, incluyendo 510 horas de desarrollo y 70 horas dedicadas a la redacción, maquetación y revisión de la memoria.

Como referencia de carga académica, este TFG tiene 12 créditos del Sistema Europeo de Transferencia y Acumulación de Créditos (ECTS), lo que equivale a una estimación de entre 300 y 360 horas de trabajo del estudiante [26, 27]. La comparación debe realizarse con las 580 horas atribuibles al TFG, sin incluir las 400 horas correspondientes al contrato de apoyo técnico. La dedicación registrada supera esa referencia debido a la integración con UVLHub y Flamapy, la adaptación de las pruebas y las correcciones derivadas de las revisiones técnicas.

3.5 Plan de comunicaciones y seguimiento

La Tabla 3.9 resume los canales utilizados para el seguimiento del proyecto y la comunicación con los tutores.

Actividad	Cadencia	Vía	Resultado esperado
Reuniones con los tutores	Según avance del proyecto	Presencial	Revisión del estado del desarrollo, validación de avances, detección de posibles deficiencias y definición de las próximas tareas a realizar.

Actividad	Cadencia	Vía	Resultado esperado
Mensajería asíncro- na	Bajo de- manda	Telegram y co- rreo electrónico	Resolución de dudas sobre el diseño y desarrollo del siste- ma, recepción de indicaciones adicionales, concertación de tutorías y evaluación de avan- ces menores.
Seguimiento del de- sarrollo	Continua	Repositorio Git/GitHub	Trazabilidad de la evolución del proyecto mediante el his- torial de cambios y commits realizados durante la imple- mentación.

Tabla 3.9: Plan de comunicaciones y seguimiento del proyecto.

Durante el desarrollo se mantuvieron reuniones presenciales con los tutores para revisar el estado de la solución y validar las decisiones tomadas. La primera revisión relevante tuvo lugar el 5 de mayo de 2026, cuando se había alcanzado aproximadamente el 80 % del desarrollo. En ella se consideró que la solución era funcionalmente aceptable, aunque presentaba deficiencias técnicas que debían corregirse.

Tras aplicar las primeras correcciones, el 29 de junio de 2026 se realizó una segunda revisión y el 10 de julio se valoró positivamente la versión principal. Posteriormente, la revisión técnica de los tutores identificó nuevos aspectos relativos a la integración con Flamapy, la validación de los modelos generados y la organización de las pruebas. Estas observaciones motivaron correcciones que continuaron durante septiembre de 2026.

3.6 Gestión de riesgos

La Tabla 3.10 recoge los riesgos identificados al inicio del desarrollo. La exposición se ha valorado cualitativamente a partir de la probabilidad y el impacto: se considera alta cuando el impacto es alto y la probabilidad al menos media, o cuando la probabilidad es alta y el impacto al menos medio.

ID	Riesgo	Prob.	Impacto	Exposición	Mitigación
R1	Complejidad de integración del generador dentro del ecosistema UVLHub-Flamapy	Alta	Alto	Alta	Realizar un estudio inicial de la arquitectura existente, validar progresivamente los cambios realizados y comprobar la compatibilidad entre los diferentes componentes del sistema.
R2	Problemas derivados de dependencias externas y configuración del entorno de ejecución	Media	Alto	Alta	Mantener actualizadas las dependencias necesarias, documentar la configuración del entorno y realizar pruebas de integración tras modificar versiones o paquetes externos.
R3	Errores en la generación de modelos tras la incorporación de nuevas funcionalidades	Alta	Medio	Alta	Realizar validaciones funcionales tras cada modificación e incorporar pruebas automatizadas que permitan detectar errores en las funcionalidades implementadas.
R4	Subestimación del esfuerzo necesario para completar la implementación y validación del sistema	Media	Alto	Alta	Realizar un seguimiento periódico del progreso, priorizar las tareas críticas y reservar tiempo adicional para corrección de errores y mejora de las pruebas.

Tabla 3.10: Riesgos identificados, exposición y planes de mitigación.

3.6.1 Seguimiento de riesgos al cierre

La Tabla 3.11 indica cuáles de esos riesgos se materializaron, qué respuesta se aplicó y qué limitaciones permanecieron al finalizar la evaluación.

ID	¿Se materializó?	Respuesta aplicada	Riesgo residual
R1	Sí	El riesgo apareció principalmente durante la fase inicial de comprensión de Flamapy y durante la integración del generador dentro de UVLHub. Para solucionarlo se estudió la arquitectura de Flamapy, se realizaron modificaciones sobre la estructura del código y se aplicó una estrategia de pruebas progresivas para validar cada incorporación realizada.	La integración se verificó mediante las pruebas de ejecución, serialización y generación desde UVLHub descritas en el Capítulo 7. Futuras modificaciones de la arquitectura pueden requerir nuevas comprobaciones.
R2	Sí	Se produjeron problemas relacionados con las dependencias externas del proyecto, especialmente con la actualización de Flamapy, la creación y configuración de los wheels necesarios para Pyodide [28] y los problemas encontrados para que estos paquetes fueran localizados y utilizados correctamente. La solución consistió en actualizar dependencias, crear nuevos wheels cuando fue necesario e investigar la configuración requerida para su correcta integración.	El funcionamiento se comprobó con las versiones de dependencias utilizadas durante la evaluación. Sus futuras actualizaciones pueden requerir ajustes de compatibilidad.

ID	¿Se materializó?	Respuesta aplicada	Riesgo residual
R3	Sí	Durante el desarrollo se detectaron en varias ocasiones problemas en la generación de modelos debido a que determinadas modificaciones no producían los resultados esperados. Para solucionarlos se comprobaba inicialmente el fallo mediante la generación de diferentes modelos, posteriormente se analizaba la causa, se corregía la implementación y se realizaban nuevas validaciones manuales, actualizando las pruebas unitarias cuando aportaban valor.	La generación se verificó mediante las pruebas automatizadas y los escenarios de evaluación descritos en los Capítulos 7 y 8. Pueden existir casos límite no cubiertos por esos escenarios.
R4	Sí	La estimación inicial del esfuerzo necesario se vio superada debido a diversos factores relacionados con la calidad final del producto, como los errores detectados durante la implementación, la necesidad de mejorar el enfoque de las pruebas, los problemas de integración y las tareas de refactorización necesarias. La gestión de este riesgo se realizó priorizando las correcciones necesarias, reorganizando tareas y utilizando las revisiones con los tutores para obtener una valoración sobre el estado del proyecto.	El principal riesgo residual está relacionado con la posible existencia de deuda técnica pendiente. Una mayor reducción de esta deuda mediante refactorizaciones y mejoras arquitectónicas podría facilitar futuras ampliaciones y reducir el esfuerzo necesario para incorporarlas.

ID	¿Se materializó?	Respuesta aplicada	Riesgo residual
----	------------------	--------------------	-----------------

Tabla 3.11: Seguimiento de riesgos al cierre: materialización, respuesta y residuales.

3.7 Control de cambios

La Tabla 3.12 registra los cambios principales introducidos durante el desarrollo y su relación con las revisiones del proyecto.

ID	Fecha	Cambio	Justificación y resultado
C1	12/05/2025	Ampliación de las opciones de configuración del generador para aumentar la flexibilidad del proceso de generación.	Causa: Necesidad de proporcionar una mayor capacidad de adaptación de los modelos generados a distintos escenarios de uso. Impacto: Incremento del alcance funcional mediante la incorporación de nuevas opciones de configuración relacionadas con la generación de operadores y atributos. Aprobado por: Tutores / investigadores
C2	Durante el desarrollo	Ampliación del plazo previsto para completar la implementación y validación del generador automático.	Causa: La estimación inicial no contemplaba completamente la complejidad de la integración con el ecosistema UVLHub-Flamapy, así como los problemas surgidos durante la implementación y corrección de errores. Impacto: Desplazamiento del hito previsto de finalización del desarrollo y aumento del esfuerzo dedicado a la implementación y validación del sistema. Aprobado por: Tutores

ID	Fecha	Cambio	Justificación y resultado
C3	05/05/2026	Revisión del enfoque técnico y refactorización de la estructura interna del generador para mejorar su integración y mantenibilidad.	Causa: Durante la revisión del estado del proyecto se identificaron deficiencias técnicas relacionadas con la organización interna del código y la integración del generador dentro del ecosistema existente. Impacto: Necesidad de reorganizar componentes internos, mejorar la arquitectura de la solución y adaptar parte de la implementación existente. Aprobado por: Tutores, reunión de seguimiento 05/05/2026
C4	05/2026	Adaptación de la integración del generador al nuevo asistente de UVLHub, desarrollado previamente por David Romero.	Causa: La evolución del asistente existente modificó el flujo de configuración de la generación. Impacto: Adaptación del intercambio de parámetros y de la ejecución del generador al asistente de seis pasos. Seguimiento: Revisado con los tutores
C5	14– 26/09/2026	Correcciones técnicas del generador tras la revisión de los tutores.	Causa: Observaciones relativas a la integración con Flamapy, la validez de los modelos UVL, la comprobación de satisfacibilidad y la organización de las pruebas. Impacto: Continuación de las correcciones y validaciones técnicas durante septiembre de 2026, sin modificar el alcance funcional del proyecto. Seguimiento: Revisado con los tutores

Tabla 3.12: Registro de control de cambios sobre la línea base.

3.8 Plan de calidad

El plan de calidad definido para el proyecto establece los criterios utilizados para garantizar la calidad tanto de la solución software desarrollada como de la documentación asociada al TFG.

- **Código:** se ha buscado mantener una implementación organizada, legible y mantenible, siguiendo una estructura coherente con la arquitectura del generador. Durante el desarrollo se realizaron tareas de refactorización orientadas a mejorar la calidad interna del código. La trazabilidad de los cambios se mantuvo mediante Git y el uso progresivo de la convención *Conventional Commits*. Los workflows de GitHub Actions mencionados en la Sección 6.6 pertenecen al repositorio upstream de UVLHub.
- **Entregables:** cada bloque funcional desarrollado debía cumplir con los objetivos establecidos y aportar una funcionalidad verificable dentro del sistema. Para ello, las nuevas incorporaciones se validaban progresivamente antes de continuar con el desarrollo de nuevas capacidades, asegurando que los componentes implementados mantenían el comportamiento esperado.
- **Pruebas:** la calidad funcional del sistema se ha comprobado mediante la combinación de validaciones manuales y pruebas automatizadas. Tras la incorporación de nuevas funcionalidades se verificaba que los modelos generados cumplían los criterios esperados y que la integración con el asistente de UVLHub funcionaba correctamente. Además, se incorporaron pruebas unitarias adicionales cuando estas aportaban valor al detectar posibles errores relevantes en el sistema.
- **Memoria:** la calidad de la documentación se ha garantizado mediante revisiones periódicas de la estructura y contenido de la memoria, así como mediante la revisión de los avances con los tutores del proyecto.

3.9 Presupuesto

CapEx — Costes de capital

La Tabla 3.13 recoge los desembolsos de capital registrados. No se realizaron compras de equipos ni pagos por licencias atribuibles a este proyecto.

Recurso	Descripción	Coste directo registrado
Equipo de desarrollo	Equipo personal ya disponible, utilizado para la implementación y las pruebas del sistema.	0,00 €
Licencias software	Herramientas y tecnologías utilizadas sin pagos por adquisición de licencias atribuibles al proyecto.	0,00 €

Total CapEx	0,00 €
--------------------	---------------

Tabla 3.13: Desembolsos de capital registrados durante el proyecto.

Los recursos utilizados durante el desarrollo no supusieron una inversión económica directa conocida, ya que se empleó principalmente infraestructura propia y herramientas basadas en software libre o de coste nulo. Por tanto, el coste real asociado a los costes de capital del proyecto es de 0,00 €.

OpEx — Costes operacionales

Personal

El coste de personal corresponde a la dedicación asociada al contrato de apoyo técnico vinculado al proyecto TASOVA Plus. Dicho contrato tuvo una duración de cuatro meses, entre abril y julio de 2025, y contemplaba una dedicación de 25 horas semanales junto con una remuneración bruta mensual de 930,60 €.

La tarifa utilizada para calcular el coste estimado se ha fijado de forma uniforme en 9,31 € por hora y se aplica a las 750 horas de desarrollo previstas inicialmente. Por este motivo, los costes estimados de la tabla se obtienen multiplicando las horas estimadas de cada fase por dicha tarifa.

El coste real refleja únicamente la remuneración correspondiente al contrato de TASOVA Plus. Las cuatro mensualidades abonadas durante el periodo contractual suponen un coste real total de $4 \times 930,60 = 3722,40$ €, equivalente aproximadamente a 9,306 € por hora para las 400 horas contractuales. Las horas realizadas específicamente para el TFG se desarrollaron fuera del periodo financiado y no representan un coste directo adicional de personal. Véase la Tabla 3.14.

Actividad	H. esti- madas	H. TASO- VA	H. TFG	H. reales	Coste estimado (€)	Salario bruto (€)
Análisis inicial y planificación del proyecto	12	12	0	12	111,72	111,67
Análisis y diseño de la solución	25	25	0	25	232,75	232,65
Desarrollo e integración del generador automático	593	363	340	703	5520,83	3378,08
Pruebas y validación de la solución	120	0	170	170	1117,20	0,00
Total personal	750	400	510	910	6982,50	3722,40

Tabla 3.14: Desglose de horas y costes de personal.

Herramientas y servicios de desarrollo

La Tabla 3.15 indica los pagos registrados por las herramientas y los servicios utilizados durante el desarrollo.

Herramienta	Uso	Coste directo registrado
GitHub	Alojamiento del código fuente y segui- miento de cambios mediante el plan utilizado durante el proyecto.	0,00 €
Render	Despliegue de una versión accesible de UVLHub durante las revisiones funciona- les.	0,00 €
Python, Flask, Flamapy y de- pendencias	Tecnologías utilizadas para la implemen- tación y ejecución del sistema.	0,00 €
Total herra- mientas		0,00 €

Tabla 3.15: Pagos registrados por herramientas y servicios de desarrollo.

Resumen de costes directos cuantificados

La Tabla 3.16 resume los costes directos cuantificados para el proyecto.

Categoría	Tipo	Coste real	Valor de mercado
Infraestructura y licencias	CapEx	0,00 €	0,00 €
Personal	OpEx	3722,40 €	3722,40 €
Herramientas y servicios	OpEx	0,00 €	0,00 €
Total cuantificado		3722,40 €	3722,40 €

Tabla 3.16: Costes directos cuantificados para el proyecto.

El desembolso cuantificado corresponde a los 3722,40 € de salario bruto asociados al contrato de apoyo técnico de TASOVA Plus. No se han imputado compras de equipos, licencias ni pagos por los servicios de desarrollo utilizados. Esta cifra no representa el coste económico total del trabajo: no incluye las posibles cotizaciones a cargo de la entidad empleadora, la amortización del equipo propio ni una valoración monetaria de las 580 horas atribuibles al TFG.

3.10 Cierre del proyecto y lecciones aprendidas

Al finalizar el proyecto, todos los paquetes de trabajo definidos en la EDT han sido completados. Las fases planificadas se han ejecutado en su totalidad, sin necesidad de descartar ningún paquete de trabajo. Los cambios relevantes realizados durante el desarrollo han quedado registrados en la Sección 3.7, manteniendo la coherencia con el alcance inicial definido.

La desviación temporal global del proyecto se ha concentrado principalmente en las fases de desarrollo e integración del generador automático y pruebas y validación de la solución, debido a la complejidad añadida durante la integración con componentes externos y a la necesidad de realizar iteraciones adicionales para asegurar el correcto funcionamiento del sistema.

La Tabla 3.17 relaciona las principales dificultades encontradas con los cambios que se aplicarían en un desarrollo similar.

Lección	Qué se haría distinto
----------------	------------------------------

La estimación inicial del esfuerzo necesario para las fases de integración y validación fue inferior al esfuerzo realmente requerido.	Realizar una estimación más conservadora, incluyendo un margen adicional para problemas derivados de dependencias externas y correcciones durante la fase final.
El conocimiento inicial sobre algunas tecnologías utilizadas en el proyecto no era suficiente para prever completamente las dificultades de integración.	Dedicar una fase inicial más extensa al estudio de las herramientas y componentes externos antes de comenzar la implementación.
La estrategia de pruebas podría haberse definido con mayor profundidad desde las primeras etapas del desarrollo.	Diseñar desde el inicio una planificación de pruebas más completa, identificando las funcionalidades críticas y los casos de validación necesarios.
La acumulación progresiva de cambios durante el desarrollo generó cierta deuda técnica.	Realizar revisiones y refactorizaciones periódicas del código para mantener una arquitectura más mantenible durante todo el ciclo de desarrollo.

Tabla 3.17: Lecciones aprendidas al cierre del proyecto.

En conjunto, el proyecto se ha completado con una dedicación real de 980 horas, de las cuales 910 corresponden al desarrollo y 70 a la elaboración de la memoria. Aunque la planificación inicial se vio afectada por un periodo de inactividad y por varias iteraciones adicionales de integración y validación, todos los paquetes de trabajo definidos en la EDT fueron completados.

Con el proyecto planificado, ejecutado y cerrado, el siguiente capítulo precisa qué debe hacer exactamente la solución desarrollada. El Capítulo 4 presenta el catálogo de requisitos funcionales y no funcionales que sirve como base para el diseño, la implementación y la validación del sistema.

4 ANÁLISIS DE REQUISITOS

4.1 Descripción del sistema

El sistema desarrollado es un generador automático de modelos de características expresados en UVL, integrado en el módulo *generator* de UVLHub. El generador se organiza mediante un modelo de configuración y una operación compatibles con Flamapy, y permite configurar y ejecutar la generación desde la interfaz web de la plataforma.

El objetivo principal del sistema es facilitar la creación automática de modelos de características expresados en UVL con distintos niveles de complejidad y expresividad. Estos modelos permiten disponer de conjuntos de prueba más completos para la evaluación, validación y mejora de herramientas relacionadas con la Ingeniería de Líneas de Productos Software (SPL), especialmente aquellas orientadas al análisis de variabilidad. De esta forma, el sistema aborda la necesidad de disponer de modelos con características avanzadas que permitan evaluar el comportamiento de dichas herramientas en diferentes escenarios.

La entrada principal del sistema consiste en una configuración definida por el usuario mediante los parámetros disponibles en la interfaz proporcionada por UVLHub. Estos parámetros permiten establecer las características del modelo que se desea generar, incluyendo aspectos relacionados con su estructura jerárquica, atributos y restricciones.

Como resultado, el sistema produce modelos expresados en el lenguaje UVL con los niveles de expresividad soportados. Estos modelos pueden visualizarse directamente tras finalizar el proceso de generación dentro de la plataforma UVLHub y también pueden descargarse como ficheros independientes con extensión `.uvl` para su posterior utilización en otras herramientas del ecosistema de variabilidad software.

El sistema puede dividirse principalmente en los siguientes subsistemas:

- **Subsistema de configuración del generador:** responsable de gestionar los parámetros definidos por el usuario mediante la interfaz web proporcionada en UVLHub. Este subsistema permite establecer las características del modelo que se desea generar y preparar la configuración necesaria para ejecutar el proceso desde el navegador.
- **Subsistema de generación automática:** constituye el núcleo funcional del sistema y es el encargado de producir los modelos de características a partir de

la configuración proporcionada, haciendo uso de los componentes y mecanismos disponibles dentro del *framework* Flamapy.

- **Subsistema de integración con UVLHub:** encargado de conectar la operación generadora desarrollada con el módulo *generator* de UVLHub, permitiendo que la funcionalidad de la herramienta pueda ser utilizada dentro de la plataforma mediante los mecanismos definidos por esta.

El actor principal es el usuario investigador o desarrollador, que configura el generador y obtiene modelos UVL para experimentación, evaluación o validación. El acceso a esta funcionalidad no requiere autenticación previa.

Las herramientas externas de análisis pueden utilizar los ficheros descargados, pero su uso posterior no constituye una interacción directa con el generador descrito en estos casos de uso.

4.2 Requisitos de información

El sistema gestiona la información necesaria para configurar el proceso de generación automática de modelos de características. Esta información se representa mediante un conjunto de entidades de configuración que permiten definir los parámetros utilizados durante la generación.

La entidad principal del modelo conceptual es **FmgeneratorModel**, que agrupa las diferentes configuraciones específicas del generador. Cada una de estas configuraciones representa un aspecto concreto del modelo que se desea generar, como la nomenclatura, los niveles de expresividad UVL soportados, las características, la estructura jerárquica, las restricciones o los atributos.

Estas entidades son utilizadas posteriormente por la operación de generación automática para producir modelos de características expresados en UVL. La operación de generación no se considera una entidad de información, sino una funcionalidad del sistema que será descrita en los requisitos funcionales.

RI-01: FmgeneratorModel

La entidad **FmgeneratorModel** representa la configuración global del generador automático de modelos de características. Contiene los parámetros generales del proceso y las configuraciones específicas asociadas a cada aspecto del modelo generado.

La Tabla 4.1 recoge sus parámetros generales y las configuraciones que agrupa.

Atributo y tipo	Descripción
num_models int	Número de modelos que se desean generar.
seed int	Semilla utilizada para controlar la generación aleatoria.
ensure_satisfiable bool	Indica si debe aplicarse la generación booleana con comprobación de satisfacibilidad mediante SAT.
naming NamingConfig	Configuración relacionada con la nomenclatura de los elementos generados.
levels LevelConfig	Configuración de los niveles de expresividad soportados por UVL.
features FeaturesConfig	Configuración relacionada con la generación de características.
hierarchy HierarchyConfig	Configuración de la estructura jerárquica del modelo.
constraints ConstraintsConfig	Configuración de las restricciones generadas.
attributes AttributesConfig	Configuración de los atributos asociados a las características.

Tabla 4.1: Atributos del requisito de información RI-01.

RI-02: NamingConfig

La entidad **NamingConfig** representa la configuración utilizada para definir la nomenclatura de los elementos generados por el sistema.

La Tabla 4.2 presenta sus parámetros de nomenclatura.

Atributo y tipo	Descripción
name_prefix str	Prefijo utilizado para los nombres generados.
include_feature_count_suffix bool	Indica si se incluye el número de características en el nombre del modelo generado.

Atributo y tipo	Descripción
<code>include_constraint_count</code> <code>_suffix</code> <code>bool</code>	Indica si se incluye el número de restricciones en el nombre del modelo generado.

Tabla 4.2: Atributos del requisito de información RI-02.

RI-03: LevelConfig

La entidad **LevelConfig** representa los niveles de expresividad UVL que estarán disponibles durante el proceso de generación.

La Tabla 4.3 recoge los niveles configurables.

Atributo y tipo	Descripción
<code>boolean_level</code> <code>bool</code>	Representa el nivel booleano, que constituye la base obligatoria de los modelos generados.
<code>arithmetic_level</code> <code>bool</code>	Indica si se habilitan elementos aritméticos en los modelos generados.
<code>type_level</code> <code>bool</code>	Indica si se permite generar características tipadas y las construcciones de UVL asociadas al nivel de tipos.
<code>group_cardinality</code> <code>bool</code>	Indica si se habilitan cardinalidades de grupo.
<code>feature_cardinality</code> <code>bool</code>	Indica si se habilitan cardinalidades individuales de características.
<code>aggregate_functions</code> <code>bool</code>	Indica si se permite la generación de funciones agregadas.
<code>string_constraints</code> <code>bool</code>	Indica si se habilitan restricciones sobre valores de tipo cadena.

Tabla 4.3: Atributos del requisito de información RI-03.

RI-04: FeaturesConfig

La entidad **FeaturesConfig** contiene los parámetros relacionados con la generación de características del modelo.

La Tabla 4.4 presenta los límites y distribuciones aplicables a las características.

Atributo y tipo	Descripción
min_features int	Número mínimo de características adicionales a la raíz que se solicita generar.
max_features int	Número máximo de características adicionales a la raíz que se solicita generar.
dist_boolean float	Probabilidad de generación de características booleanas.
dist_integer float	Probabilidad de generación de características enteras.
dist_real float	Probabilidad de generación de características reales.
dist_string float	Probabilidad de generación de características de tipo cadena.
min_feature_cardinality int / list[int]	Valor mínimo permitido para las cardinalidades de características.
max_feature_cardinality int / list[int]	Valor máximo permitido para las cardinalidades de características.
prob_feature_cardinality float	Probabilidad de aplicar cardinalidades a las características.

Tabla 4.4: Atributos del requisito de información RI-04.

RI-05: HierarchyConfig

La entidad **HierarchyConfig** representa la configuración relacionada con la estructura jerárquica del modelo generado.

La Tabla 4.5 recoge los parámetros de profundidad y de relaciones jerárquicas.

Atributo y tipo	Descripción
max_tree_depth int	Profundidad máxima permitida para la jerarquía del modelo.
dist_optional float	Probabilidad de generar relaciones opcionales.
dist_mandatory float	Probabilidad de generar relaciones obligatorias.

Atributo y tipo	Descripción
dist_alternative float	Probabilidad de generar relaciones alternativas.
dist_or float	Probabilidad de generar relaciones OR.
group_cardinality_min int	Cardinalidad mínima para grupos de características.
group_cardinality_max int	Cardinalidad máxima para grupos de características.
dist_group_cardinality float	Probabilidad de generar relaciones con cardinalidad de grupo.

Tabla 4.5: Atributos del requisito de información RI-05.

RI-06: ConstraintsConfig

La entidad **ConstraintsConfig** contiene los parámetros utilizados para definir la generación de restricciones entre elementos del modelo.

La Tabla 4.6 presenta los límites y distribuciones empleados para generar restricciones.

Atributo y tipo	Descripción
min_constraints int	Número mínimo de restricciones generadas.
max_constraints int	Número máximo de restricciones generadas.
extra_constraint_representativeness int	Factor utilizado para aumentar la representatividad de las restricciones.
min_vars_per_constraint int	Número mínimo de variables utilizadas en una restricción.
max_vars_per_constraint int	Número máximo de variables utilizadas en una restricción.
ctc_dist_boolean float	Probabilidad de generar restricciones booleanas.

Atributo y tipo	Descripción
ctc_dist_integer float	Probabilidad de generar restricciones enteras.
ctc_dist_real float	Probabilidad de generar restricciones reales.
ctc_dist_string float	Probabilidad de generar restricciones sobre cadenas.
prob_not float	Probabilidad de aplicar negación en restricciones booleanas.
prob_and float	Probabilidad de utilizar operadores AND.
prob_or_ct float	Probabilidad de utilizar operadores OR.
prob_implication float	Probabilidad de utilizar implicaciones.
prob_equivalence float	Probabilidad de utilizar equivalencias.
prob_sum float	Probabilidad de utilizar operaciones de suma.
prob_subtract float	Probabilidad de utilizar operaciones de resta.
prob_multiply float	Probabilidad de utilizar operaciones de multiplicación.
prob_divide float	Probabilidad de utilizar operaciones de división.
prob_equals float	Probabilidad de utilizar comparaciones de igualdad.
prob_less float	Probabilidad de utilizar comparaciones menor que.
prob_greater float	Probabilidad de utilizar comparaciones mayor que.
prob_less_equals float	Probabilidad de utilizar comparaciones menor o igual que.

Atributo y tipo	Descripción
prob_greater_equals float	Probabilidad de utilizar comparaciones mayor o igual que.
prob_sum_function float	Probabilidad de utilizar funciones agregadas de suma.
prob_avg_function float	Probabilidad de utilizar funciones agregadas de promedio.
prob_len_function float	Probabilidad de utilizar funciones de longitud sobre cadenas.

Tabla 4.6: Atributos del requisito de información RI-06.

RI-07: AttributesConfig

La entidad **AttributesConfig** representa la configuración relacionada con los atributos asociados a las características generadas.

La Tabla 4.7 recoge los parámetros de generación automática y manual de atributos.

Atributo y tipo	Descripción
random_attributes bool	Indica si los atributos deben generarse automáticamente o definirse manualmente.
min_attributes int / None	Número mínimo de atributos generados en modo aleatorio.
max_attributes int / None	Número máximo de atributos generados en modo aleatorio.
dist_boolean_atr float	Probabilidad de seleccionar el tipo booleano en la generación aleatoria de atributos.
dist_integer_atr float	Probabilidad de seleccionar el tipo entero en la generación aleatoria de atributos.
dist_real_atr float	Probabilidad de seleccionar el tipo real en la generación aleatoria de atributos.
dist_string_atr float	Probabilidad de seleccionar el tipo cadena en la generación aleatoria de atributos.

Atributo y tipo	Descripción
<code>attributes_list</code> <code>list[dict]</code>	Lista de atributos definidos manualmente para la generación. Cada definición puede incluir su tipo, dominio, probabilidad de asociación y participación en restricciones.
<code>attribute_attach_probs</code> <code>list[float]</code>	Probabilidades de asociación de atributos a características.
<code>attribute_in_constraints</code> <code>list[bool]</code>	Indica si cada atributo definido manualmente puede participar en restricciones.

Tabla 4.7: Atributos del requisito de información RI-07.

4.3 Requisitos funcionales

El sistema debe proporcionar las funcionalidades necesarias para permitir la configuración, ejecución y obtención de modelos de características generados automáticamente. Los requisitos funcionales identificados se muestran en la Tabla 4.8.

ID	Nombre	Descripción	Prior.
RF-01	Configuración del generador	El sistema debe proporcionar un asistente de seis pasos para configurar la generación. El usuario debe poder indicar: el número de modelos, entre 1 y 1000; una semilla entera positiva; un prefijo opcional; los <i>major levels</i> y <i>minor levels</i> de UVL; los límites y distribuciones del <i>feature tree</i> ; el número, composición y operadores de las <i>cross-tree constraints</i> ; la generación automática o manual de <i>attributes</i> ; y las opciones de satisficibilidad y nomenclatura de los ficheros. Los parámetros y sus dependencias se detallan en las reglas RN-01 a RN-06.	Alta

ID	Nombre	Descripción	Prior.
RF-02	Validación de la configuración	El sistema debe validar cada paso antes de permitir el avance. Debe comprobar que los valores enteros sean válidos, que los límites mínimos no superen los máximos, que las probabilidades pertenecan al intervalo $[0, 1]$, que las distribuciones activas sumen 1 y que los <i>minor levels</i> sean compatibles con sus <i>major levels</i> . Cuando una validación falle, el sistema debe mantener los valores introducidos, permanecer en el mismo paso y mostrar el mensaje asociado al campo incorrecto. Por ejemplo: Number of models cannot exceed 1000., Requires Arithmetic level. o Current sum: 0.8000. Must be 1.0.	Alta
RF-03	Generación automática de modelos UVL	En el flujo ordinario, el sistema debe transformar la configuración validada en una instancia de <code>FmgeneratorModel</code> en el navegador y ejecutar <code>GenerateFeatureModel.execute()</code> para cada modelo solicitado. La operación debe producir un <code>FeatureModel</code> y aplicar la semilla configurada. Posteriormente, el resultado debe serializarse mediante <code>UVLWriter</code> . Cuando se active la comprobación de satisfacibilidad, la generación y la verificación mediante SAT deben realizarse en el servidor. La semilla efectiva debe depender de la semilla base, el índice del modelo y, cuando proceda, el índice del intento. Los errores deben comunicarse al usuario sin presentar un modelo incompleto como resultado válido.	Alta

ID	Nombre	Descripción	Prior.
RF-04	Soporte de niveles de expresividad UVL	El sistema debe generar modelos del <i>Boolean level</i> y permitir la activación del <i>Arithmetic level</i> , el <i>Type level</i> y los <i>minor levels</i> de <i>group cardinality</i> , <i>feature cardinality</i> , <i>aggregate functions</i> y <i>string constraints</i> . Cuando la comprobación de satisfacibilidad esté desactivada, las opciones seleccionadas deben habilitar las construcciones correspondientes y reflejarse en las declaraciones include pertinentes. Su activación no garantiza que cada construcción aparezca en todos los modelos generados: Boolean.group-cardinality , Arithmetic.feature-cardinality , Arithmetic.aggregate-function , Arithmetic.* o Type.string-constraints . Cuando esté activada, se aplicará la limitación al nivel booleano descrita en RN-06.	Alta
RF-05	Visualización del modelo generado	Tras completar correctamente la generación, el sistema debe mostrar en UVLHub los resultados obtenidos, incluido el nombre del fichero y los recuentos de características y restricciones. El usuario debe poder abrir la vista del contenido UVL de cada modelo. Estas acciones solo deben estar disponibles para modelos generados correctamente; si la generación falla, el sistema debe informar del error.	Media
RF-06	Descarga de los modelos generados	El sistema debe permitir descargar los resultados como ficheros con extensión .uvl . Cuando se generen varios modelos, estos deben incluirse en un archivo comprimido. El nombre de cada fichero debe utilizar el prefijo indicado o fm como valor predeterminado y aplicar, cuando hayan sido seleccionados, los sufijos con el número de características y restricciones. Si no se emplean sufijos y se genera más de un modelo, el sistema debe añadir su índice para evitar nombres idénticos.	Alta

Tabla 4.8: Requisitos funcionales del sistema.

4.4 Requisitos no funcionales

La Tabla 4.9 reúne las condiciones de integración, compatibilidad, ejecución y calidad del sistema.

ID	Nombre	Descripción	Prior.
RNF-01	Modularidad e integración	El generador debe mantenerse como un componente independiente compatible con la arquitectura de Flamapy. La operación GenerateFeatureModel debe implementar la interfaz de operación de Flamapy, poder localizarse mediante DiscoverMetamodels y utilizarse desde UVLHub sin duplicar su implementación.	Alta
RNF-02	Compatibilidad sintáctica con UVL	Los modelos serializados deben poder procesarse mediante UVLReader sin errores sintácticos. Las declaraciones include necesarias para las construcciones generadas deben conservarse durante la serialización.	Alta
RNF-03	Ejecución en el navegador	El flujo ordinario de generación debe ejecutarse en el navegador mediante Pyodide y WebAssembly. Para ese flujo, el servidor almacena la configuración como un diccionario en la sesión y la expone como JSON (<i>JavaScript Object Notation</i>); la instancia de FmgeneratorModel se construye en el navegador. Cuando el usuario activa la comprobación de satisfacibilidad, la generación y la verificación mediante SAT se realizan en el servidor.	Alta
RNF-04	Usabilidad del asistente	La configuración debe dividirse en seis pasos agrupados por funcionalidad. El sistema debe conservar los valores al navegar hacia atrás o después de una validación fallida, ocultar o desactivar las opciones que no sean aplicables y mostrar los errores junto al parámetro que debe corregirse.	Media

ID	Nombre	Descripción	Prior.
RNF-05	Calidad del software	La cobertura de sentencias ejecutables del conjunto de ficheros de código de producción evaluados en la Tabla 7.2 debe ser igual o superior al 80 %. La solución debe superar la suite automatizada descrita en el Capítulo 7. Los workflows de integración continua del repositorio upstream de UVLHub no forman parte de los criterios de aceptación específicos de este TFG.	Alta

Tabla 4.9: Requisitos no funcionales del sistema.

4.5 Reglas de negocio

La Tabla 4.10 concreta las validaciones y dependencias aplicables a la configuración.

ID	Nombre	Descripción
RN-01	Parámetros generales	El número de modelos, <code>num_models</code>, debe ser un entero comprendido entre 1 y 1000. Los valores inferiores producen el mensaje Number of models must be at least 1.; los superiores, Number of models cannot exceed 1000.; y los valores no enteros, Number of models must be an integer. La semilla, <code>seed</code>, debe ser un entero positivo. En caso contrario se muestra Seed must be a positive integer. El prefijo es opcional; si está vacío, se normaliza al valor <code>fm</code>.
RN-02	Dependencias entre niveles UVL	El <i>Boolean level</i> debe permanecer siempre activado. La selección del <i>Type level</i> activa automáticamente el <i>Arithmetic level</i>. Los <i>minor levels</i> de <i>feature cardinality</i> y <i>aggregate functions</i> requieren el <i>Arithmetic level</i>; si este no está activo, se muestra Requires Arithmetic level. El nivel de <i>string constraints</i> requiere el <i>Type level</i>. Si se selecciona sin él, se muestra Requires Type level. La <i>group cardinality</i> pertenece al <i>Boolean level</i> y no requiere activar niveles superiores.

ID	Nombre	Descripción
RN-03	Configuración del <i>feature tree</i>	Los parámetros <code>min_features</code> y <code>max_features</code> deben ser enteros mayores o iguales que 1 y el mínimo no puede superar al máximo. Los mensajes utilizados son <code>Min. features must be at least 1.</code>, <code>Max. features must be at least 1.</code> y <code>Max. features must be >= Min. features.</code> La profundidad <code>max_tree_depth</code> debe ser un entero comprendido entre 1 y <code>max_features</code>; en caso contrario se muestra <code>Maximum tree depth must be between 1 and Max. features.</code> Las probabilidades de las relaciones <i>optional</i>, <i>mandatory</i>, <i>alternative</i>, <i>or</i> y, cuando esté habilitada, <i>group cardinality</i>, deben pertenecer a $[0, 1]$ y sumar 1. Se admite una tolerancia de 0,001 en la interfaz. Los errores muestran <code>Value must be between 0 and 1.</code>, <code>The relation-distribution probabilities must sum to 1.0.</code> y el valor actual mediante <code>Current sum: X. Must be 1.0.</code> Cuando se active la <i>group cardinality</i>, sus límites deben ser enteros mayores o iguales que 0, el mínimo no puede superar al máximo y el máximo no puede superar <code>max_features</code>. Cuando se active la <i>feature cardinality</i>, sus límites también deben ser mayores o iguales que 1, el mínimo no puede superar al máximo y <code>prob_feature_cardinality</code> debe pertenecer a $[0, 1]$. La distribución de características booleanas, enteras, reales y de cadena debe estar formada por probabilidades de $[0, 1]$ cuya suma sea 1.

ID	Nombre	Descripción
RN-04	Configuración de restricciones	Los parámetros <code>min_constraints</code> y <code>max_constraints</code> deben ser enteros mayores o iguales que 1 y el mínimo no puede superar al máximo. Los parámetros <code>min_vars_per_constraint</code> y <code>max_vars_per_constraint</code> siguen la misma regla, y el máximo de variables no puede superar <code>max_features</code>. <code>extra_constraint_representativeness</code> debe ser un entero mayor o igual que 1 y menor o igual que <code>max_vars_per_constraint</code>. Los mensajes asociados son <code>Must be an integer.</code>, <code>Must be >= 1.</code> y <code>Must be <= Max. vars per constraint.</code> Las probabilidades de <code>and</code>, <code>or</code>, implicación y equivalencia deben sumar 1. La probabilidad de negación se valida de forma independiente dentro de $[0, 1]$. Si el <i>Arithmetic level</i> está activo, las probabilidades de suma, resta, multiplicación y división deben sumar 1. Cuando también estén activadas las <i>aggregate functions</i>, se incorporan a esa misma distribución las probabilidades de <code>sum()</code> y <code>avg()</code>. Las probabilidades de los operadores de comparación deben sumar 1. La distribución de tipos de restricciones debe sumar 1 considerando únicamente los tipos disponibles según los niveles activos. Por último, <code>prob_len_function</code> debe pertenecer a $[0, 1]$ cuando estén activos el <i>Type level</i> y las <i>string constraints</i>. Ante una suma incorrecta, el sistema muestra el valor calculado y mensajes como <code>The boolean-connective probabilities must sum to 1.0.</code>, <code>Arithmetic-level probabilities must sum to 1.0.</code>, <code>Comparison-operator probabilities must sum to 1.0.</code> o <code>Active type probabilities must total 1.0.</code>

ID	Nombre	Descripción
RN-05	Configuración de <i>attributes</i>	En modo automático, <code>min_attributes</code> y <code>max_attributes</code> deben ser enteros comprendidos entre 0 y 1000 y el mínimo no puede superar al máximo. Los mensajes asociados son <code>Min. attributes must be between 0 and 1000.</code>, <code>Max. attributes must be between 0 and 1000.</code> y <code>Max. attributes must be >= Min.</code> La distribución de <i>attributes</i> booleanos, enteros, reales y de cadena debe utilizar valores de $[0, 1]$ y sumar 1 considerando únicamente los tipos habilitados por los niveles seleccionados. En modo manual, cada <i>attribute</i> debe tener un nombre y una probabilidad de asociación perteneciente a $[0, 1]$. Si falta alguno de estos valores se muestran <code>Attribute name is required.</code> o <code>Attach probability is required.</code> Un <i>attribute</i> booleano debe permitir al menos uno de los valores verdadero o falso. Los atributos enteros, reales y de cadena deben definir límites mínimo y máximo numéricos, con el mínimo menor o igual que el máximo. En los atributos de cadena, ambos límites deben ser no negativos. Los atributos enteros o reales solo pueden participar en restricciones cuando está activo el <i>Arithmetic level</i>. Los atributos de cadena requieren el <i>Type level</i> y las <i>string constraints</i>. En caso contrario se muestran, respectivamente, <code>Integer/Real attrs can be used in constraints only when Arithmetic level is on.</code> y <code>String attrs can be used in constraints only when Type level + String constraints are on.</code>

ID	Nombre	Descripción
RN-06	Satisfacibilidad y nomenclatura de salida	Cuando se active ensure_satisfiable, la configuración efectiva utilizada para generar los modelos se limitará al nivel booleano. Se desestimarán los niveles aritmético y de tipos, sus niveles menores y la cardinalidad de grupo, aunque se hubieran seleccionado en el asistente. Tampoco se generarán atributos, características tipadas ni restricciones ajenas al dominio booleano. Cada modelo dispondrá de un máximo de 20 intentos. Cada candidato se comprobará mediante SAT y solo se devolverá si se confirma que es satisfacible. Si se agotan los intentos, se producirá SatisfiableModelGenerationError, cuyo mensaje identifica el número de modelo y los 20 intentos realizados. El error se comunicará al usuario sin presentar el último candidato como resultado válido. Cuando ensure_satisfiable esté desactivado, se tendrán en cuenta los niveles de expresividad, las cardinalidades y los atributos habilitados en la configuración. Los resultados se serializarán con extensión .uvl. El nombre base será el prefijo configurado o fm. Los sufijos opcionales tendrán la forma _Nf y _Mc, donde <i>N</i> es el número de características y <i>M</i> el número de restricciones. Cuando no se activen sufijos y num_models sea mayor que 1, se añadirá el índice del modelo.

Tabla 4.10: Reglas de negocio del dominio.

4.6 Restricciones del proyecto

La Tabla 4.11 recoge las restricciones tecnológicas, organizativas y temporales del proyecto.

ID	Tipo	Descripción
RES-01	Tecnológica	El desarrollo debe integrarse dentro del ecosistema existente de herramientas de variabilidad software, utilizando como base el <i>framework</i> Flamapy y la plataforma UVLHub. Esta condición limita la arquitectura de la solución, ya que el generador debe seguir los mecanismos de extensión definidos por Flamapy, adaptarse a la estructura proporcionada por UVLHub y utilizar las librerías propias del <i>framework</i> y componentes externos necesarios para la generación y representación de modelos de características.
RES-02	Tecnológica / interoperabilidad	Los modelos generados deben utilizar el lenguaje UVL como formato de salida, debido a que constituye el estándar empleado dentro del ecosistema de herramientas con el que se integra el proyecto.
RES-03	Organizativa	El desarrollo debe realizarse dentro del contexto del grupo de investigación y de los proyectos software existentes asociados, respetando los procedimientos, recursos y estructuras de trabajo establecidos.
RES-04	Temporal	El desarrollo y la entrega de la memoria deben ajustarse al calendario académico del TFG, cuya fecha límite de entrega es el 7 de octubre de 2026.

Tabla 4.11: Restricciones del proyecto.

4.7 Casos de uso

4.7.1 Diagrama general de casos de uso

El diagrama de casos de uso representa las principales interacciones entre los actores externos y el sistema generador de modelos UVL. El usuario investigador o desarrollador es el encargado de configurar los parámetros del generador y, una vez finalizado el proceso, puede visualizar los modelos generados dentro de la plataforma o descargarlos para su posterior utilización. El sistema se encarga de validar la configuración proporcionada y ejecutar el proceso de generación automática de modelos de características UVL. Los modelos descargados pueden ser utilizados posteriormente por herramientas externas de análisis de variabilidad para realizar procesos de evaluación y análisis sobre los modelos generados.

La Figura 4.1 muestra el diagrama general de casos de uso del sistema.

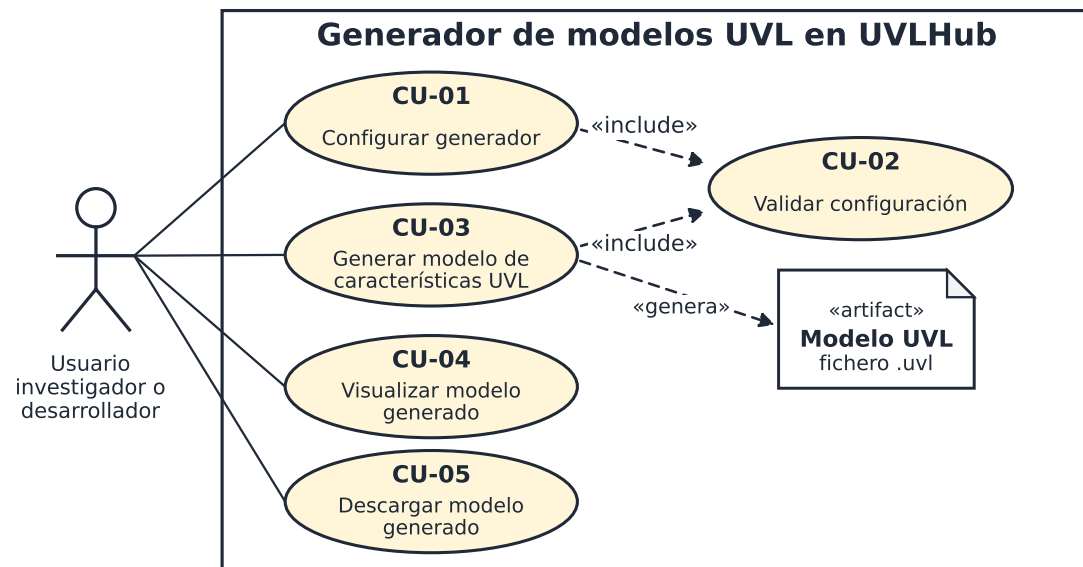


Figura 4.1: Diagrama general de casos de uso del sistema. Elaboración propia.

4.7.2 Especificación de casos de uso principales

La Tabla 4.12 especifica el caso de uso de configuración del generador.

CU-01 — Configurar generador	
Actores	Usuario investigador o desarrollador.
Descripción	Permite al usuario definir la configuración necesaria para el proceso de generación automática de modelos de características UVL. El usuario puede seleccionar los niveles de expresividad y establecer los parámetros relacionados con características, estructura jerárquica, restricciones y atributos.
Precondiciones	El usuario debe acceder a la interfaz de configuración del generador disponible dentro de UVLHub.
Flujo principal	1. El usuario accede al formulario de configuración del generador. 2. El usuario establece los parámetros asociados a la generación del modelo en cada paso del asistente. 3. El sistema registra la configuración proporcionada.
Flujo alternativo	Si los valores introducidos no cumplen las restricciones establecidas, el sistema informa al usuario de los errores detectados durante la introducción de dichos valores o antes de pasar al siguiente paso.
Postcondiciones	La configuración validada queda disponible para que el usuario solicite la generación desde el último paso.

Tabla 4.12: Especificación del caso de uso CU-01.

La Tabla 4.13 describe la validación que se realiza al avanzar por el asistente.

CU-02 — Validar configuración	
Actores	Usuario investigador o desarrollador.
Descripción	Permite comprobar que la configuración proporcionada por el usuario cumple las reglas de negocio definidas antes de ejecutar el proceso de generación automática.
Precondiciones	Debe existir una configuración del generador proporcionada por el usuario.
Flujo principal	1. El usuario solicita avanzar al siguiente paso del asistente. 2. El sistema comprueba los valores, las distribuciones y las dependencias correspondientes al paso actual. 3. Si la configuración es válida, el sistema guarda los parámetros y permite continuar.
Flujo alternativo	Si alguna regla de negocio no se cumple, el sistema conserva los valores introducidos, muestra los errores detectados e impide avanzar hasta que se corrija la configuración.
Postcondiciones	Si la configuración de cada paso es válida, se le permite al usuario pasar al siguiente paso.

Tabla 4.13: Especificación del caso de uso CU-02.

La Tabla 4.14 especifica la solicitud y ejecución de la generación.

CU-03 — Generar modelo de características UVL	
Actores	Usuario investigador o desarrollador.
Descripción	Permite generar automáticamente un modelo de características expresado en UVL a partir de una configuración previamente validada.
Precondiciones	La configuración proporcionada por el usuario debe haberse validado en los pasos anteriores y el entorno necesario para el flujo de generación seleccionado debe estar disponible.
Flujo principal	1. Desde el último paso, el usuario solicita la generación y el sistema inicia el flujo correspondiente a las opciones seleccionadas. 2. El sistema genera, para cada modelo y de forma aleatoria, la estructura jerárquica del modelo de características, incorporando los niveles de expresividad, atributos y restricciones pertinentes, cumpliendo con la configuración seleccionada por el usuario. 3. El sistema genera la representación final de los modelos en formato UVL y los almacena temporalmente para su visualización o descarga.
Flujo alternativo	Si se produce un error, el sistema informa del fallo y no presenta como válidos los modelos cuya generación o comprobación no se haya completado.
Postcondiciones	Existen uno o más modelos de características UVL generados correctamente disponibles para su posterior visualización o descarga.

Tabla 4.14: Especificación del caso de uso CU-03.

La Tabla 4.15 describe la visualización de un resultado generado.

CU-04 — Visualizar modelo generado	
Actores	Usuario investigador o desarrollador.
Descripción	Permite al usuario consultar visualmente el modelo de características generado dentro de la plataforma UVLHub.
Precondiciones	Debe existir al menos un modelo generado correctamente por el sistema.
Flujo principal	1. El usuario selecciona la acción de visualizar uno de los modelos generados. 2. El sistema abre una vista con su contenido UVL. 3. El usuario consulta el modelo y cierra la vista cuando termina.
Flujo alternativo	Si no se ha podido realizar la generación previa a la visualización, el sistema lanza un error informativo.
Postcondiciones	El usuario puede consultar el modelo generado dentro de la plataforma.

Tabla 4.15: Especificación del caso de uso CU-04.

La Tabla 4.16 recoge las opciones de descarga de los resultados.

CU-05 — Descargar modelo generado	
Actores	Usuario investigador o desarrollador.
Descripción	Permite descargar individualmente los modelos como ficheros <code>.uvl</code> o descargar el conjunto generado en un archivo comprimido.
Precondiciones	Debe existir al menos un modelo de características generado correctamente.
Flujo principal	1. El usuario selecciona la descarga de un modelo concreto o la descarga del conjunto de resultados. 2. El sistema prepara el fichero <code>.uvl</code> correspondiente o el archivo comprimido con los modelos generados. 3. El navegador inicia la descarga del fichero preparado.
Flujo alternativo	Si el modelo no se encuentra disponible o se produce un error durante la generación del fichero, el sistema informa al usuario y cancela la descarga.
Postcondiciones	El usuario dispone del modelo descargado en formato <code>.uvl</code> o del archivo comprimido con el conjunto de modelos.

Tabla 4.16: Especificación del caso de uso CU-05.

4.8 Matriz de trazabilidad

4.8.1 Requisitos funcionales ↔ casos de uso

La Tabla 4.17 relaciona cada requisito funcional con el caso de uso que lo desarrolla.

	CU-01	CU-02	CU-03	CU-04	CU-05
RF-01	×				
RF-02		×			
RF-03			×		
RF-04	×		×		
RF-05				×	
RF-06					×

Tabla 4.17: Matriz de trazabilidad entre requisitos funcionales y casos de uso.

4.8.2 Requisitos $\leftrightarrow$ pruebas

La Tabla 4.18 recoge las pruebas y evidencias utilizadas para verificar los requisitos funcionales y no funcionales.

Req.	Tipo de verificación	Pruebas y evidencia proporcionada
RF-01	Integración del asistente	<code>test_wizard_flow.py::test_full_happy_path_produces_valid_params</code> recorre los seis pasos y comprueba el diccionario de parámetros resultante; <code>test_wizard_persisters.py::test_apply_step1_persists_batch_fields</code> verifica la persistencia de los parámetros generales; <code>test_selenium.py::test_step2_configuration_is_propagated_to_step3</code> comprueba la propagación de la configuración entre pasos; y <code>test_wizard_outputs.py::test_back_navigation_preserves_all_choices</code> verifica que las opciones se conservan al navegar hacia atrás.
RF-02	Pruebas unitarias y de contrato	<code>test_validators.py::test_step1_rejects_batch_size_that_could_break_generation</code> comprueba el límite de 1000 modelos; <code>test_validators.py::test_step1_rejects_non_numeric_values</code> valida los valores numéricos de los parámetros generales; <code>test_validators.py::test_step2_rejects_min_or_levels_without_required_major_levels</code> comprueba las dependencias entre niveles; <code>test_validators.py::test_step3_rejects_invalid_relation_distribution</code> , <code>test_validators.py::test_step4_rejects_invalid_arithmetic_and_constraint_type_distributions</code> y <code>test_validators.py::test_step5_rejects_invalid_random_attribute_distribution</code> verifican las distribuciones de los pasos correspondientes. Las pruebas de <code>test_params_contract.py</code> comprueban además el contrato entre los parámetros del asistente y el modelo de configuración del generador.

Req.	Tipo de verificación	Pruebas y evidencia proporcionada
RF-03	Pruebas unitarias, integración y sistema	<code>test_engine.py::test_num_models_respected</code> comprueba el tamaño del lote; <code>test_engine.py::test_determinism_same_seed_same_output</code> y <code>test_engine.py::test_different_seed_different_output</code> verifican el comportamiento asociado a la semilla; <code>test_wizard_outputs.py::test_wrapper_generates_feature_model_with_received_arguments</code> comprueba los argumentos entregados a <code>GenerateFeatureModel</code> ; <code>test_wizard_outputs.py::test_wrapper_generate_models_returns_serialized_batch</code> valida la generación y serialización de un lote; <code>test_wizard_outputs.py::test_ensure_satisfiable_retries_until_sat</code> comprueba los reintentos de generación satisfiable; y <code>test_selenium.py::test_step6_generates_one_uvl_model_with_pyodide</code> ejecuta el flujo ordinario de generación desde la interfaz.
RF-04	Pruebas unitarias e integración	<code>test_engine.py::test_relation_kinds_create_expected_cardinalities</code> comprueba las relaciones <i>mandatory</i> , <i>optional</i> , <i>alternative</i> , <i>or</i> y de cardinalidad; <code>test_engine.py::test_group_cardinality_on_with_weight_produces_group_groups</code> y <code>test_engine.py::test_feature_cardinality_on_produces_some</code> validan las cardinalidades; <code>test_engine.py::test_arithmetic_level_produces_arith_constraints</code> , <code>test_engine.py::test_aggregate_functions_produce_sum_avg</code> y <code>test_engine.py::test_string_level_produces_len_constraints</code> comprueban los niveles aritmético, agregado y de cadenas; y <code>test_engine.py::test_generated_model_declares_correct_uvl_includes</code> verifica las declaraciones <code>include</code> correspondientes.
RF-05	Pruebas de sistema e integración	<code>test_wizard_outputs.py::test_wrapper_generate_one_model_returns_download_payload</code> comprueba que el resultado contiene el nombre, el contenido UVL y los contadores de características y restricciones; <code>test_selenium.py::test_step6_generates_one_uvl_model_with_pyodide</code> verifica que la interfaz muestra el resultado y sus acciones; y <code>test_selenium.py::test_step6_shows_output_options</code> comprueba que las opciones de salida aparecen antes de generar.

Req.	Tipo de verificación	Pruebas y evidencia proporcionada
RF-06	Pruebas unitarias, integración y sistema	<code>test_validators.py::test_step6_accepts_all_output_options</code> comprueba la configuración de los sufijos; <code>test_wizard_persisters.py::test_apply_step6_output_persists_checkboxes</code> verifica su persistencia; <code>test_wizard_outputs.py::test_wrapper_builds_expected_filename</code> y <code>test_wizard_outputs.py::test_serialize_generated_model_covers_all_filename_branches</code> comprueban las combinaciones de nombres y sufijos; <code>test_wizard_outputs.py::test_wrapper_generate_one_model_returns_download_payload</code> valida el contenido preparado para la descarga; y <code>test_selenium.py::test_step6_generates_one_uvl_model_with_pyodide</code> comprueba que la acción de descarga queda disponible tras generar el modelo. La descarga efectiva del archivo comprimido se verificó manualmente.
RNF-01	Integración arquitectónica	<code>test_engine.py::test_generate_feature_model_is_discoverable_by_flamapy</code> comprueba que <code>DiscoverMetamodels</code> localiza la operación <code>GenerateFeatureModel</code> ; <code>test_wizard_outputs.py::test_wrapper_build_one_delegates_to_feature_model_generator</code> verifica la delegación desde el <i>wrapper</i> ; y <code>test_params_contract.py::test_wizard_referenced_params_keys_are_mapped_to_generator_model</code> comprueba el contrato entre los parámetros de <code>UVLHub</code> y <code>FmgeneratorModel</code> .
RNF-02	Compatibilidad sintáctica con UVL	<code>test_wizard_outputs.py::test_o1_level_combinations_roundtrip_to_uvl_and_boolean_sat</code> genera combinaciones de niveles, serializa los modelos y los vuelve a leer mediante <code>UVLReader</code> ; <code>test_wizard_outputs.py::test_wrapper_serializes_uvl_and_reads_includes</code> comprueba la conservación de los <code>include</code> ; y <code>test_engine.py::test_generated_model_declares_correct_uvl_includes</code> verifica que cada configuración declara los niveles UVL necesarios.

Req.	Tipo de verificación	Pruebas y evidencia proporcionada
RNF-03	Ejecución en el navegador e integración con el servidor	<code>test_selenium.py::test_wizard_reaches_step5_with_pyodide_ready</code> comprueba la carga de Pyodide, la instalación de dependencias y la disponibilidad de la API <code>generateOne</code> ; <code>test_selenium.py::test_step6_generates_one_uvl_model_with_pyodide</code> ejecuta la generación ordinaria en el navegador; <code>test_wizard_flow.py::test_generate_sat_returns_models</code> comprueba la generación solicitada con satisfacibilidad; y <code>test_wizard_flow.py::test_params_json_returns_400_without_params</code> verifica la respuesta del endpoint de configuración cuando no hay parámetros en la sesión.
RNF-04	Usabilidad, navegación y estado	<code>test_selenium.py::test_step1_numeric_fields_expose_expected_browser_constraints</code> comprueba las restricciones de los campos numéricos; <code>test_selenium.py::test_step2_type_level_enables_arithmetic_level_in_ui</code> y <code>test_selenium.py::test_step2_arithmetic_toggles_arithmetic_minor_levels_visibility</code> verifican el comportamiento de las opciones dependientes; <code>test_selenium.py::test_wizard_preserves_step1_values_after_next_and_previous</code> y <code>test_wizard_state.py::test_step_state_is_preserved_after_returning_to_previous_step</code> comprueban la conservación de los valores; y <code>test_wizard_flow.py::test_step1_invalid_post_renders_errors</code> verifica que los errores de validación se muestran en el formulario.
RNF-05	Suite automatizada y cobertura	La suite automatizada ejecuta 220 pruebas de UVLHub y 44 pruebas del paquete <code>fm_generator</code> . Incluye pruebas unitarias en <code>test_engine.py</code> , <code>test_validators.py</code> y <code>test_params_contract.py</code> ; pruebas de integración en <code>test_wizard_flow.py</code> , <code>test_wizard_outputs.py</code> , <code>test_wizard_persisters.py</code> , <code>test_wizard_state.py</code> y <code>test_wizard_builders.py</code> ; y pruebas de sistema en <code>test_selenium.py</code> . La Tabla 7.2, presentada en el Capítulo 7, registra una cobertura conjunta del código de producción del 86,5 %, superior al umbral fijado del 80 %.

Tabla 4.18: Matriz de trazabilidad entre requisitos funcionales y no funcionales y sus verificaciones.

La descarga efectiva del archivo comprimido se comprobó manualmente, ya que las pruebas automatizadas verifican la preparación de los ficheros y la disponibilidad de la acción de descarga, pero no inspeccionan el archivo descargado por el navegador.

Una misma prueba puede aportar evidencia para varios requisitos cuando sus aserciones comprueban propiedades diferentes del flujo. En cada fila se indica la propiedad concreta verificada para evitar interpretar esta reutilización como una duplicación de cobertura. La matriz recoge pruebas representativas y no pretende enumerar la totalidad de la suite automatizada.

Con los requisitos funcionales, no funcionales, reglas de negocio y restricciones del proyecto definidos y trazados, queda establecido el comportamiento esperado del sistema y las condiciones que debe cumplir la solución desarrollada. El siguiente capítulo aborda cómo satisfacer estos requisitos mediante la definición de la arquitectura del sistema, los componentes principales y las decisiones de diseño adoptadas durante el desarrollo.

5 DISEÑO DE LA SOLUCIÓN

5.1 Modelo conceptual del dominio

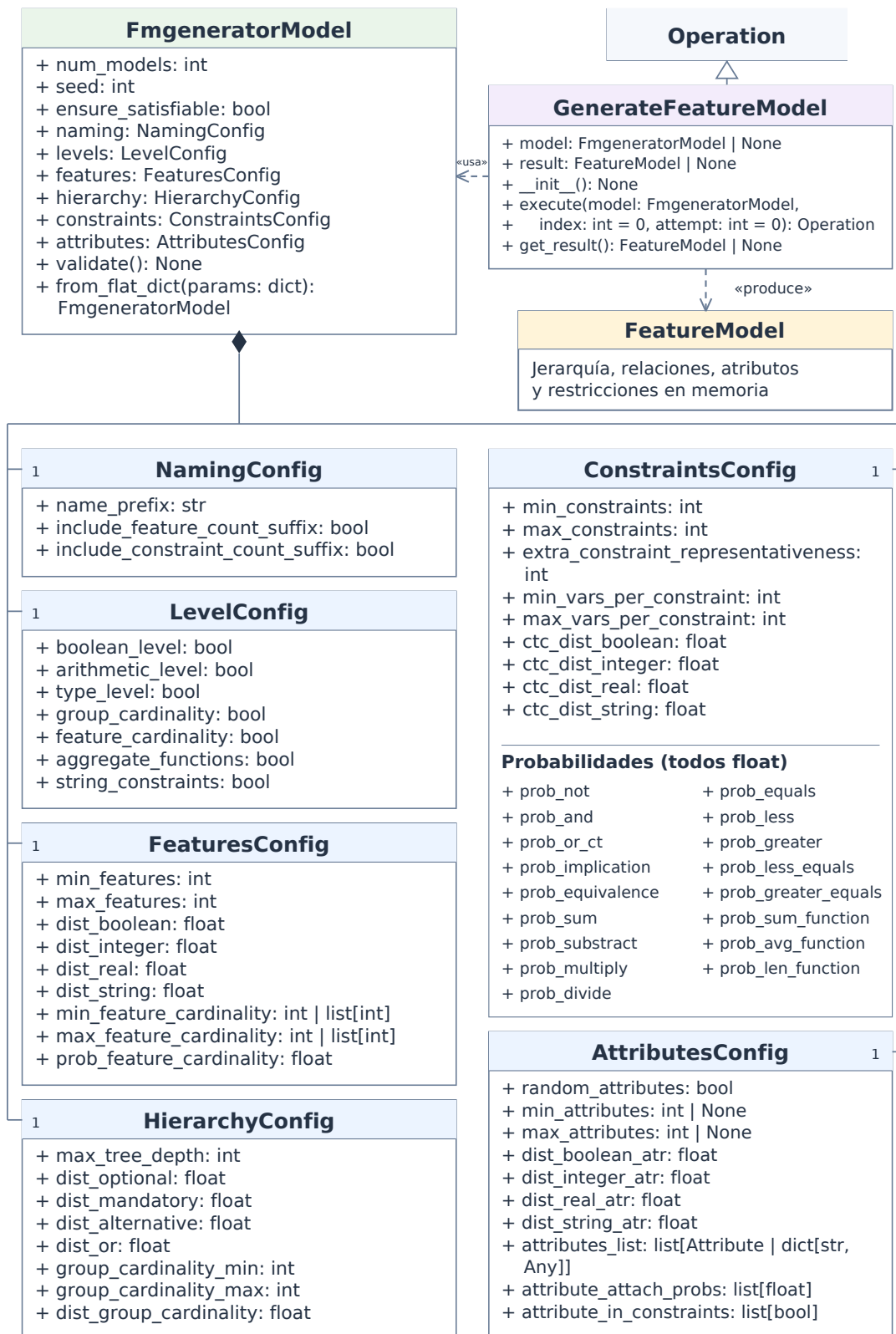
El modelo conceptual utilizado para el desarrollo de la solución representa la estructura estándar mediante la cual se implementa un componente compatible con el ecosistema Flamapy. En este caso, dicho modelo ha sido utilizado para representar un generador automático de modelos de características expresados en UVL, definiendo las entidades necesarias para configurar y ejecutar el proceso de generación. La solución se ha desarrollado siguiendo el paradigma de programación orientada a objetos de Python, donde cada entidad del modelo se implementa mediante clases organizadas en archivos con extensión `.py`. Esta estructura permite separar las responsabilidades de los diferentes componentes de la herramienta, diferenciando la configuración del generador de la operación encargada de producir los modelos de características.

La integración con UVLHub comienza con la recogida y comprobación de los parámetros en los distintos pasos del asistente. Durante este recorrido, UVLHub conserva la configuración como un diccionario en la sesión de Flask. Al solicitar la generación, el navegador obtiene dicho diccionario en formato JSON. Si no se solicita la comprobación de satisfacibilidad, `fmgen_wrapper.py` construye `FmgeneratorModel` y ejecuta `GenerateFeatureModel` dentro de Pyodide. Si se activa dicha opción, el navegador envía los parámetros al servidor, donde se generan y comprueban los modelos booleanos mediante SAT. Los resultados se ofrecen para su visualización o descarga, sin almacenarlos de forma permanente.

5.1.1 Estructura del modelo

El modelo conceptual representa la estructura interna del componente desarrollado siguiendo una arquitectura compatible con Flamapy. En este caso, dicho modelo ha sido adaptado para representar un generador automático de modelos de características expresados en UVL, separando la configuración del generador, la operación encargada de ejecutar la generación y el resultado producido. Véase la Figura 5.1.

Modelo conceptual del generador de modelos de características



Composición: seis configuraciones (1 de cada tipo). Flechas discontinuas: uso y producción.

execute() devuelve self y genera un FeatureModel; get_result() recupera el modelo.

La conversión a UVL y el nombre de fichero se gestionan fuera de GenerateFeatureModel.

Figura 5.1: Modelado conceptual del sistema. Elaboración propia.

Las entidades principales que componen el modelo son las siguientes:

FmgeneratorModel

La entidad **FmgeneratorModel** representa la configuración global del generador de modelos de características. Actúa como entidad raíz del modelo conceptual y agrupa las diferentes configuraciones necesarias para definir las características del modelo que se desea generar, incluyendo aspectos relacionados con la nomenclatura, los niveles de expresividad UVL, la estructura jerárquica, las restricciones y los atributos. Esta entidad constituye la representación completa del estado del generador antes de iniciar el proceso de generación.

NamingConfig

La entidad **NamingConfig** representa la configuración relacionada con la generación de nombres de los elementos creados durante el proceso de generación. Su finalidad es establecer las reglas utilizadas para definir la nomenclatura de los modelos y sus elementos asociados.

LevelConfig

La entidad **LevelConfig** representa la configuración de los niveles de expresividad disponibles en el lenguaje UVL durante la generación. Esta entidad permite definir las capacidades del lenguaje que estarán habilitadas en los modelos generados, estableciendo las construcciones que podrán aparecer en el resultado final.

FeaturesConfig

La entidad **FeaturesConfig** representa la configuración asociada a la generación de características del modelo. Su responsabilidad es definir las condiciones que controlan la creación de los elementos principales que forman parte de la estructura del modelo de características.

HierarchyConfig

La entidad **HierarchyConfig** representa la configuración relacionada con la estructura jerárquica del modelo de características generado. Esta entidad define los parámetros necesarios para construir el árbol de características y establecer las relaciones jerárquicas entre sus elementos.

ConstraintsConfig

La entidad **ConstraintsConfig** representa la configuración utilizada para la generación de restricciones entre los elementos del modelo. Su objetivo es definir cómo se crean las relaciones adicionales entre características y atributos, permitiendo generar modelos con diferentes niveles de complejidad y expresividad.

AttributesConfig

La entidad **AttributesConfig** representa la configuración asociada a la generación de atributos vinculados a las características del modelo. Esta entidad permite establecer cómo se incorporan atributos al modelo generado, tanto mediante generación automática como mediante configuraciones definidas manualmente.

GenerateFeatureModel

La clase **GenerateFeatureModel** representa la operación encargada de generar un objeto **FeatureModel** a partir de la configuración definida en **FmgeneratorModel**. Hereda de **Operation** y dispone de un constructor sin parámetros. Su método **execute** recibe la configuración, el índice del modelo y el número de intento, valida los parámetros y coordina la generación de la jerarquía, los atributos y las restricciones.

La configuración recibida se almacena en el atributo **model**, mientras que el objeto generado se guarda en **result**. El método **execute** devuelve la propia operación, y el resultado se obtiene mediante **get_result**. Cada ejecución genera un único modelo de características. La conversión a texto UVL y la asignación del nombre del fichero se realizan fuera de esta clase.

FeatureModel generado

El objeto **FeatureModel** representa el resultado de una ejecución del generador. Contiene la raíz del modelo, la jerarquía de características, sus relaciones, los atributos y las restricciones. Constituye una representación en memoria compatible con el metamodelo de características de Flamapy. Su transformación a texto UVL se realiza posteriormente mediante **UVLWriter**, fuera de la operación **GenerateFeatureModel**.

5.1.2 Restricciones del dominio

- **Dependencia entre niveles de expresividad UVL:** El nivel booleano constituye la base de los modelos generados. En la configuración implementada, la activación del nivel de tipos requiere que también esté activo el nivel aritmético. Las opciones de los niveles secundarios se habilitan de acuerdo con los niveles principales seleccionados.

- **Validez sintáctica y satisfacibilidad:** La construcción de un objeto `FeatureModel` no demuestra por sí sola que su representación textual pueda ser interpretada por un lector UVL. Tampoco equivale a demostrar que el modelo sea satisfacible. La opción basada en SAT permite comprobar la satisfacibilidad de los modelos booleanos; la generación de modelos con niveles aritmético y de tipos proporciona artefactos con los que evaluar y desarrollar razonadores para esas construcciones. Cuando se activa la opción de satisfacibilidad, el sistema utiliza una configuración efectiva limitada a construcciones booleanas básicas; la comprobación SAT y los posibles reintentos se realizan en el servidor.
- **Reproducibilidad mediante el uso de semillas:** El usuario puede establecer la semilla utilizada para controlar la aleatoriedad del proceso de generación. Esta decisión permite repetir una ejecución utilizando la misma semilla y los mismos parámetros de configuración, siempre que se mantengan la versión del generador y las condiciones de ejecución. De esta forma, se facilita la reproducción de experimentos, la comparación de resultados y el análisis de posibles errores. Para ello, la clase `FmgeneratorModel` incorpora el atributo `seed` como parte de la configuración del generador.

5.1.3 Espacio de soluciones y configuraciones

El espacio de configuraciones del generador está determinado por los valores que pueden tomar los diferentes parámetros configurables de la clase `FmgeneratorModel`. Para calcular el número de configuraciones posibles sería necesario revisar los límites entre los cuales puede variar cada parámetro. Debido a que algunos de ellos no presentan un límite superior establecido, como ocurre con el número de características, además de que existen parámetros numéricos reales, el espacio de configuraciones posibles es ilimitado.

Asimismo, el uso de mecanismos pseudoaleatorios permite explorar distintas soluciones para una misma configuración mediante la variación de la semilla. Esto favorece la diversidad de los modelos generados, aunque el uso de semillas diferentes no garantiza que todos los resultados sean distintos.

5.2 Arquitectura del sistema

5.2.1 Visión general

La arquitectura diseñada organiza los diferentes componentes del sistema en capas lógicas, separando las responsabilidades asociadas a la interacción con el usuario, la

gestión de la configuración, la generación de modelos y la infraestructura necesaria para su ejecución.

La **capa de presentación** está formada por la interfaz web proporcionada por UVLHub, mediante la cual el usuario configura los diferentes parámetros del generador. Esta interfaz sigue un diseño basado en un flujo tipo asistente, permitiendo organizar la configuración del modelo en diferentes pasos y facilitar la comprensión del proceso por parte del usuario.

La **capa de API/controladores** está formada principalmente por las *routes* del módulo *generator* de UVLHub. Estas funciones se encargan de gestionar la comunicación entre los diferentes pasos del flujo de configuración, enviando y recibiendo los parámetros introducidos por el usuario hasta completar la configuración necesaria para ejecutar el proceso de generación.

La **capa de dominio/lógica** está formada por la lógica interna del generador desarrollado siguiendo la arquitectura de extensiones de Flamapy, junto con los servicios encargados de gestionar la configuración dentro de UVLHub. El componente compatible con Flamapy contiene el modelo de configuración y la operación que produce los modelos de características. Las rutas y la lógica del asistente de UVLHub comprueban los parámetros y mantienen el diccionario de configuración en la sesión. En la generación ordinaria, `FmgeneratorModel` se construye y la operación se ejecuta en el navegador mediante Pyodide. En la generación satisfacible, el servicio del asistente construye el mismo modelo de configuración y ejecuta la operación en el servidor, donde se aplica la comprobación mediante PySAT.

La **capa de infraestructura** está formada por los elementos necesarios para la ejecución y despliegue de la aplicación. Para el desarrollo local se ha utilizado Docker junto con sus correspondientes contenedores y configuraciones, permitiendo construir y levantar el entorno completo de la aplicación de forma reproducible. Además, se ha utilizado Render como plataforma de despliegue principalmente para facilitar a los tutores la revisión del funcionamiento de la aplicación durante el desarrollo del proyecto. Asimismo, esta capa incluye el entorno de ejecución basado en WebAssembly mediante Pyodide, utilizado para ejecutar el generador en el navegador en el flujo ordinario, y las dependencias de PySAT utilizadas por el servidor cuando se solicita la comprobación de satisfacibilidad.

La funcionalidad de generación no añade persistencia para las configuraciones ni para los modelos producidos. El diccionario de la sesión se utiliza de forma temporal y los resultados se ofrecen al usuario para su visualización o descarga. Esta descripción se refiere al generador integrado, sin excluir los mecanismos de persistencia que UVLHub utiliza para otras funcionalidades.

5.2.2 Diagrama de componentes

El diagrama de componentes mostrado en la Figura 5.2 representa la distribución de los principales módulos del sistema y la comunicación existente entre ellos.

El diagrama ofrece una vista estructural general; los recorridos de generación ordinaria y satisfacible se detallan en las Figuras 5.4, 5.5 y 5.7.

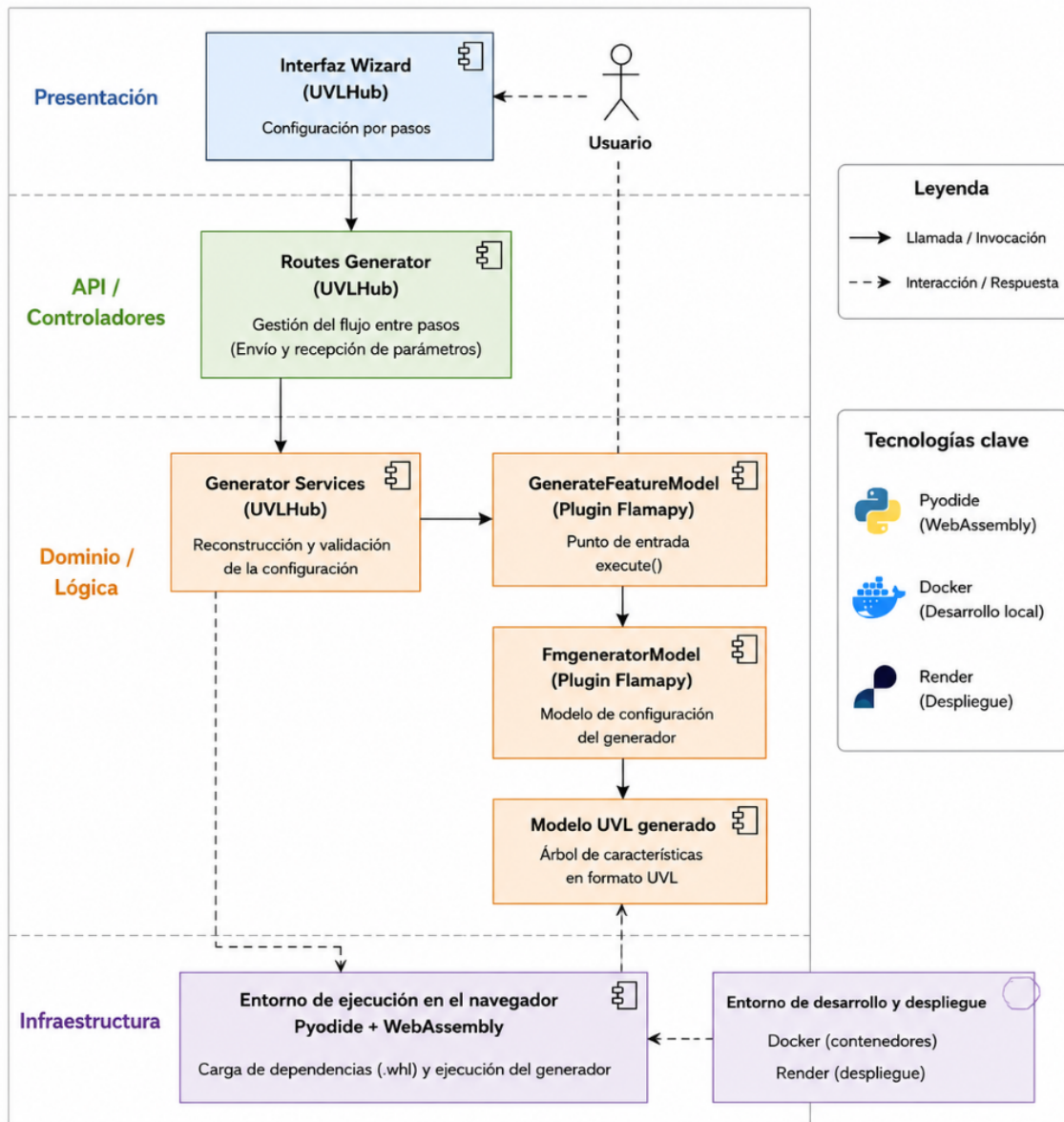


Figura 5.2: Diagrama de componentes del sistema. Elaboración propia.

5.2.3 Descripción secuencial del flujo de generación

1. **Inicio de la configuración:** El usuario accede al generador mediante la interfaz basada en asistente proporcionada por UVLHub y comienza a introducir los parámetros necesarios para definir las características del modelo que desea generar.

2. **Comunicación entre los diferentes pasos del asistente:** Las *routes* del módulo *generator* reciben los parámetros introducidos por el usuario en cada paso del flujo de configuración y se encargan de enviarlos al siguiente paso junto con la información necesaria para continuar con la configuración del generador.

En esta fase, la interfaz también realiza comprobaciones básicas, como la validación de las distribuciones de probabilidad, antes de enviar los valores al servidor.
3. **Actualización y validación de los parámetros:** Tras cada paso, las rutas y los servicios internos de UVLHub comprueban los parámetros recibidos y actualizan el estado del asistente almacenado en la sesión de Flask. Estas comprobaciones permiten detectar valores inválidos antes de continuar. En caso de error, se informa al usuario para que corrija los parámetros correspondientes. Una vez completado el asistente, los valores se consolidan en un diccionario de parámetros; en esta fase todavía no se construye la instancia de `FmgeneratorModel`.
4. **Obtención de la configuración y selección del flujo:** En el último paso del asistente, el navegador obtiene la configuración en formato JSON mediante el endpoint `/generator/random/params-json`. Si se solicita la generación ordinaria, prepara Pyodide y carga las dependencias necesarias para ejecutar el código Python mediante WebAssembly. Si se activa la opción de satisfacibilidad, envía los parámetros JSON mediante una petición **POST** a `/generator/random/generate-sat`.
5. **Construcción de la configuración y ejecución del generador:** En el flujo ordinario, `fngen_wrapper.py` construye `FmgeneratorModel` mediante `from_flat_dict` dentro de Pyodide. Después ejecuta `GenerateFeatureModel` para cada modelo solicitado, obtiene el objeto `FeatureModel` y lo transforma en texto UVL mediante `UVLWriter`. En el flujo de satisfacibilidad, el servicio del asistente construye la configuración efectiva booleana en el servidor y genera cada candidato con la misma operación. Cada candidato se comprueba mediante PySAT; si no es satisfacible, se realiza otro intento, hasta alcanzar el límite establecido. Solo los modelos satisfacibles se serializan y devuelven al navegador. Los errores de preparación, comprobación o generación se comunican al usuario.
6. **Visualización y descarga del resultado:** Los resultados generados correctamente se muestran en la interfaz de UVLHub para que el usuario pueda consultar su contenido UVL o descargarlos. La interfaz no presenta como válido ningún candidato cuya generación o comprobación haya fallado.

La Figura 5.3 muestra el flujo de configuración. Las Figuras 5.4 y 5.5 muestran, respectivamente, la generación ordinaria en el navegador y la generación satisfacible en el servidor.

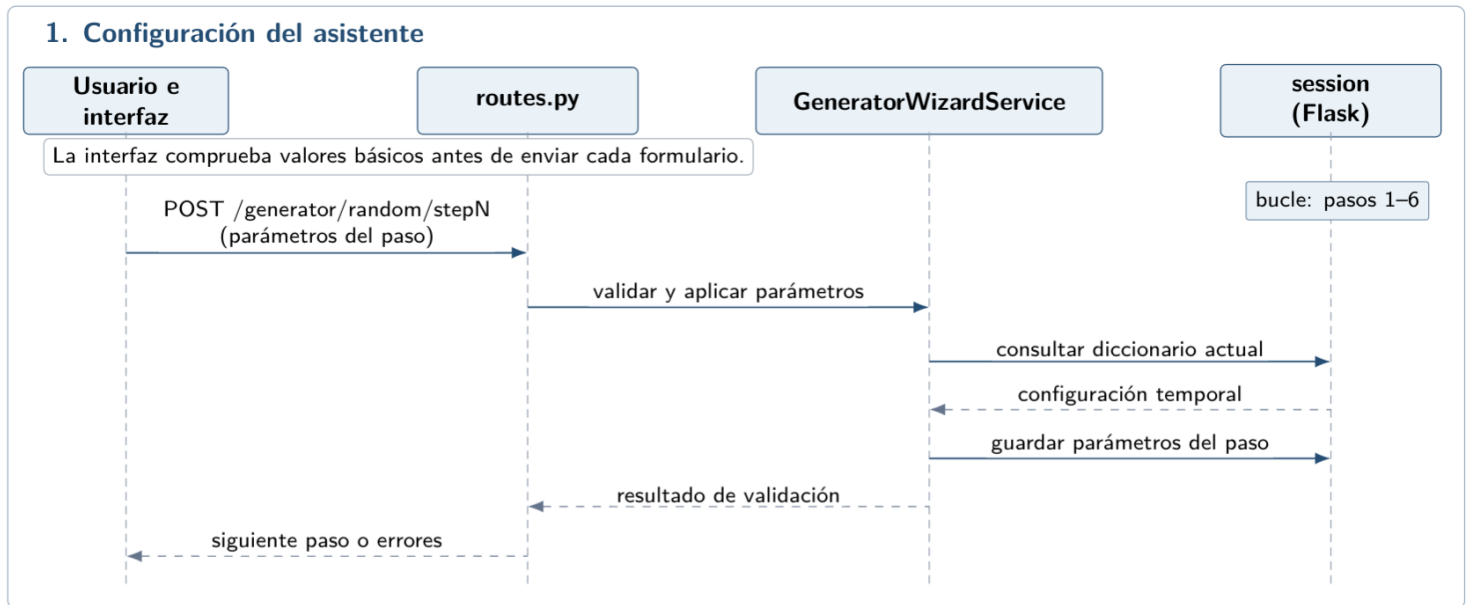


Figura 5.3: Diagrama de secuencia del flujo de configuración. Elaboración propia.

Generación ordinaria en el navegador

La configuración llega como JSON; el generador y UVLWriter se ejecutan en Pyodide

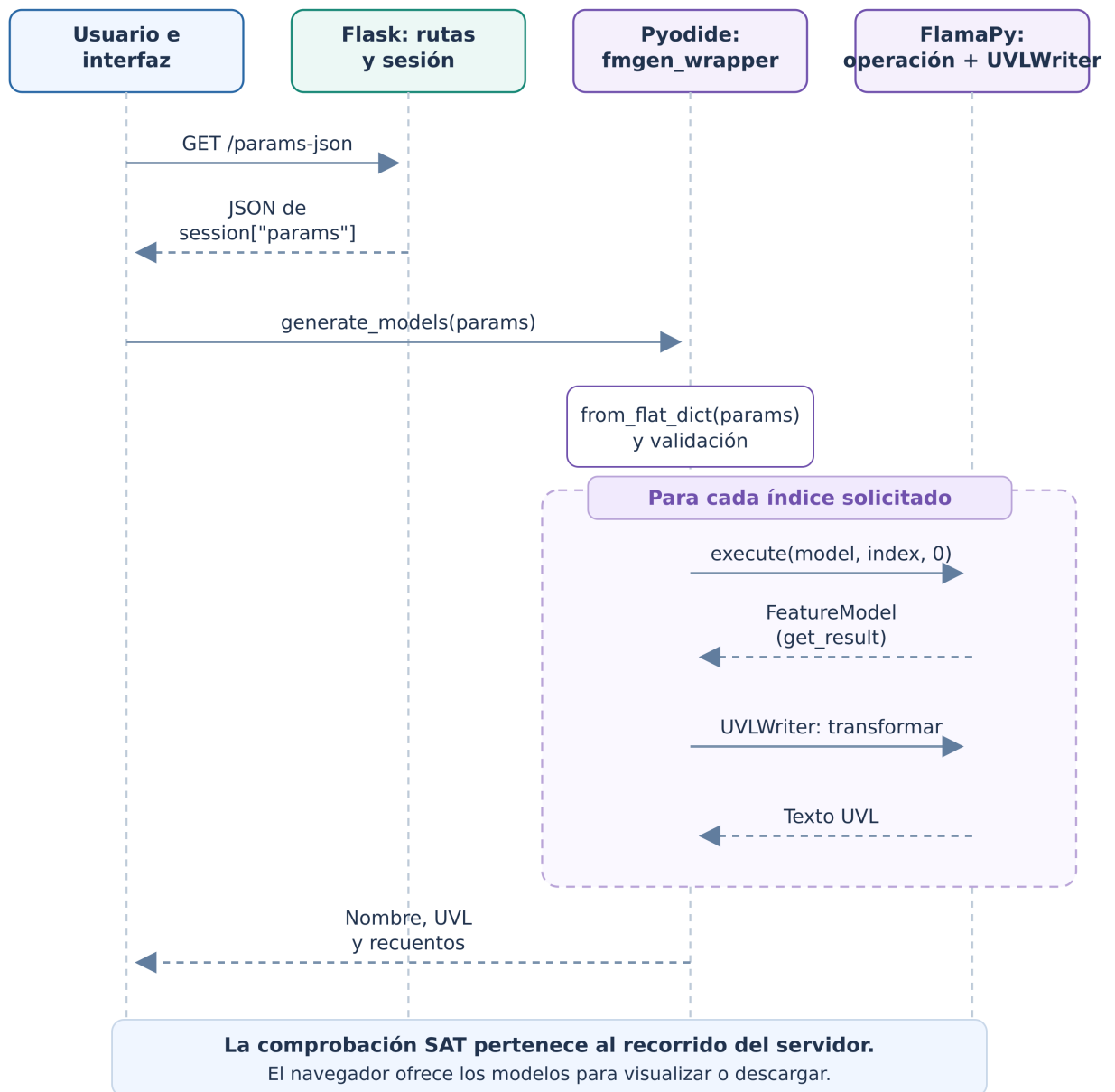


Figura 5.4: Diagrama de secuencia de la generación ordinaria de modelos UVL en el navegador mediante Pyodide. Elaboración propia.

Generación satisfacible en el servidor

Generar · comprobar con PySAT · serializar solo los candidatos aceptados

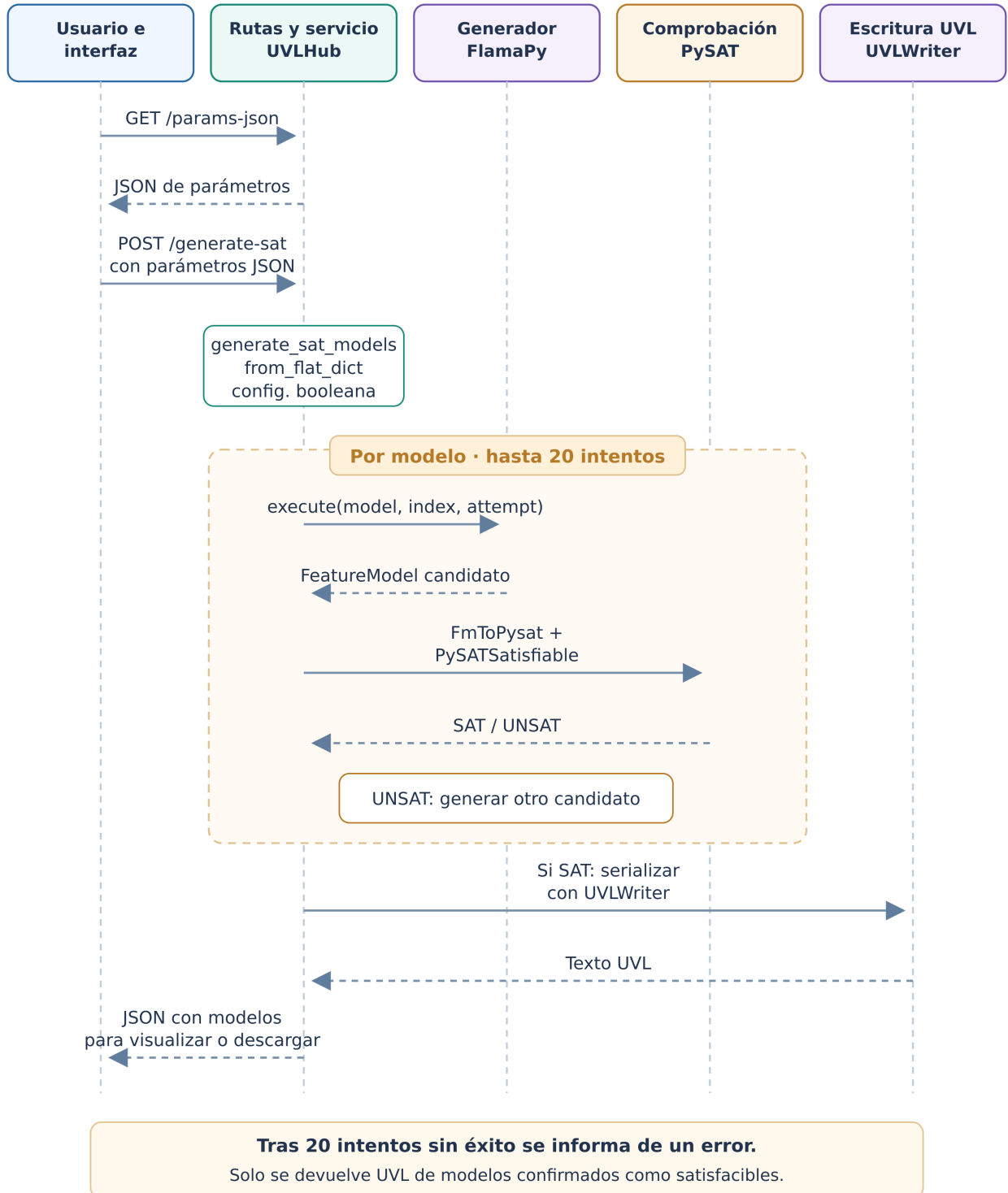


Figura 5.5: Diagrama de secuencia de la generación de modelos booleanos satisfacibles en el servidor mediante PySAT. Elaboración propia.

5.3 Patrones de diseño aplicados

Arquitectura por capas (*Layered Architecture*)

La solución sigue una organización basada en capas lógicas, separando las responsabilidades asociadas a la interfaz de usuario, comunicación con el sistema, lógica de generación e infraestructura. Este patrón permite reducir el acoplamiento entre componentes y facilitar la evolución del sistema, permitiendo modificar una capa sin afectar al resto siempre que se mantengan sus interfaces de comunicación.

Arquitectura basada en componentes independientes (*Component-based Architecture*)

El generador desarrollado se estructura como un componente independiente compatible con el ecosistema Flamapy, utilizando los mecanismos y modelos proporcionados por el *framework* para interactuar con él. Esta decisión permite separar la lógica de generación del resto de la aplicación, favoreciendo la reutilización, el mantenimiento y la evolución del sistema.

5.3.1 Patrón de interacción estilo asistente (*wizard*)

La interfaz de configuración del generador utiliza un patrón de interacción basado en asistente, mediante el cual la introducción de parámetros se divide en diferentes pasos independientes. Este patrón permite organizar una gran cantidad de opciones de configuración de forma progresiva, facilitando la comprensión del usuario y permitiendo realizar validaciones durante el proceso de configuración. Además, se incorporó un panel de progreso que permite consultar en todo momento el estado actual de la configuración realizada.

5.4 Diseño de la interfaz de usuario

La interfaz del asistente de configuración utilizada por el generador procede del trabajo previo realizado por David Romero dentro de UVLHub. En particular, la ampliación del asistente a seis pasos, el panel lateral de seguimiento de la configuración y el selector visual de distribuciones fueron diseñados e implementados con anterioridad a este TFG. Estas decisiones se describen en esta sección porque forman parte de la interfaz desde la que se utiliza el generador integrado, pero no constituyen una aportación propia de este trabajo. Las contribuciones realizadas en el TFG se centran en la implementación del componente generador compatible con Flamapy y en las adaptaciones necesarias para integrarlo con UVLHub.

5.4.1 Mapa de navegación

El mapa de navegación del generador sigue una estructura secuencial basada en un flujo tipo asistente, donde la configuración del modelo de características se divide en diferentes etapas independientes. Esta organización permite separar los diferentes grupos de parámetros configurables según su funcionalidad, facilitando al usuario la definición progresiva de la configuración necesaria para realizar la generación del modelo.

El mapa de navegación y las capturas de los seis pasos se recogen en el Apéndice B.

Las vistas principales del sistema son las siguientes:

Paso 1: Batch

Esta vista permite establecer los parámetros generales del proceso de generación, incluyendo el número de modelos a generar, la semilla utilizada para controlar la aleatoriedad y la nomenclatura de los modelos generados.

Paso 2: Language levels

Esta vista permite seleccionar los niveles de expresividad UVL que estarán disponibles durante la generación, incluyendo la activación de los diferentes niveles principales y secundarios soportados.

Paso 3: Feature tree

Esta vista permite configurar los parámetros relacionados con la generación de la estructura del árbol de características, incluyendo el número de características, la profundidad máxima y la distribución de relaciones jerárquicas.

Paso 4: Constraints

Esta vista permite definir los parámetros asociados a la generación de restricciones entre características y atributos, incluyendo la distribución de los diferentes tipos de restricciones y operadores disponibles.

Paso 5: Attributes

Esta vista permite configurar los parámetros relacionados con la generación de atributos asociados a las características, incluyendo la posibilidad de generar atributos aleatorios mediante una distribución de tipos configurable.

Paso 6: Output

Esta vista permite ejecutar el proceso de generación del modelo UVL utilizando la configuración proporcionada en los pasos anteriores. Además, permite activar la comprobación de satisfacibilidad, que aplica una configuración efectiva limitada al nivel booleano, y seleccionar los sufijos utilizados en los archivos generados.

Una vez finalizada la generación, el usuario puede visualizar el modelo obtenido dentro de la plataforma o descargarlo para su posterior utilización.

La Figura 5.6 muestra una vista general acerca de la navegación a través del sistema.

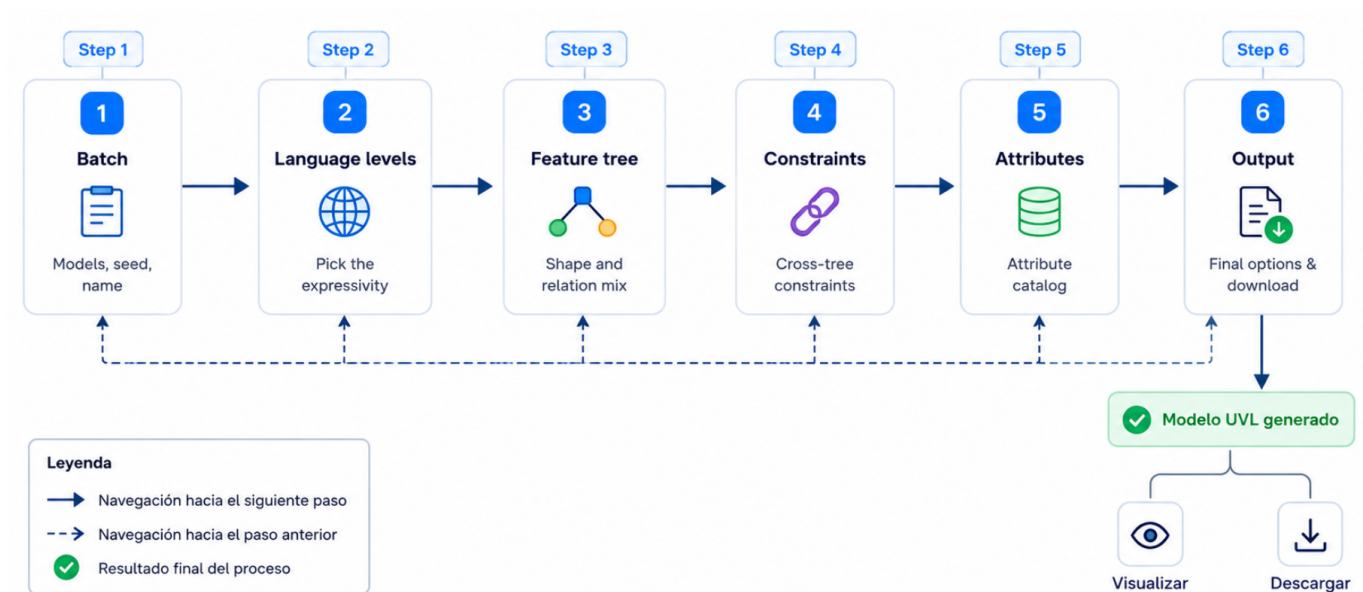


Figura 5.6: Mapa de navegación del generador UVL. Elaboración propia.

5.4.2 Prototipo visual inicial

Durante la fase inicial del desarrollo se realizó un prototipo de la interfaz con el objetivo de cubrir los requisitos iniciales identificados para el sistema. Este prototipo no representa completamente la versión final de la interfaz, ya que durante el desarrollo se fueron incorporando diferentes mejoras y modificaciones orientadas a mejorar la usabilidad y adaptar el sistema a las nuevas necesidades surgidas durante el desarrollo.

Sus cinco bocetos pueden consultarse en el Apéndice B.2.

5.4.3 Decisiones de diseño visual

- **Paleta de colores:** La interfaz mantiene la identidad visual proporcionada por UVLHub, utilizando principalmente tonos azules, blancos y grises para los diferentes elementos de la interfaz. Además, se utilizan colores específicos para

representar diferentes estados del sistema, empleando rojo para mensajes de error, amarillo anaranjado para avisos y verde para operaciones completadas correctamente.

- **Tipografía:** Se mantiene la tipografía utilizada por UVLHub, basada en las familias *Inter*, *sans-serif* y *Helvetica*, utilizadas como estándares dentro de la plataforma, conservando la coherencia visual con el resto de la aplicación y proporcionando una lectura clara de los elementos de la interfaz.
- **Organización de la interfaz y flujo de configuración:** Debido a la cantidad de parámetros necesarios para configurar el generador, la interfaz utiliza un diseño basado en un flujo tipo asistente, separando la configuración en diferentes pasos independientes. Esta decisión permite reducir la complejidad percibida por el usuario y mejorar la comprensión del proceso de configuración.

Además, dispone de un panel lateral de progreso que permite consultar en todo momento el estado actual del proceso, los pasos completados y un resumen de la configuración seleccionada sin necesidad de volver atrás durante el flujo.

- **Componentes visuales y sistema de diseño:** La interfaz utiliza los componentes proporcionados por el sistema de diseño utilizado en UVLHub, basado principalmente en Bootstrap. Además, la interfaz incorpora componentes específicos para el generador, como el selector de distribuciones probabilísticas utilizado para configurar la generación de modelos, permitiendo adaptar la interfaz a las necesidades particulares del sistema.
- **Ayudas visuales y gestión de la información:** Para facilitar la configuración de un sistema con una gran cantidad de parámetros, se muestran únicamente las opciones relacionadas con los niveles de expresividad seleccionados por el usuario. También se incluyen textos explicativos y *tooltips* en determinados parámetros con el objetivo de proporcionar contexto adicional sobre su funcionamiento.
- **Responsive y adaptación visual:** Aunque la aplicación está orientada principalmente a su uso desde equipos de escritorio, la interfaz se ha diseñado utilizando una estructura adaptable que permite su correcta visualización en diferentes tamaños de pantalla.

5.5 Flujo de datos principal

El flujo de datos del sistema comienza con los parámetros introducidos por el usuario mediante la interfaz basada en asistente. Las rutas del módulo **generator** de UVLHub reciben estos datos y, junto con los servicios internos, realizan las

comprobaciones correspondientes a cada paso. La configuración se mantiene en forma de diccionario dentro de la sesión de Flask, permitiendo conservar los valores introducidos durante el recorrido por el asistente.

Una vez completada la configuración, el navegador obtiene estos parámetros en formato JSON mediante el endpoint `/generator/random/params-json`. A partir de este punto existen dos recorridos. En la generación ordinaria, `fmgen_wrapper.py` construye `FmgeneratorModel` dentro de Pyodide mediante `from_flat_dict` y ejecuta `GenerateFeatureModel`. Los objetos `FeatureModel` obtenidos se transforman en texto UVL mediante `UVLWriter` dentro del navegador. Cuando se activa la comprobación de satisfacibilidad, el navegador envía los parámetros JSON mediante una petición `POST` a `/generator/random/generate-sat`. El servidor construye la configuración efectiva booleana, genera los candidatos, comprueba su satisfacibilidad mediante PySAT y devuelve los modelos aceptados en formato JSON. Ambos recorridos terminan en la interfaz, que permite visualizar y descargar los resultados. Esta separación responde al requisito RNF-03.

La Figura 5.7 representa el recorrido común de los parámetros y las dos vías de generación.

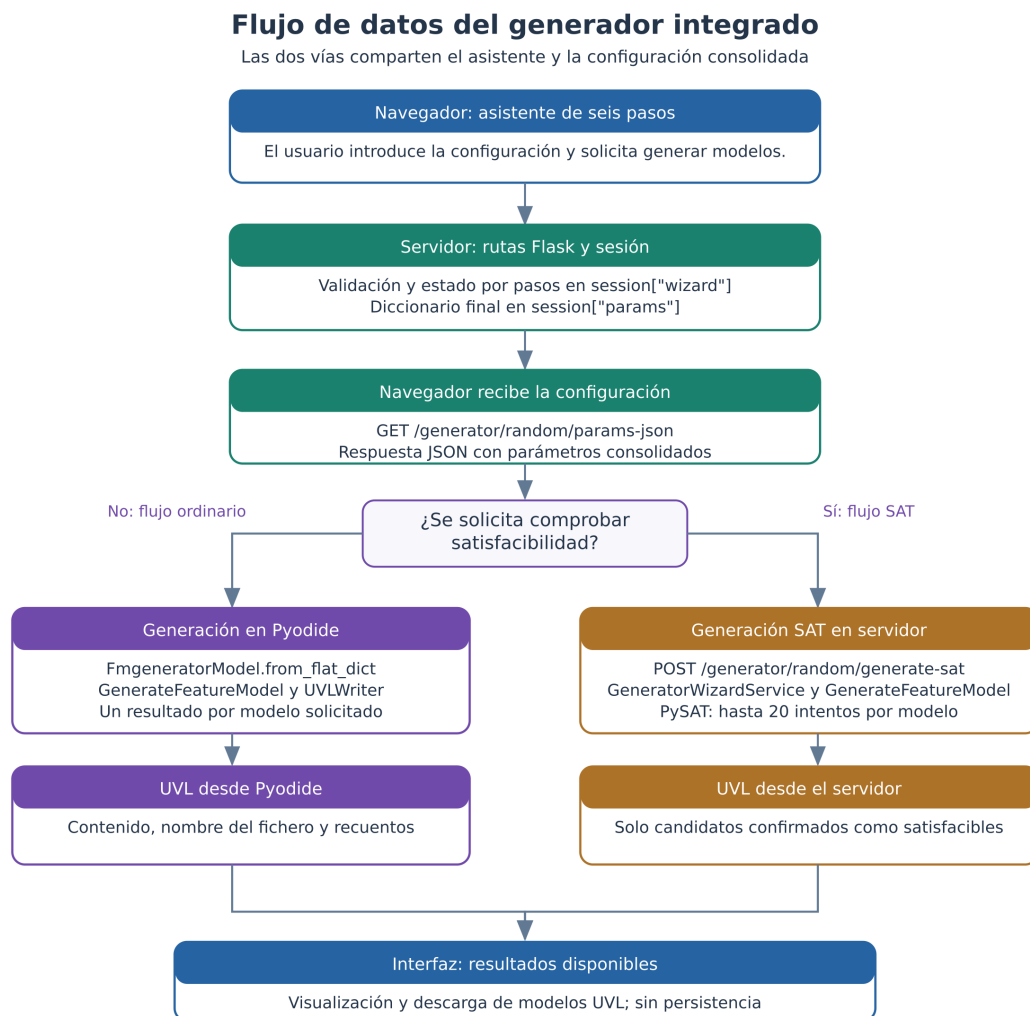


Figura 5.7: Flujo de datos de la generación ordinaria en Pyodide y de la generación satisfacible mediante SAT en el servidor. Elaboración propia.

5.5.1 Estructura de la representación intermedia

La representación intermedia de la configuración consiste en un diccionario de parámetros almacenado temporalmente en la sesión de Flask. Este diccionario permite mantener los valores introducidos por el usuario entre los diferentes pasos del asistente, sin construir todavía una instancia del modelo de dominio del generador.

Las rutas y los servicios del módulo **generator** gestionan y comprueban los parámetros correspondientes a cada paso. El estado se conserva por pasos en `session["wizard"]` y la configuración consolidada se almacena en `session["params"]`. Cuando el navegador solicita la configuración, el servidor proporciona este último diccionario en formato JSON. En la generación ordinaria, `fmgen_wrapper.py` lo convierte en una instancia de `FmgeneratorModel` mediante `from_flat_dict` dentro de Pyodide. En la generación satisfacible, el navegador envía el mismo diccionario al servidor y allí se construye la configuración efectiva utilizada por la operación.

El Código 5.1 muestra cómo el módulo recupera de la sesión los valores guardados para un paso y los combina con los valores predeterminados.

```
1 def load_step_state(step: int, defaults: dict):
2     wizard = session.get("wizard", {})
3     saved = wizard.get(str(step), {})
4
5     out = defaults.copy()
6     out.update(saved)
7
8     return out
```

Código 5.1: Lectura del estado de un paso desde la sesión de Flask.

Esta representación se utiliza para mantener el estado temporal del asistente y transferir los parámetros al entorno donde se realiza la generación. No constituye un registro permanente de las configuraciones utilizadas ni de los modelos obtenidos.

5.5.2 Mapeo entre niveles

El mapeo entre el diccionario de parámetros y el modelo interno de configuración se realiza mediante el método `from_flat_dict` de `FmgeneratorModel`. En la generación ordinaria, el método se ejecuta desde `fmgen_wrapper.py` en el navegador; cuando se solicita la comprobación de satisfacibilidad, se ejecuta desde el servicio del asistente en el servidor. Esta transformación permite pasar de la representación utilizada durante el asistente a la representación orientada a objetos que necesita el generador.

Algunos ejemplos de este mapeo son:

- *NUM_MODELS* $\rightarrow$ *num_models*: número de modelos de características que serán generados.
- *TYPE_LEVEL* $\rightarrow$ *levels.type_level*: activación de las construcciones asociadas al nivel de tipos del lenguaje UVL.
- *MIN_FEATURES* / *MAX_FEATURES* $\rightarrow$ *features.min_features* / *features.max_features*: rango utilizado para determinar el número de características del modelo generado.

Una vez construida la instancia de **FmgeneratorModel**, la configuración se valida y se utiliza para ejecutar **GenerateFeatureModel**. En el flujo ordinario, la construcción y la ejecución tienen lugar en el navegador; en el flujo satisfacible, se realizan en el servidor con una configuración efectiva booleana y cada candidato se comprueba mediante PySAT. Los objetos **FeatureModel** aceptados se transforman posteriormente a su representación textual en UVL.

Con la arquitectura del sistema, los modelos conceptuales, los flujos de datos y las principales decisiones de diseño definidos, queda establecida la solución desde un punto de vista estructural y conceptual. El siguiente capítulo aborda la implementación de dicha solución, detallando las tecnologías utilizadas y cómo los diferentes componentes definidos en este diseño han sido desarrollados para obtener el sistema final.

6 IMPLEMENTACIÓN

6.1 Análisis de tecnologías

La implementación del sistema requiere tecnologías compatibles con la arquitectura existente de UVLHub y con el ecosistema Flamapy. Su selección se ha realizado teniendo en cuenta la integración con la plataforma, la ejecución del generador en los entornos necesarios y la disponibilidad de un entorno reproducible para el desarrollo y el despliegue.

Necesidad técnica 1: Integración con la plataforma existente

El generador debe integrarse en UVLHub, una plataforma web desarrollada con Flask [29]. La solución debe aprovechar sus rutas, plantillas Jinja2 y mecanismos de sesión, sin introducir cambios estructurales innecesarios en la aplicación.

Necesidad técnica 2: Desarrollo de la herramienta generadora

La lógica de generación debe seguir la arquitectura de extensiones de Flamapy. Para ello, se necesita un componente independiente que encapsule la configuración del generador y la operación encargada de producir modelos de características. La conversión posterior a UVL se realiza mediante `UVLWriter`.

Necesidad técnica 3: Entornos de ejecución

La generación ordinaria se ejecuta en el navegador mediante Pyodide y WebAssembly. Cuando se solicita la comprobación de satisfacibilidad, la generación y los reinicios se ejecutan en el servidor, que dispone de las dependencias necesarias para utilizar PySAT. Ambos recorridos emplean la misma operación `GenerateFeatureModel`.

Necesidad técnica 4: Desarrollo y despliegue reproducibles

El sistema debe disponer de un entorno reproducible para el desarrollo local y de una versión accesible durante las revisiones del proyecto. Esto requiere contenerizar UVLHub y gestionar las dependencias de los componentes que se ejecutan tanto en el navegador como en el servidor.

Comparativa de tecnologías del backend

- **Flask:** *Framework* utilizado por UVLHub sobre el que se ha integrado el generador. Permite aprovechar directamente sus rutas, plantillas y mecanismos de sesión.
- **Django:** *Framework* web con numerosas funcionalidades integradas. Su utilización habría requerido adaptar la arquitectura existente de UVLHub para incorporar el generador.
- **FastAPI:** *Framework* orientado al desarrollo de APIs. Su adopción para esta funcionalidad habría añadido otro entorno web a una aplicación que ya utiliza Flask.

Comparativa de tecnologías del front-end

- **Jinja2, JavaScript y Bootstrap [30]:** Son las tecnologías utilizadas por UVLHub para construir las vistas y gestionar la interacción. Permiten integrar los pasos del asistente en la interfaz existente.
- **React:** Biblioteca de interfaces basada en componentes. Incorporarla habría exigido una arquitectura adicional para las pantallas del generador y su comunicación con UVLHub.
- **Vue.js:** *Framework* de interfaces reactivas. No se seleccionó porque los pasos del asistente ya se integran en las plantillas y los recursos JavaScript de UVLHub.

Análisis de la base de datos

El generador integrado no requiere una base de datos propia. Durante el asistente, la configuración se conserva temporalmente en la sesión de Flask; los modelos obtenidos se ofrecen para su visualización o descarga. No se ha añadido persistencia para guardar estas configuraciones ni los resultados de la generación. Esta decisión no modifica los mecanismos de almacenamiento utilizados por otras funcionalidades de UVLHub.

Plataformas de despliegue

- **Render [31]:** Plataforma utilizada para ofrecer una versión accesible de UVLHub durante las revisiones. Su elección responde a la facilidad de configuración y a la experiencia previa del autor.

- **Servicios *cloud* tradicionales (AWS, Azure y Google Cloud):** Ofrecen más posibilidades de configuración de infraestructura, que no eran necesarias para el despliegue de revisión previsto.
- **Heroku:** Habría permitido un despliegue similar, pero se disponía de experiencia previa y de una configuración establecida con Render.

Tecnologías finales seleccionadas

- **Servidor y lógica de generación:** Python, Flask, Flasky y PySAT para la comprobación de satisfacibilidad solicitada por el usuario.
- **Interfaz:** Plantillas Jinja2, JavaScript y Bootstrap, siguiendo la estructura existente de UVLHub.
- **Ejecución ordinaria en el navegador:** Pyodide y WebAssembly, junto con las dependencias empaquetadas para el generador.
- **Persistencia propia del generador:** No se incorpora almacenamiento permanente de configuraciones ni de modelos generados.
- **Entorno local y despliegue de revisión:** Docker y Docker Compose para el desarrollo local, y Render para ofrecer la aplicación a los tutores durante las revisiones.

6.2 Tecnologías utilizadas

Esta sección describe cómo se han utilizado las tecnologías seleccionadas y en qué componentes de la aplicación aparecen.

Backend y lógica del servidor

El backend de UVLHub está desarrollado en Python y Flask. Las rutas del módulo *generator* se encuentran en `app/features/generator/routes.py`. La lógica del asistente definida en `app/features/generator/wizard.py` reconstruye y comprueba los valores introducidos durante los pasos de configuración. La Figura 6.1 muestra la estructura del módulo.



Figura 6.1: Estructura del módulo generator de UVLHub. Elaboración propia.

El estado del asistente se mantiene de forma temporal en la sesión de Flask. Una vez consolidada la configuración, el endpoint `/generator/random/params-json` la proporciona en formato JSON. En la generación ordinaria, el navegador construye `FmgeneratorModel` mediante `from_flat_dict` y ejecuta el generador en Pyodide. Cuando se activa la opción de satisfacibilidad, el navegador envía los parámetros al endpoint `/generator/random/generate-sat`; el servidor genera los candidatos y los comprueba mediante PySAT.

Generador compatible con Flamapy

La lógica principal se ha implementado como un componente compatible con la arquitectura de extensiones de Flamapy. El directorio `fm_generator` contiene `FmgeneratorModel` y las clases que representan los diferentes bloques de configuración.

La operación `GenerateFeatureModel` recibe dicha configuración y construye un objeto `FeatureModel` por ejecución. Su método `execute` devuelve la propia operación y `get_result` permite recuperar el modelo obtenido. La transformación a UVL se realiza después mediante `UVLWriter`. La Figura 6.2 muestra la estructura de la herramienta.

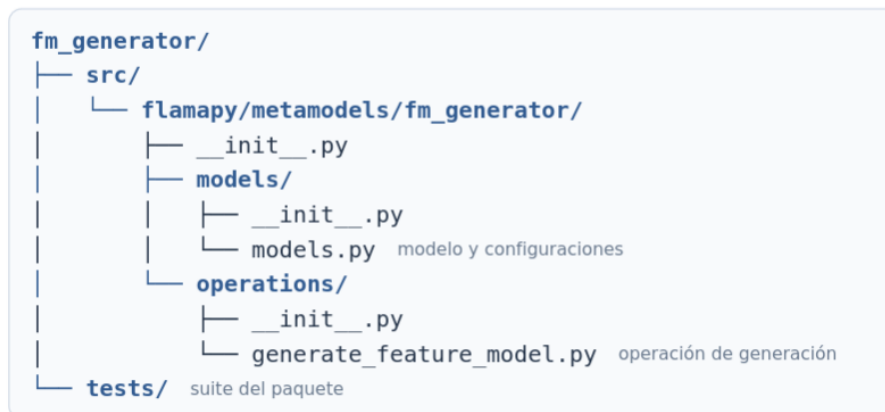


Figura 6.2: Estructura de la herramienta generadora. Elaboración propia.

Capa de presentación

La interfaz mantiene la arquitectura existente de UVLHub. Las vistas se definen con plantillas Jinja2; JavaScript gestiona la interacción y Bootstrap proporciona los componentes visuales. Los archivos de los seis pasos se encuentran en `app/features/generator/templates/generator/`, cuya estructura se muestra en la Figura 6.3.



Figura 6.3: Plantillas HTML utilizadas para los pasos del asistente. Elaboración propia.

Ejecución mediante Pyodide y WebAssembly

Pyodide [28] permite ejecutar Python en el navegador mediante WebAssembly [15]. En el flujo ordinario, JavaScript prepara el entorno, carga las dependencias necesarias y utiliza `fmgen_wrapper.py` para construir la configuración, generar cada modelo y convertirlo a UVL. Los scripts y paquetes *wheel* se encuentran en `app/features/generator/assets/js`. La Figura 6.4 muestra su estructura sin desplegar las dependencias empaquetadas ni los archivos del entorno WebAssembly.

```
assets/js/
├─ fmgen_wrapper.py  integra el modelo y la operación
├─ flamapy_ide.py    carga de operaciones
├─ scripts.js        inicializa Pyodide
├─ wizard.js         comportamiento del asistente
├─ wizard-spa.js     flujo de navegación
├─ webworker.js      ejecución en worker
├─ llm-page.js y llm-worker.js  integración de la página LLM
└─ webpack.config.js configuración de compilación
```

Figura 6.4: Scripts utilizados para preparar y ejecutar el generador mediante Pyodide. Elaboración propia.

La generación con comprobación SAT sigue el recorrido del servidor: allí se construye la configuración efectiva booleana, se generan los candidatos y se comprueba su satisfacibilidad. Los modelos aceptados se devuelven al navegador en formato JSON.

Infraestructura de desarrollo

Durante el desarrollo local se ha utilizado Docker [32] junto con Docker Compose para ejecutar el entorno de UVLHub. Los archivos de configuración se encuentran en el directorio `docker`. Para el desarrollo se emplean `docker/docker-compose.dev.yml` y `docker/images/Dockerfile.dev`; `docker/entrypoints` contiene, entre otros, los scripts `development_entrypoint.sh`, `production_entrypoint.sh`, `webhook_entrypoint.sh` y `worker_entrypoint.sh`.

También existen configuraciones de producción, como `docker/docker-compose.prod.yml` y `docker/docker-compose.prod.webhook.yml`. Estos archivos pertenecen a UVLHub y su presencia no implica que todos esos entornos se hayan desplegado durante el TFG.

La Figura 6.5 recoge la estructura del directorio.

```
docker/
├─ docker-compose.dev.yml  entorno de desarrollo
├─ docker-compose.prod.yml  entorno de producción
├─ docker-compose.prod.webhook.yml  servicio webhook
├─ docker-compose.prod.watchtower.yml  actualización de imágenes
├─ entrypoints/  scripts de arranque
├─ images/  definiciones de imágenes
├─ letsencrypt/  certificados
├─ nginx/  proxy web
├─ public/  recursos públicos
└─ redis/  servicio de sesión
```

Figura 6.5: Estructura del directorio `docker` de UVLHub. Elaboración propia.

Entre los servicios auxiliares se encuentra Redis, utilizado durante el despliegue de revisiones para conservar temporalmente la sesión de Flask. La generación ordinaria sigue ejecutándose en el navegador, mientras que la comprobación SAT y sus reintentos se realizan en el servidor.

Suite de pruebas

Las pruebas de UVLHub se ejecutan mediante *Rosemary*, una interfaz de línea de comandos incluida en el entorno de la aplicación [33]. La suite del módulo *generator* incluye comprobaciones del asistente, validación de parámetros, integración de la generación y pruebas de interfaz mediante Selenium. Su estructura aparece en la Figura 6.6.



Figura 6.6: Suite de pruebas del módulo *generator*. Elaboración propia.

La herramienta *fm_generator* dispone además de pruebas para su modelo de configuración y su operación de generación, organizadas como indica la Figura 6.7.



Figura 6.7: Suite de pruebas de *fm_generator*. Elaboración propia.

Automatización CI/CD

UVLHub dispone de flujos de GitHub Actions en su repositorio *upstream*. Sus comprobaciones pueden ejecutarse al presentar una *pull request* antes de una posible fusión. Estos flujos pertenecen a UVLHub y no fueron creados durante este TFG. Su

alcance y los comandos locales utilizados para verificar el generador se describen en la Sección 6.6.

6.3 Módulo de validación y lógica principal

La validación se organiza en dos niveles. Durante el asistente se comprueban los valores de cada paso y se mantiene el estado temporal en la sesión. Estas tareas corresponden a `GeneratorWizardService`. Tras construir la configuración completa, `FmgeneratorModel` comprueba las restricciones de dominio.

La operación `GenerateFeatureModel` recibe esta configuración y produce un `FeatureModel` por ejecución. Antes de generar el modelo, `execute` vuelve a llamar a la validación del objeto:

```
1 self.model.validate()
```

Esta comprobación protege la operación cuando se invoca directamente, tanto desde el navegador como desde el servidor. En el flujo satisfacible, el servidor construye una configuración efectiva limitada a construcciones booleanas y comprueba mediante SAT cada candidato generado.

6.3.1 Validación sintáctica y comprobaciones básicas

Las comprobaciones de los formularios se implementan principalmente en `wizard.py`, mediante funciones asociadas a los seis pasos del asistente:

```
1 validate_step1_form()
2 validate_step2_form()
3 validate_step3_form()
4 validate_step4_form()
5 validate_step5_form()
6 validate_step6_form()
```

Estas funciones detectan valores ausentes o inválidos antes de continuar con el asistente. También comprueban dependencias entre opciones y distribuciones introducidas por el usuario. La validación de la configuración completa se realiza posteriormente en `FmgeneratorModel`.

Validación de parámetros generales

En el primer paso se comprueban el número de modelos y la semilla:

$$1 \leq \text{NUM_MODELS} \leq 1000, \quad \text{SEED} \in \mathbb{Z}^+.$$

Validación de rangos numéricos

Los parámetros que representan cantidades positivas deben cumplir los límites definidos para cada campo. En los pares de mínimo y máximo se comprueba, de forma general:

$$\text{MIN_VALUE} \leq \text{MAX_VALUE}.$$

Esta relación se aplica, entre otros, al número de características, restricciones y atributos. Algunos campos admiten el valor cero, por lo que el límite inferior se comprueba según el significado de cada parámetro.

Validación de distribuciones probabilísticas

Las probabilidades utilizadas en una decisión aleatoria deben pertenecer al intervalo $[0, 1]$ y formar las distribuciones definidas para cada grupo de opciones:

$$\forall p_i \in P, \quad 0 \leq p_i \leq 1.$$

En la interfaz, la distribución de relaciones jerárquicas admite una diferencia de hasta 10^{-3} respecto a uno, como se especifica en RN-03. La validación semántica de `FmgeneratorModel` utiliza una tolerancia de 10^{-6} para comprobar las distribuciones normalizadas.

Validación de dependencias básicas

Algunas opciones requieren que se activen antes los niveles de expresividad correspondientes:

$$\text{FEATURE_CARDINALITY} \Rightarrow \text{ARITHMETIC_LEVEL}.$$

$$\text{AGGREGATE_FUNCTIONS} \Rightarrow \text{ARITHMETIC_LEVEL}.$$

$$\text{STRING_CONSTRAINTS} \Rightarrow \text{TYPE_LEVEL}.$$

Validación de atributos manuales

En el modo manual, cada atributo debe tener un nombre y una definición compatible con su tipo. Si se indica una probabilidad de asociación, esta debe estar comprendida entre cero y uno. Los dominios expresados mediante un rango deben cumplir que el límite inferior no supere al superior.

La inclusión de atributos numéricos en restricciones requiere el nivel aritmético. Para utilizar atributos de cadena en dichas restricciones deben estar activos el nivel de tipos y las restricciones sobre cadenas.

6.3.2 Validación semántica y restricciones de dominio

La validación semántica se encapsula en `FmgeneratorModel`, definido en `fm_generator`. La Tabla 6.1 resume los bloques que agrupan su configuración.

Configuración	Responsabilidad
<code>NamingConfig</code>	Nombres y opciones de salida
<code>LevelConfig</code>	Niveles de expresividad UVL
<code>FeaturesConfig</code>	Parámetros de características
<code>HierarchyConfig</code>	Estructura jerárquica
<code>ConstraintsConfig</code>	Restricciones generadas
<code>AttributesConfig</code>	Gestión de atributos

Tabla 6.1: Bloques de configuración de `FmgeneratorModel`. Elaboración propia.

El método `FmgeneratorModel.validate()` comprueba la coherencia global antes de iniciar la generación. Entre otras condiciones, verifica las dependencias entre niveles, los rangos numéricos y la normalización de las distribuciones con una tolerancia de 10^{-6} .

Coherencia de niveles UVL

El nivel booleano constituye la base de los modelos generados. La activación del nivel de tipos requiere el aritmético, y los niveles secundarios dependen de sus niveles principales:

`BOOLEAN_LEVEL = True,`

`TYPE_LEVEL ⇒ ARITHMETIC_LEVEL.`

`FEATURE_CARDINALITY ⇒ ARITHMETIC_LEVEL.`

`AGGREGATE_FUNCTIONS ⇒ ARITHMETIC_LEVEL.`

`STRING_CONSTRAINTS ⇒ TYPE_LEVEL.`

Validación de la jerarquía de características

La profundidad máxima se comprueba frente al número máximo de características:

$$1 \leq \text{MAX_TREE_DEPTH} \leq \text{MAX_FEATURES}.$$

Las probabilidades de selección de relaciones obligatorias, opcionales, alternativas, disyuntivas y de cardinalidad de grupo deben formar una distribución normalizada.

Validación de restricciones

La configuración valida las probabilidades de los tipos de restricciones y de los operadores disponibles. Una probabilidad positiva para restricciones numéricas requiere el nivel aritmético; las restricciones sobre cadenas requieren tanto el nivel de tipos como la opción correspondiente:

$$P(\text{Integer}) > 0 \vee P(\text{Real}) > 0 \Rightarrow \text{ARITHMETIC_LEVEL},$$

$$P(\text{String}) > 0 \Rightarrow \text{TYPE_LEVEL} \wedge \text{STRING_CONSTRAINTS}.$$

Para las restricciones booleanas, las probabilidades de conjunción, disyunción, implicación y equivalencia deben sumar uno. La probabilidad de negación se valida de forma independiente dentro del intervalo $[0, 1]$.

Las operaciones aritméticas se comprueban según las capacidades activadas. Cuando se permiten funciones agregadas, sus probabilidades se incorporan a la distribución utilizada para seleccionarlas. Los operadores de comparación se validan de forma separada. De esta forma, solo se consideran opciones compatibles con los niveles UVL habilitados.

Validación de atributos

En el modo aleatorio se comprueba el rango del número de atributos y la distribución de sus tipos:

$$0 \leq \text{MIN_ATTRIBUTES} \leq \text{MAX_ATTRIBUTES}.$$

En el modo manual se comprueba cada elemento de `attributes_list`, incluidos su dominio y, cuando se proporciona, `attach_probability`:

$$0 \leq \text{attach_probability} \leq 1.$$

La posibilidad de utilizar atributos en restricciones se valida de acuerdo con los niveles aritmético y de tipos activos.

6.3.2.1. Consumo por otros módulos

La instancia validada de `FmgeneratorModel` se utiliza para la generación. La operación `GenerateFeatureModel` construye la jerarquía, asigna los atributos y genera las restricciones. El archivo `tests/test_validators.py` incluye casos de rechazo de configuraciones inconsistentes, como distribuciones no normalizadas, dependencias incumplidas y rangos inválidos.

6.4 Módulo de generación de artefactos

La generación se encapsula en la operación `GenerateFeatureModel`, perteneciente a `fm_generator`. Recibe una instancia de `FmgeneratorModel` y produce un objeto `FeatureModel` en memoria:

`FmgeneratorModel` $\xrightarrow{\text{GenerateFeatureModel}}$ `FeatureModel`.

El resultado contiene la jerarquía de características, sus relaciones, atributos y restricciones en las estructuras del metamodelo de Flamapy. La serialización a UVL y la asignación del nombre del archivo se realizan fuera de la operación. Una invocación completa puede expresarse como:

```
1 operation = GenerateFeatureModel().execute(
2     config, index=index, attempt=attempt
3 )
4 feature_model = operation.get_result()
```

6.4.1 Lógica principal de generación de modelos

La lógica se implementa en `generate_feature_model.py`, dentro del directorio de operaciones de `fm_generator`. Los siguientes fragmentos muestran la coordinación del proceso, la inicialización de la aleatoriedad, la construcción de la jerarquía, la asignación de atributos y la creación de restricciones.

Coordinación de la operación y semilla

El método `execute` recibe la configuración y genera un único `FeatureModel` por llamada. Primero valida los parámetros; después construye la jerarquía, añade atributos y restricciones y guarda el resultado. La operación devuelve `self` y el modelo se recupera mediante `get_result`. El Código 6.1 muestra esta secuencia.

```
1 def execute(
2     self,
3     model: FmgeneratorModel,
```

```

4     index: int = 0,
5     attempt: int = 0,
6 ) -> Operation:
7
8     self.model = model
9
10    self.model.validate()
11    self._seed_generation(index, attempt)
12
13    fm, features = self._generate_hierarchy()
14    self._assign_attributes(features)
15    self._add_constraints(fm, features)
16
17    setattr(fm, "uvl_includes", self._build_uvl_includes())
18
19    self.result = fm
20    return self
21
22 def get_result(self) -> FeatureModel | None:
23     return self.result

```

Código 6.1: Coordinación de la generación en `GenerateFeatureModel`.

El archivo define las constantes del Código 6.2. La primera separa las semillas de distintos intentos; la segunda fija la probabilidad por defecto de incluir un atributo aleatorio entre los candidatos de una restricción.

```

1 SAT_SEED_STRIDE = 100000
2 RANDOM_ATTR_CONSTRAINT_PROB = 0.8

```

Código 6.2: Constantes utilizadas para las semillas y los atributos aleatorios.

La semilla aplicada a un modelo del lote se calcula como:

$$s_{\text{efectiva}} = s_{\text{base}} + i + 100000a,$$

donde i representa el índice del modelo y a el número de intento. El Código 6.3 muestra su implementación.

```

1 def _seed_generation(self, index: int, attempt: int) -> None:
2     seed_value = self.model.seed + index + (attempt * SAT_SEED_STRIDE)
3     random.seed(seed_value)

```

Código 6.3: Cálculo de la semilla para cada modelo e intento.

La misma configuración, semilla, versión del generador y condiciones de ejecución permiten repetir una ejecución. Los reintentos con distintos valores de `attempt` emplean secuencias pseudoaleatorias diferentes.

Construcción de la jerarquía

La raíz se denomina **F0**. El método `_generate_hierarchy` construye las demás características dentro del rango configurado, las distribuye entre los niveles permitidos por `max_tree_depth` y crea sus relaciones con los padres. El Código 6.4 muestra cómo selecciona los tipos de relación y conecta los elementos de un nivel.

```
1 def _select_relation_types(self, total: int) -> list[str]:
2     return random.choices(
3         population=["mand", "opt", "alt", "or", "group"],
4         weights=[
5             self.model.hierarchy.dist_mandatory,
6             self.model.hierarchy.dist_optional,
7             self.model.hierarchy.dist_alternative,
8             self.model.hierarchy.dist_or,
9             self.model.hierarchy.dist_group_cardinality,
10        ],
11        k=total,
12    )
13
14 def _add_relations_to_level(
15     self,
16     parents: list[Feature],
17     children: list[Feature],
18 ) -> None:
19     total = len(children)
20     relation_types = self._select_relation_types(total)
21     random.shuffle(relation_types)
22
23     pool = children[:]
24     parent_index = 0
25
26     while pool:
27         relation_kind = relation_types[parent_index % len(relation_types)]
28         parent = parents[parent_index % len(parents)]
29         parent_index += 1
30
31         size = self._determine_group_size(len(pool))
32         group = [pool.pop() for _ in range(size)]
33
34         relations = self._create_relation(parent, group, relation_kind)
35
36         for relation in relations:
37             parent.add_relation(relation)
38
39             for child in relation.children:
40                 child.parent = parent
```

41

```
self._maybe_apply_feature_cardinality(child)
```

Código 6.4: Selección y conexión de relaciones entre características.

`random.choices` obtiene tipos de relación según los pesos indicados por el usuario. Después se mezclan y se agrupan los hijos. Por ello, las probabilidades influyen en la selección, pero no garantizan un número exacto de relaciones de cada tipo. `_create_relation` establece los límites de cada relación: obligatoria $[1, 1]$, opcional $[0, 1]$, alternativa $[1, 1]$ para el grupo y disyuntiva $[1, n]$ para un grupo de n hijos. En la cardinalidad de grupo, los límites se eligen de acuerdo con el tamaño del grupo.

El método `_maybe_apply_feature_type` asigna el tipo a cada característica. Si el nivel de tipos está desactivado, las características son booleanas. Cuando se activa, el método evalúa los pesos configurados para los tipos booleano, entero, real y cadena, y elige entre los candidatos obtenidos. Si no resulta seleccionado ningún tipo alternativo, se conserva el booleano.

Asignación de atributos

`_assign_attributes` distingue entre generación aleatoria y definición manual. En el primer caso se decide el número de atributos y se reparten entre las características; en el segundo se recorren las definiciones introducidas por el usuario y se aplica su probabilidad de asociación. El Código 6.5 muestra la selección del modo.

```
1 def _assign_attributes(self, features: list[Feature]) -> None:
2     if self.model.attributes.random_attributes:
3         self._generate_random_attributes(features)
4     else:
5         self._assign_manual_attributes(features)
```

Código 6.5: Selección del modo de asignación de atributos.

En el modo aleatorio, cuando el nivel aritmético está activado y la distribución permite crear tipos numéricos, se reserva una cantidad inicial de atributos numéricos. Si $v_{\min}$ representa `min_vars_per_constraint`, r representa `extra_constraint_representativeness`, A el número de atributos solicitados y F las características disponibles, el cálculo utilizado es:

$$A_{\text{num}} = \min \left(\left\lceil \frac{v_{\min}}{r} \right\rceil, A, F \right).$$

Esta reserva proporciona candidatos para las restricciones numéricas. Los restantes tipos de atributo se eligen con `random.choices` según la distribución configurada. Sus dominios y valores se materializan mediante objetos `Domain`, `Range` y `Attribute` antes de asociarse a las características.

Construcción de restricciones

`_add_constraints` organiza las características y los atributos en conjuntos de candidatos booleanos, numéricos y de cadenas. En modo `ensure_satisfiable`, `_constraints_must_be_boolean_only` limita los candidatos a expresiones booleanas. Esta limitación permite aplicar el comprobador SAT, pero la satisfacibilidad se verifica posteriormente en el servidor.

`extra_constraint_representativeness` limita cuántas veces puede aparecer una característica en una restricción. El Código 6.6 muestra cómo se seleccionan variables respetando este máximo y el límite de características distintas que recibe el método.

```

1 def _sample_keys_with_ecr(
2     self,
3     groups: dict[str, list[str]],
4     target_occurrences: int,
5     max_repetitions: int,
6     max_features_param: int,
7     feature_usage: dict[str, int] | None = None,
8     selected_features: set[str] | None = None,
9 ) -> list[str] | None:
10     if target_occurrences < 1:
11         return None
12
13     feature_usage = {} if feature_usage is None else feature_usage
14     selected_features = set() if selected_features is None else selected_features
15
16     selected_keys: list[str] = []
17
18     for _ in range(target_occurrences):
19         allowed_feature_ids = [
20             feature_id
21             for feature_id in groups
22             if feature_usage.get(feature_id, 0) < max_repetitions
23             and (
24                 feature_id in selected_features
25                 or len(selected_features) < max_features_param
26             )
27         ]
28
29         if not allowed_feature_ids:
30             return None
31
32         feature_id = random.choice(allowed_feature_ids)
33         selected_keys.append(random.choice(groups[feature_id]))
34         feature_usage[feature_id] = feature_usage.get(feature_id, 0) + 1

```

```

35     selected_features.add(feature_id)
36
37     random.shuffle(selected_keys)
38     return selected_keys

```

Código 6.6: Muestreo de variables con límite de repeticiones.

Si g es el número de características candidatas, $f_{\text{máx}}$ el máximo de características distintas y r el máximo de repeticiones por característica, la capacidad calculada para ese conjunto es:

$$C_{\text{máx}} = \min(g, f_{\text{máx}}) r.$$

Cuando no hay suficientes candidatos para alcanzar el mínimo de variables exigido, se omite la restricción. En caso contrario, se construyen los predicados mediante nodos **Node** y operaciones **ASTOperation** del árbol de sintaxis abstracta. El Código 6.7 muestra cómo se incorpora una restricción construida al modelo.

```

1  if root is None:
2      continue
3
4  fm.ctcs.append(Constraint(name=f"ctc{index}", ast=AST(root)))

```

Código 6.7: Incorporación de una restricción al modelo de características.

`min_constraints` y `max_constraints` determinan el número de intentos de creación. El resultado puede contener menos restricciones efectivas cuando no existen candidatos compatibles o no se consigue construir un predicado válido.

Directivas de expresividad UVL

`_build_uvl_includes` reúne las directivas de los niveles secundarios activados. Si se habilitan a la vez la cardinalidad de característica y las funciones agregadas, se agrupan en **Arithmetic.***, como recoge el Código 6.8.

```

1  def _build_uvl_includes(self) -> list[str]:
2      includes: list[str] = []
3
4      if self.model.levels.group_cardinality:
5          includes.append("Boolean.group-cardinality")
6
7      feature_cardinality = self.model.levels.feature_cardinality
8      aggregate_functions = self.model.levels.aggregate_functions
9
10     if feature_cardinality and aggregate_functions:
11         includes.append("Arithmetic.*")
12     else:

```

```
13     if aggregate_functions:
14         includes.append("Arithmetic.aggregate-function")
15     if feature_cardinality:
16         includes.append("Arithmetic.feature-cardinality")
17
18     if self.model.levels.type_level and self.model.levels.string_constraints:
19         includes.append("Type.string-constraints")
20
21     return includes
```

Código 6.8: Construcción de las directivas `include` del resultado.

Estas directivas se guardan en el `FeatureModel` y se utilizan al transformarlo posteriormente a texto UVL mediante `UVLWriter`.

6.4.2 Síntesis del artefacto final generado

Preparación de la configuración

Al finalizar el asistente, el navegador obtiene los parámetros en formato JSON mediante `/generator/random/params.json`. Si el usuario solicita la generación ordinaria, `fmgen_wrapper.py` construye `FmgeneratorModel` dentro de Pyodide mediante `from_flat_dict`. Si solicita la comprobación de satisfacibilidad, envía los parámetros al servidor, donde se construye una configuración efectiva limitada al nivel booleano. Se desactivan los niveles aritmético y de tipos, sus niveles secundarios, la cardinalidad de grupo y la generación de atributos.

Generación del modelo intermedio

`GenerateFeatureModel` construye un `FeatureModel` por ejecución. En el navegador, cada resultado obtenido se prepara para su serialización. En el servidor, cada resultado es un candidato que se comprueba mediante PySAT. Si no es satisfacible, se genera otro candidato con un número de intento distinto, hasta un máximo de 20 intentos por modelo. Si se agotan los intentos, se produce `SatisfiableModelGenerationError` y no se devuelve el último candidato como resultado válido.

Transformación a formato UVL

`UVLWriter` transforma los objetos `FeatureModel` aceptados en texto UVL. La operación `GenerateFeatureModel` termina al producir el modelo en memoria; no realiza esta transformación.

Entrega del resultado

En el flujo ordinario, el navegador dispone directamente de los textos UVL generados. En el flujo satisfacible, el servidor devuelve los resultados aceptados en formato JSON. La interfaz permite visualizarlos y descargarlos, individualmente o agrupados en un ZIP según la opción disponible. Esta funcionalidad no añade almacenamiento permanente de los modelos.

6.5 Rutas HTTP y API del generador

6.5.1 Endpoints implementados

El módulo ofrece rutas para mostrar y procesar los formularios del asistente, además de endpoints que intercambian información en JSON.

Endpoints de entrada al generador

- **GET /generator:** inicia el asistente y muestra la página inicial.
- **GET /generator/random:** dirige al primer paso de la configuración para la generación aleatoria.

Endpoints del asistente de configuración

Cada paso admite peticiones **GET** para mostrar su formulario y **POST** para procesar los valores introducidos:

- **GET/POST /generator/random/step1:** número de modelos, semilla y opciones generales.
- **GET/POST /generator/random/step2:** niveles de expresividad UVL.
- **GET/POST /generator/random/step3:** estructura de características y relaciones jerárquicas.
- **GET/POST /generator/random/step4:** restricciones y operadores.
- **GET/POST /generator/random/step5:** atributos.
- **GET/POST /generator/random/step6:** opciones finales y presentación de la salida. La ejecución del generador se inicia posteriormente desde el navegador o mediante la ruta SAT.

Endpoints auxiliares y de generación

- **GET /generator/random/params-json:** proporciona al navegador la configuración consolidada del asistente en formato JSON.
- **POST /generator/random/summary-refresh/<step>:** actualiza el resumen de la configuración mostrado en el asistente.
- **POST /generator/random/generate-sat:** recibe los parámetros JSON, construye la configuración efectiva booleana y genera candidatos en el servidor. Comprueba cada candidato mediante PySAT, realiza hasta 20 intentos por modelo y devuelve en formato JSON los modelos satisfacibles. Si se agotan los intentos, devuelve el error correspondiente.

La generación ordinaria se ejecuta en Pyodide después de consultar `params-json`; no requiere una petición al servidor para generar cada `FeatureModel`.

6.5.2 Descarga de modelos UVL generados

Una vez obtenidos los textos UVL, la interfaz permite consultar los modelos y descargarlos, incluidos los lotes ofrecidos en ZIP. El servidor no conserva permanentemente los modelos generados. Para repetir la generación se utilizan la misma configuración, semilla, versión del generador y condiciones de ejecución.

6.6 Integración continua

El repositorio *upstream* de UVLHub utiliza GitHub Actions. Cuando se propone integrar cambios mediante una *pull request*, las comprobaciones configuradas allí sirven como control previo a una posible fusión. Los flujos de la Figura 6.8 pertenecen a UVLHub y no fueron desarrollados durante este TFG. La captura muestra sus nombres, pero no permite identificar sus disparadores ni los comandos de cada archivo `.yml`.

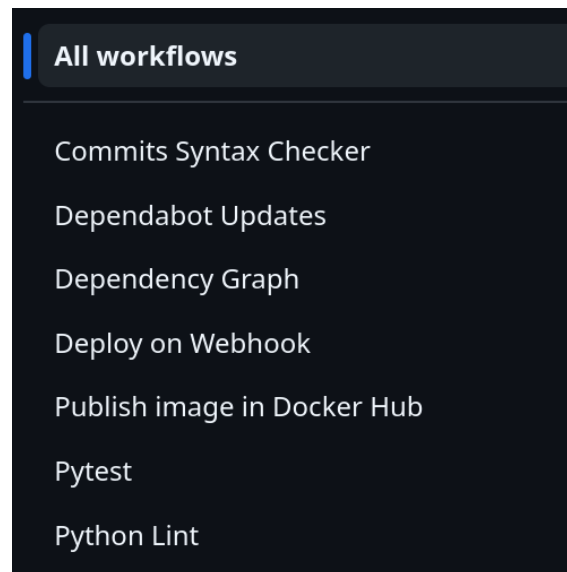


Figura 6.8: *Workflows* visibles en GitHub Actions del repositorio *upstream* de UVLHub. Captura facilitada por el autor.

En la captura aparecen *Commits Syntax Checker*, *Pytest* y *Python Lint*, relacionados por sus nombres con comprobaciones previas a la integración. También se muestran *Deploy on Webhook*, *Publish image in Docker Hub*, *Dependabot Updates* y *Dependency Graph*. Su presencia no demuestra que todos se ejecuten al abrir una *pull request* ni que las pruebas desplieguen automáticamente la versión desarrollada en este TFG.

Las comprobaciones de este trabajo se ejecutaron localmente. Desde la raíz del repositorio, los siguientes comandos permiten levantar el entorno de desarrollo, ejecutar la suite del módulo integrado, medir su cobertura y ejecutar las pruebas del paquete independiente:

```
1 docker compose -f docker/docker-compose.dev.yml up -d --build
2
3 docker compose -f docker/docker-compose.dev.yml exec web \
4     rosemary test generator --all
5
6 docker compose -f docker/docker-compose.dev.yml exec web \
7     rosemary coverage generator --all
8
9 pytest fm_generator/tests -v \
10     --cov=fm_generator \
11     --cov-report=term-missing \
12     --cov-report=html
```

El comando de cobertura de UVLHub también ejecuta sus pruebas, por lo que sus resultados no se suman como casos nuevos a los de **rosemary test**. Los resultados y el desglose de cobertura del código de producción se presentan en la Sección 7.5. El despliegue en Render descrito en el Apéndice C se utilizó para las revisiones de los tutores y no se considera una ejecución de los flujos de *upstream*.

La implementación descrita comprende la configuración del asistente, la generación ordinaria en Pyodide y la generación satisfacible en el servidor. El siguiente capítulo presenta las pruebas utilizadas para verificar el comportamiento de estos componentes.

7 PRUEBAS Y VALIDACIÓN

7.1 Estrategia de pruebas

La estrategia de pruebas aplicada tiene como objetivo verificar el correcto funcionamiento del generador integrado en UVLHub, validando tanto los componentes individuales como el flujo completo de configuración y generación.

La suite de pruebas utilizada combina trabajo previo y aportaciones realizadas durante este TFG. Las pruebas `test_engine.py`, `test_validators.py`, `test_wizard_outputs.py` y `test_wizard_flow.py` fueron desarrolladas mayoritariamente por David Romero. Por su parte, durante este trabajo se desarrollaron las pruebas `test_selenium.py`, `test_wizard_persisters.py`, `test_wizard_state.py`, `test_wizard_builders.py` y `test_params_contract.py`, además de adaptar pruebas existentes para validar la integración con el nuevo componente generador.

Las pruebas desarrolladas se organizan en los siguientes niveles:

- **Pruebas unitarias:** enfocadas en validar componentes aislados del sistema, incluyendo las reglas de validación, la construcción de configuraciones y la lógica interna del generador.
- **Pruebas de integración:** comprueban la interacción entre los distintos componentes del módulo `generator`, incluyendo la gestión del estado del asistente, la persistencia de parámetros y la generación de salidas.
- **Pruebas de sistema:** verifican el funcionamiento completo del flujo desde la perspectiva del usuario, incluyendo la interacción con el asistente de configuración mediante pruebas automatizadas.
- **Pruebas manuales:** se han realizado comprobaciones manuales del flujo completo de generación, verificando el comportamiento de la aplicación durante el desarrollo.

La suite integrada de UVLHub se ejecutó en el contenedor web de Docker con Python 3.13.15 y `pytest` 9.0.3. La suite independiente de `fm_generator` se ejecutó desde la raíz del repositorio con Python 3.13.11 y `pytest` 9.0.3. Esta separación permite verificar el módulo integrado con los servicios de UVLHub y el componente generador por separado.

Los comandos siguientes reproducen las ejecuciones documentadas en el Apéndice C. Se ejecutan desde la raíz del repositorio, con el entorno Docker previamente levantado mediante `docker compose -f docker/docker-compose.dev.yml up -build`. En la primera pareja se ejecutan las suites; en la segunda se generan los informes de cobertura correspondientes.

```
1 docker compose -f docker/docker-compose.dev.yml exec web \  
2     rosemary test generator --all  
3 pytest fm_generator/tests -v  
4  
5  
6  
7 docker compose -f docker/docker-compose.dev.yml exec web \  
8     rosemary coverage generator --all  
9 pytest fm_generator/tests -v --cov=fm_generator \  
10    --cov-report=term-missing --cov-report=html
```

La ejecución de una suite y su repetición con cobertura contienen los mismos tests; esta segunda ejecución no se contabiliza como un conjunto de casos adicional. Los casos de Selenium necesitan los servicios del entorno Docker.

7.2 Pruebas unitarias

Las pruebas de esta sección ejercitan las funciones de generación y las piezas del asistente sin recorrer la interfaz del navegador. Se describen por fichero para identificar los casos y la responsabilidad que comprueba cada uno.

7.2.1 `test_engine.py` del paquete `fm_generator`

Este fichero pertenece a la suite independiente de `fm_generator` y comprueba el comportamiento de `GenerateFeatureModel` y de los modelos producidos. `test_num_models_respected`, `test_feature_count_in_range` y `test_max_tree_depth_respected` verifican los límites configurados; `test_determinism_same_seed_same_output` y `test_different_seed_different_output` ejercitan el uso de semillas. Los casos sobre cardinalidades, atributos y restricciones contrastan la presencia o ausencia de las construcciones UVL según los niveles activados. Por ejemplo, `test_arithmetic_level_produces_arith_constraints` y `test_string_level_produces_len_constraints` comprueban salidas de mayor expresividad. La prueba `test_generate_feature_model_is_discoverable_by_flamapy` comprueba también el descubrimiento de la operación mediante Flamapy.

7.2.2 `test_validators.py`

Agrupar casos negativos para los formularios de los pasos 1 a 5 y un caso de aceptación de las opciones del paso 6. Entre los casos se encuentran `test_step1_rejects_non_numeric_values`, `test_step2_rejects_minor_levels_without_required_major_levels`, `test_step3_rejects_invalid_relation_distribution` y `test_step4_rejects_invalid_boolean_operator_distribution`. Las parametrizaciones comprueban varios valores incorrectos por caso, incluidos rangos invertidos, campos no numéricos y distribuciones inválidas, y verifican que se señalen los campos afectados.

7.2.3 `test_wizard_builders.py`

Los casos de este fichero verifican la reconstrucción de valores al abrir un paso del asistente. `test_build_step1_values_uses_saved_parameters` contrasta los datos recuperados con los guardados; `test_build_step3_normalises_feature_cardinality_values` comprueba la normalización de cardinalidades; y `test_build_step6_values_reads_output_options` verifica las opciones finales. También se prueban los valores por defecto y las entradas vacías o de distintos tipos.

7.2.4 `test_wizard_persisters.py`

Verifica la actualización de la configuración temporal al completar los formularios. `test_apply_step1_persists_batch_fields` comprueba los datos generales; `test_apply_step2_type_enables_arithmetic` comprueba la dependencia entre niveles; y `test_apply_step6_output_persists_checkboxes` comprueba las casillas de salida. Otros casos cubren distribuciones de relaciones y atributos, desactivación de probabilidades no aplicables y conversión de valores inválidos.

7.2.5 `test_wizard_state.py`

Sus cuatro funciones comprueban el guardado y la recuperación del estado de cada paso. `test_save_step_state_preserves_checkbox_false_values` verifica que los valores falsos no desaparecen; `test_save_step_state_does_not_overwrite_other_steps` comprueba el aislamiento entre pasos; y `test_step_state_is_preserved_after_returning_to_previous_step` reproduce la navegación hacia atrás.

7.3 Pruebas de integración

Esta sección reúne pruebas del contrato de parámetros y pruebas que combinan rutas Flask, sesión, configuración y componente generador. Las realizadas con el

cliente de pruebas de Flask recorren el contrato HTTP sin abrir un navegador; las pruebas con navegador real se describen en la Sección 7.4.

7.3.1 `test_params_contract.py` del paquete `fm_generator`

Este fichero verifica el contrato entre las claves del asistente y las admitidas por `FmgeneratorModel`. El caso `test_wizard_referenced_params_keys_are_mapped_to_generator_model` detecta claves sin correspondencia; `test_generator_model_defaults_are_valid_for_generation_contract` comprueba los valores por defecto. Los casos restantes verifican la forma de los parámetros y rechazan las distribuciones cuya suma se aleja de uno más allá de la tolerancia aplicada por el validador del modelo, de 10^{-6} . Sus cinco casos se superaron, al igual que los 39 del otro fichero de `fm_generator`.

7.3.2 `test_wizard_flow.py`

El cliente de Flask recorre las rutas `/generator/random/step1` a `/generator/random/step6` y comprueba las respuestas, las redirecciones y el estado de sesión. `test_full_happy_path_produces_valid_params` verifica que el recorrido proporciona una configuración completa; `test_params_json_returns_400_without_params` comprueba la respuesta cuando falta configuración; y los casos `test_back_nav_from_step2_preserves_level_flags` y `test_step4_configuration_survives_forward_and_back_navigation` comprueban la conservación de valores al navegar. También se prueban los errores de validación y la respuesta de rutas ante estados incompletos.

7.3.3 `test_wizard_outputs.py`

Estas pruebas recorren los seis pasos mediante el cliente Flask, recuperan `/generator/random/params-json` y utilizan esa configuración para generar modelos UVL. `test_boolean_only_wizard_produces_only_boolean`, `test_arithmetic_level_wizard_produces_arithmetic_constraints` y `test_group_cardinality_on_produces_groups` comprueban que las opciones del asistente afectan al resultado. Otros casos verifican el número de atributos y restricciones, las funciones de `fmgen_wrapper.py`, la serialización y los nombres de fichero. En particular, `test_o1_level_combinations_roundtrip_to_uvl_and_boolean_sat` ejercita 26 combinaciones de niveles, serializa los modelos a UVL y vuelve a cargarlos con `UVLReader`. En las configuraciones booleanas también comprueba la satisfacibilidad. Este caso verifica los modelos generados y su compatibilidad con las herramientas de lectura y análisis; la descripción de la suite no identifica una prueba de extremo a extremo que invoque el endpoint `/generator/random/generate-sat`.

Este fichero contiene también pruebas unitarias del *wrapper* que sustituyen algunas dependencias mediante objetos simulados; se agrupan aquí porque comparten el fichero con las pruebas de integración.

7.4 Pruebas de extremo a extremo

El fichero `test_selenium.py` contiene doce pruebas con Selenium WebDriver [34]. Comprueba la aplicación desde un navegador real, incluidas la respuesta de los formularios, la navegación y la preparación del entorno de Pyodide. Por ejemplo, `test_step2_type_level_enables_arithmetic_level_in_ui` verifica una dependencia visible entre opciones; `test_wizard_preserves_step1_values_after_next_and_previous` verifica la navegación; y `test_wizard_reaches_step5_with_pyodide_ready` comprueba la preparación del entorno de generación.

El caso más completo es `test_step6_generates_one_uvl_model_with_pyodide`. Recorre los seis pasos, espera a que el *runtime* esté disponible, solicita un modelo y comprueba que aparece un único resultado, que se ofrece la descarga y que la vista del archivo contiene la sección **features**. Otros casos comprueban los límites de campos numéricos, la disponibilidad de las pantallas y la aparición de opciones de cardinalidad o niveles UVL. Así, las doce pruebas no realizan todas una generación completa; cada una examina un tramo concreto del flujo de usuario.

La Tabla 7.1 relaciona cada fichero con sus casos ejecutados y una función de prueba representativa. Los casos se cuentan tras expandir las parametrizaciones: una función puede producir varias ejecuciones. Los ficheros de UVLHub se encuentran en `app/features/generator/tests/` y los del componente independiente en `fm_generator/tests/`.

Fichero de test	Pasan	Caso representativo
<i>Paquete independiente fm_generator</i>		
test_engine.py	39/39	test_determinism_same_seed_same_output
test_params_cont ract.py	5/5	test_wizard_referenced_params_keys_are_mapped_to_generat or_model
Subtotal	44/44	
fm_generator		
<i>Módulo generator de UVLHub</i>		
test_selenium.py	12/12	test_step6_generates_one_uvl_model_with_pyodide
test_validators. py	51/51	test_step3_rejects_invalid_relation_distribution
test_wizard_buil ders.py	16/16	test_build_step1_values_uses_saved_parameters
test_wizard_flow .py	35/35	test_full_happy_path_produces_valid_params
test_wizard_outp uts.py	90/90	test_o1_level_combinations_roundtrip_to_uvl_and_boolean_ sat
test_wizard_pers isters.py	12/12	test_apply_step1_persists_batch_fields
test_wizard_stat e.py	4/4	test_save_step_state_does_not_overwrite_other_steps
Subtotal UVLHub	220/220	

Tabla 7.1: Casos superados y ejemplos por fichero de las dos suites de pruebas.

7.5 Cobertura y métricas de calidad

La calidad de la solución se ha evaluado mediante pruebas automatizadas y cobertura de sentencias ejecutables, utilizando `pytest` y `coverage.py`. Se ha establecido un umbral mínimo del **80 %** sobre el código de producción.

Este umbral busca mantener una cobertura honesta. Las sentencias no cubiertas corresponden principalmente a comprobaciones defensivas, configuraciones inválidas bloqueadas por el asistente y rutas de error cuya ejecución forzada no aportaría evidencia funcional relevante.

El cálculo de la Tabla 7.2 considera únicamente el código de producción del generador en UVLHub y en `fm_generator`. Los informes completos incluyen también ficheros de prueba; estos no se incluyen en los subtotales de producción. Tampoco se suman dependencias externas, cachés ni artefactos generados.

Fichero	Sent.	Sin cubrir	Cobertura
<i>Módulo generator de UVLHub</i>			
<code>__init__.py</code>	2	0	100 %
<code>assets/js/fmgen_wrapper.py</code>	49	0	100 %
<code>routes.py</code>	214	6	97 %
<code>wizard.py</code>	810	159	80 %
<i>Paquete fm_generator</i>			
<code>__init__.py</code>	2	0	100 %
<code>models/__init__.py</code>	2	0	100 %
<code>models/models.py</code>	325	42	87 %
<code>operations/__init__.py</code>	2	0	100 %
<code>operations/generate_feature_model.py</code>	666	73	89 %
Subtotal UVLHub	1075	165	85 %
Subtotal fm_generator	997	115	88 %
Total de producción	2072	280	86,5 %

Tabla 7.2: Cobertura de sentencias del código de producción del generador.

El informe de UVLHub muestra una cobertura global del 91 %, pero dicho valor incluye también los ficheros de prueba. Al excluirllos, el código de producción del generador alcanza un 85 %. Del mismo modo, el informe del paquete `fm_generator` muestra un 91 % incluyendo sus tests, mientras que la cobertura específica del código de producción es del 88 %. La cobertura conjunta del 86,5 % se obtiene agregando las sentencias cubiertas de ambos informes independientes: 1792 de 2072.

Los ficheros con menor cobertura son `wizard.py`, con un 80 %, y `models.py`, con un 87 %. A pesar de ello, los caminos principales de validación, persistencia de parámetros, generación, serialización y lectura de modelos se encuentran cubiertos. El motor `GenerateFeatureModel`, responsable de construir los modelos, alcanza un 89 %.

La suite completa está formada por 220 pruebas de UVLHub y 44 pruebas del paquete `fm_generator`, todas superadas en sus respectivas ejecuciones. Las dos ejecuciones del paquete independiente no representan pruebas adicionales, sino una repetición de las mismas pruebas, una de ellas con medición de cobertura.

Conjunto de pruebas	N.º de pruebas	Resultado
Pruebas de UVLHub	220	220/220 pasan
Pruebas de <code>fm_generator</code>	44	44/44 pasan
Total de la suite	264	264/264 pasan

Tabla 7.3: Resumen de la suite automatizada de pruebas.

La suite combina pruebas unitarias sobre validadores, constructores, persistencia de parámetros y reglas del motor, con pruebas de integración del flujo del asistente y de la generación de modelos. Además, incluye 12 pruebas con Selenium para verificar los flujos principales desde la interfaz web.

Destaca una prueba automatizada que recorre 26 combinaciones válidas de niveles principales y secundarios. Para cada combinación se generan tres modelos, lo que supone 78 modelos en total. Cada modelo se serializa a UVL y se vuelve a cargar mediante `UVLReader`. En las configuraciones booleanas, además, el modelo se transforma a PySAT y se ejecuta la operación de satisfacibilidad. Esta prueba comprueba la compatibilidad del resultado con las herramientas que consumirán posteriormente los modelos generados. Sus 78 modelos corresponden a la matriz de combinaciones de las pruebas de regresión, y se diferencian de los 30 modelos del experimento cuantitativo descrito en la Sección 8.2.1.

La Tabla 7.3 resume el resultado de ambas suites. La relación detallada entre requisitos y pruebas se recoge en la Tabla 4.18. El capítulo siguiente utiliza estas evidencias, junto con las evaluaciones específicas, para valorar los objetivos del trabajo.

Este capítulo ha documentado los casos ejecutados, los resultados de las suites y la cobertura del código de producción. El Capítulo 8 contrasta esta evidencia con los criterios de éxito de los objetivos y presenta las evaluaciones que requieren medidas adicionales.

8 RESULTADOS Y DISCUSIÓN

8.1 Diseño de la evaluación

La evaluación se centra en comprobar la capacidad del generador para producir modelos UVL con diferentes niveles de expresividad, la influencia de los parámetros de configuración, su integración con Flamapy y UVLHub y el funcionamiento de la batería de pruebas automatizadas. Para ello, se ejecutaron pruebas funcionales, comprobaciones de compatibilidad con el parser UVL, evaluaciones de integración y mediciones temporales bajo las configuraciones descritas en las siguientes subsecciones.

La evaluación se realizó en el siguiente entorno:

- **Sistema operativo:** Ubuntu 24.04.4 LTS.
- **Navegador:** Firefox 148.0.2 (64 bits).
- **Entorno de ejecución del generador:** Pyodide 0.27.7, utilizando Python 3.12.7 sobre WebAssembly.
- **Contenedores:** Docker 29.7.2, empleado para ejecutar UVLHub durante las pruebas de integración.

De forma adicional, el equipo anfitrión utilizado fue un Slimbook Executive con un procesador Intel Core i7-13700H y 32 GB de memoria RAM.

8.2 Resultados

8.2.1 Resultados sobre el objetivo O1

El objetivo O1 consistía en desarrollar un generador automático capaz de producir modelos UVL con diferentes niveles de complejidad y expresividad, incluyendo construcciones avanzadas del lenguaje que permitan ampliar la variedad de modelos disponibles para el análisis de variabilidad.

Para verificar este objetivo se realizó una evaluación experimental basada en tres configuraciones acumulativas de expresividad. En cada configuración se generaron diez modelos utilizando la semilla fija **20260922**. Todos los modelos se cargaron posteriormente mediante **UVLReader** para comprobar su compatibilidad sintáctica con UVL.

La primera configuración incluía el nivel booleano y la cardinalidad de grupo, pero no generaba atributos. Estos modelos se sometieron además a una comprobación

de satisfacibilidad mediante PySAT con posterioridad a la generación, manteniendo desactivada la opción de generación condicionada por satisfacibilidad. De este modo, la proporción observada de modelos satisfacibles no queda alterada por reintentos del generador. Las configuraciones aritméticas y tipadas se utilizaron para validar la generación y el parseo de construcciones de mayor expresividad, pero no se incluyeron en la evaluación SAT.

La Figura 8.1 muestra la configuración de mayor expresividad utilizada en las ejecuciones de validación sintáctica, con la semilla **20260922**. No representa la ejecución booleana sometida a SAT: esta se realizó con la generación de atributos desactivada y con *Ensure satisfiable* desactivado. La evidencia cuantitativa se recoge en las Tablas 8.1 y 8.2.

YOUR SETUP	
1 · Batch	
Models	10
Seed	20260922
2 · Language levels	
Majors	Boolean, Arithmetic, Type
Minors	Group card., Feature card., Aggregates, String ctc
3 · Feature tree	
Features	10 – 15
Depth ≤	5
Rel mix	opt 20% · mand 21% · alt 22% · or 20% · group 17%
Feat card.	50% [2..5]
Group card.	[3..6]
4 · Constraints	
Count	10 – 10
Vars/ctc	4 – 10
Type mix	B 19% · Z 22% · R 29% · S 30%
5 · Attributes	
Mode	random
Count	5 – 10
Type mix	B 26% · Z 26% · R 25% · S 23%
6 · Output	
Ensure SAT	no
Suffixes	—

Figura 8.1: Configuración usada para verificar O1. Captura realizada por el autor.

El Apéndice A.1 recoge tres ejemplos ilustrativos de modelos generados y los parámetros utilizados en esas ejecuciones. Son ejecuciones independientes de la evaluación cuantitativa de O1, que empleó la semilla **20260922**. En particular, el modelo booleano del apéndice contiene un atributo, mientras que la configuración utilizada para la comprobación SAT se ejecutó sin atributos. Por tanto, los listados del apéndice no forman parte del recuento de treinta modelos ni de los resultados SAT de las tablas.

La siguiente Tabla 8.1 resume la validación sintáctica mediante **UVLReader**, mientras que la Tabla 8.2 presenta los resultados de la comprobación de satisfacibilidad. La columna “UVLReader OK” indica el número de modelos cargados correctamente mediante el parser de Flamapy. Las columnas relativas a SAT solo se aplican a la primera configuración, formada únicamente por construcciones booleanas y sin atributos. “SAT evaluados” recoge los modelos cuya comprobación terminó, “SAT satisfacibles” aquellos para los que PySAT devolvió un resultado positivo, “SAT no satisfacibles” aquellos para los que devolvió un resultado negativo y “SAT no concluyentes” los casos que no finalizaron dentro del tiempo disponible.

Configuración	Modelos	UVLReader OK	No parsean
Booleano y cardinalidad de grupo, sin atributos	10	10	0
Booleano, aritmética, cardinalidad de características y funciones agregadas	10	10	0
Booleano, aritmética y tipos, con todos los niveles secundarios	10	10	0
Total	30	30	0

Tabla 8.1: Validación sintáctica de los modelos generados para O1.

Configuración	Modelos generados	SAT evaluados	No satisfacibles	No concluyentes
Booleano y cardinalidad de grupo, sin atributos	10	9	1	8

Tabla 8.2: Comprobación SAT de la configuración booleana evaluada para O1.

El modelo restante de la configuración booleana se identificó como un caso no concluyente, ya que la comprobación SAT no finalizó dentro del tiempo máximo establecido para cada modelo (30 segundos). Por este motivo no se contabiliza como satisfacible ni como no satisfacible.

Los resultados muestran que los treinta modelos generados fueron aceptados por **UVLReader**, por lo que las tres configuraciones produjeron artefactos compatibles

sintácticamente con el lenguaje UVL. En la configuración booleana, la comprobación SAT terminó para nueve de los diez modelos: uno resultó satisfacible y ocho no satisfacibles. Que un modelo generado sea no satisfacible no implica que su generación sea incorrecta, ya que las restricciones aleatorias pueden ser incompatibles entre sí; precisamente por ello la comprobación SAT permite distinguir entre validez sintáctica y consistencia semántica.

La comprobación SAT se aplicó únicamente a la configuración booleana, por lo que los resultados obtenidos no permiten establecer cómo varía la satisfacibilidad al aumentar la complejidad de los modelos. Como consideración teórica, un mayor número de restricciones o de características puede aumentar el espacio de búsqueda del solucionador; sin embargo, su efecto sobre el tiempo de resolución y la existencia de soluciones depende de la estructura concreta del modelo. Para evaluar esta relación sería necesario realizar experimentos específicos con distintas configuraciones y tamaños.

Por tanto, los resultados permiten verificar el cumplimiento del criterio de éxito asociado al objetivo O1 en cuanto a la generación de modelos con diferentes niveles de expresividad y a su compatibilidad sintáctica con UVL. La comprobación SAT aporta además evidencia sobre la consistencia semántica de la configuración booleana evaluada, aunque no se ha aplicado a las configuraciones con construcciones aritméticas, tipadas y de cadenas.

8.2.2 Resultados sobre el objetivo O2

El objetivo O2 consistía en desarrollar un sistema de configuración flexible que permitiera modificar los parámetros principales del proceso de generación y adaptar los modelos obtenidos a distintos escenarios.

Para verificar este objetivo se ejecutaron pruebas que comprueban tanto la propagación de los parámetros a través del asistente de configuración como su efecto sobre la estructura y el contenido de los modelos UVL generados. La Tabla 8.3 resume las comprobaciones realizadas.

Funcionalidad	Prueba automatizada	Comprobación realizada
Propagación de la configuración	<code>test_step2_configuration_is_propagated_to_step3</code>	Comprueba que las opciones seleccionadas en el segundo paso se trasladan correctamente al siguiente.
Cardinalidad de grupo	<code>test_group_cardinality_off_produces_no_group_relations</code> <code>test_group_cardinality_on_with_weight_produces_group_groups</code>	Verifica la ausencia o generación de relaciones de cardinalidad de grupo.
Cardinalidad de características	<code>test_feature_cardinality_off_produces_none</code> <code>test_feature_cardinality_on_produces_some</code>	Comprueba que la opción modifica la salida UVL generando o eliminando cardinalidades.
Niveles aritmético y de tipos	<code>test_arithmetic_level_produces_arith_constraints</code> <code>test_string_level_produces_len_constraints</code>	Verifica la generación de expresiones aritméticas y restricciones mediante <code>len</code> .
Distribución de atributos	<code>test_dist_boolean_only_produces_only_boolean_attrs</code> <code>test_dist_integer_only_produces_integer_attrs</code> <code>test_real_attribute_distribution_generates_real_attributes</code>	Comprueba que las distribuciones configuradas determinan los tipos de atributos generados.
Cantidad de restricciones y variables	<code>test_min_max_constraints_respected</code> <code>test_min_vars_per_constraint_respected</code>	Comprueba los límites aplicados durante la generación; el número efectivo de restricciones puede ser inferior al mínimo configurado si no hay variables compatibles para construirlas.
Semillas	<code>test_determinism_same_seed_same_output</code> <code>test_different_seed_different_output</code>	Verifica la reproducibilidad con la misma semilla y la variación con semillas diferentes.

Tabla 8.3: Pruebas técnicas del efecto de los parámetros de configuración.

Las pruebas anteriores no se limitan a comprobar que el asistente acepta los valores introducidos, sino que verifican su efecto sobre los modelos generados. Por ejemplo, al activar la cardinalidad de grupo se generan las relaciones correspondientes, mientras que al activar el nivel aritmético aparecen expresiones numéricas en las restricciones.

De esta forma, existe una relación verificable entre la configuración seleccionada, el comportamiento del generador y el contenido del modelo UVL obtenido.

El criterio de éxito asociado al objetivo O2 se considera cumplido, ya que las pruebas confirman tanto la correcta propagación de los parámetros por el flujo de configuración como su influencia efectiva en los modelos generados.

8.2.3 Resultados sobre el objetivo O3

El objetivo O3 consistía en integrar el generador dentro del ecosistema Flamapy y permitir su utilización desde UVLHub como herramienta de generación de modelos de características.

La integración se verifica mediante el flujo real de ejecución utilizado por la aplicación. Los parámetros almacenados por UVLHub se convierten en una instancia de `FmgeneratorModel` mediante `from_flat_dict`. Posteriormente, el *wrapper* importa directamente la operación `GenerateFeatureModel`, crea una instancia y la ejecuta con el modelo de configuración recibido. Como resultado se obtiene un `FeatureModel` compatible con el metamodelo de Flamapy.

De forma independiente, la prueba `test_generate_feature_model_is_discoverable_by_flamapy` verifica que Flamapy también localiza esta operación mediante `DiscoverMetamodels`. Así, se comprueba tanto su uso efectivo desde UVLHub como su registro como operación descubrible dentro del ecosistema Flamapy.

Esta integración se comprobó mediante las pruebas recogidas en la Tabla 8.4.

Prueba	Evidencia obtenida
<code>test_wrapper_generates_feature_model_with_received_arguments</code>	Comprueba que el <i>wrapper</i> recibe los parámetros y genera un modelo de características.
<code>test_wrapper_build_one_model_delegates_to_feature_model_generator</code>	Verifica que la generación se delega al componente <code>GenerateFeatureModel</code> .
<code>test_wrapper_serializes_uvl_and_reads_includes</code>	Comprueba la serialización del modelo mediante Flamapy y la conservación de las declaraciones <code>include</code> .
<code>test_generate_feature_model_is_discoverable_by_flamapy</code>	Comprueba que <code>DiscoverMetamodels</code> localiza la operación <code>GenerateFeatureModel</code> , que esta puede ejecutarse y que devuelve un <code>FeatureModel</code> válido.

Tabla 8.4: Pruebas técnicas de integración con Flamapy y UVLHub.

La evidencia anterior demuestra que el componente no se limita a estar presente en la estructura del proyecto, sino que es utilizado durante la ejecución, produce un `FeatureModel` de Flamapy y genera modelos UVL que pueden ser consumidos posteriormente por otras herramientas del ecosistema.

Por tanto, el criterio de éxito asociado al objetivo O3 se considera cumplido.

8.2.4 Resultados sobre el objetivo O4

El objetivo O4 consistía en evaluar el comportamiento temporal del generador en escenarios con distintos volúmenes y tamaños de modelos de características.

Para ello se definieron tres perfiles de tamaño. En cada perfil se fijaron los valores mínimo y máximo configurados para las características, los atributos y las restricciones. El resto de parámetros, incluidos los niveles de expresividad UVL y las distribuciones empleadas, se mantuvo constante. Estos perfiles fijan los valores solicitados al generador, pero no garantizan que el número de restricciones incorporadas sea siempre idéntico, ya que alguna puede omitirse si no se consigue construir con las variables disponibles.

La Tabla 8.5 recoge los valores configurados para cada perfil.

Perfil	Características configuradas	Atributos configurados	Restricciones configuradas
Pequeño	10	5	10
Mediano	50	25	50
Grande	100	50	100

Tabla 8.5: Perfiles de tamaño utilizados en la evaluación temporal.

Para cada perfil se generaron lotes de 100, 500 y 1000 modelos UVL. Cada una de las nueve combinaciones se ejecutó tres veces con la misma configuración y semilla. El tiempo de generación se midió una vez que Pyodide y las dependencias necesarias estaban disponibles, por lo que no incluye la carga inicial del entorno ni el renderizado o la descarga de los resultados.

La carga de Pyodide y de las dependencias se midió de forma independiente en diez ejecuciones. Esta separación permite distinguir el coste inicial de preparación del entorno del coste propio de generar los modelos.

La Tabla 8.6 muestra la media y la desviación típica del tiempo de preparación medidas en diez ejecuciones.

Fase	Repeticiones	Media (s)	Desv. típica (s)
Carga de Pyodide y dependencias	10	4,28	0,24

Tabla 8.6: Tiempo de preparación del entorno de ejecución.

La Tabla 8.7 recoge la media y la desviación típica muestral de los tiempos de generación. Cada resultado se ha calculado a partir de tres repeticiones.

Tamaño	Modelos	Media (s)	Desv. típica (s)
Pequeño	100	0,60	0,02
Pequeño	500	2,90	0,21
Pequeño	1000	5,86	0,23
Mediano	100	1,89	0,16
Mediano	500	8,76	0,31
Mediano	1000	17,52	0,31
Grande	100	3,77	0,04
Grande	500	18,47	0,30
Grande	1000	36,95	0,28

Tabla 8.7: Tiempo de generación de modelos UVL según el tamaño del modelo y el volumen del lote. Cada resultado corresponde a tres repeticiones.

Los resultados muestran que, para un mismo perfil, el tiempo de generación aumenta de forma aproximadamente proporcional al número de modelos solicitado. Por ejemplo, en el perfil pequeño se pasó de 0,60 segundos para 100 modelos a 5,86 segundos para 1000 modelos. Este comportamiento se mantiene en los tres tamaños evaluados.

El perfil estructural también se asocia a diferencias en el tiempo de generación. Para lotes de 1000 modelos, la media aumentó de 5,86 segundos en el perfil pequeño a 17,52 segundos en el mediano y 36,95 segundos en el grande. Esta comparación refleja el efecto conjunto de los perfiles, ya que aumentan simultáneamente las características, los atributos y las restricciones; no permite atribuir el incremento a uno solo de estos parámetros.

La variabilidad entre repeticiones fue reducida. La mayor desviación típica registrada fue de 0,31 segundos, obtenida en los lotes medianos de 500 y 1000 modelos. Los resultados son, por tanto, consistentes en el entorno evaluado.

La carga de Pyodide y de sus dependencias supuso un coste inicial medio de 4,28 segundos, independiente del tamaño y del número de modelos del lote. Una vez

preparado el entorno, los tiempos de la Tabla 8.7 representan exclusivamente el coste de producir los modelos UVL.

Por tanto, el objetivo O4 se considera cumplido, ya que se ha cuantificado el comportamiento temporal del generador en nueve escenarios que combinan tres volúmenes de generación y tres tamaños de modelo, diferenciando el coste de preparación del entorno del proceso de generación.

8.2.5 Resultados sobre el objetivo O5

El objetivo O5 consistía en validar el correcto funcionamiento del generador mediante una batería de pruebas automatizadas que cubriese tanto la lógica interna del motor de generación como el flujo de configuración integrado en UVLHub.

El Capítulo 7 documenta los casos de las dos suites, sus comandos de ejecución y su cobertura. La Tabla 7.1 recoge los casos superados por fichero, la Tabla 7.2 muestra la cobertura de sentencias del código principal, y la Tabla 7.3 recoge el resultado global de las pruebas. Las suites incluyen pruebas preexistentes y pruebas desarrolladas o adaptadas durante este TFG, como se indica en la Sección 7.1.

El criterio de éxito asociado a O5 se considera cumplido para los escenarios cubiertos por la batería automatizada: las dos suites finalizaron sin pruebas fallidas. Este resultado permite valorar la validación realizada, sin extender la conclusión a comportamientos que no formen parte de los casos probados.

8.2.6 Resultados sobre los objetivos formativos O6–O10

Los objetivos O6–O10 se refieren a competencias adquiridas durante el desarrollo del proyecto. A diferencia de los objetivos funcionales, no se evalúan mediante una métrica de rendimiento, sino mediante evidencias documentales y técnicas generadas durante el trabajo.

Obj.	Evidencia	Evidencia documentada
O6	Estudio del enfoque de generación propuesto por Sundermann et al. y adaptación de sus bases conceptuales al componente implementado.	El conocimiento científico revisado se ha aplicado en el diseño y desarrollo del generador.
O7	Integración de un componente nuevo en los proyectos Flamapy y UVLHub, utilización de Docker y adaptación de pruebas existentes.	El desarrollo se ha realizado sobre un ecosistema software real, con código previo y componentes reutilizables.
O8	Implementación en Python, adaptación de rutas y asistentes de Flask, ejecución mediante Docker y desarrollo de pruebas automatizadas.	Las tecnologías principales del proyecto se han utilizado de forma práctica durante la construcción e integración de la solución.
O9	Consulta de bibliografía científica, documentación técnica de Flamapy, UVL y Pyodide, y justificación de decisiones de diseño.	Las decisiones técnicas se han apoyado en información seleccionada y analizada durante el desarrollo.
O10	Planificación del trabajo, gestión de riesgos, documentación de diseño, implementación, pruebas y resultados.	El proyecto se ha gestionado y documentado de forma continuada durante sus distintas fases.

Tabla 8.8: Evidencias asociadas a los objetivos formativos. Elaboración propia.

La Tabla 8.8 documenta la aplicación de los objetivos formativos durante el desarrollo del proyecto. Estas evidencias no permiten medir de forma cuantitativa el

grado individual de adquisición de cada competencia, por lo que los resultados deben interpretarse como evidencia documental de su aplicación y no como una medición objetiva de su dominio.

8.3 Comparación con trabajos previos

La Tabla 2.2 del Capítulo 2 recoge una comparación cualitativa entre diferentes herramientas relacionadas con la generación automática de modelos de características. En ella se consideran tres aspectos principales: la capacidad de generación automática, el grado de personalización ofrecido y la incorporación de construcciones de expresividad avanzada.

Los resultados obtenidos durante la evaluación del generador desarrollado permiten observar que la solución propuesta cubre los tres criterios analizados. A diferencia de S.P.L.O.T. y FeatureIDE, cuyas capacidades de generación están principalmente orientadas a modelos basados en construcciones booleanas, el generador desarrollado permite producir modelos UVL incorporando elementos adicionales como atributos tipados, restricciones aritméticas, restricciones sobre cadenas, funciones agregadas y diferentes mecanismos de cardinalidad.

En comparación con BeTTY, la solución desarrollada mantiene la capacidad de configuración flexible de los parámetros de generación, pero amplía el rango de expresividad de los modelos generados. Mientras que BeTTY permite definir escenarios experimentales con diferentes niveles de complejidad estructural, el generador desarrollado incorpora además construcciones propias de lenguajes de variabilidad más recientes como UVL.

Además, la integración realizada dentro del ecosistema Flamapy y UVLHub supone una diferencia respecto a las herramientas analizadas, ya que permite utilizar el generador directamente dentro de una plataforma web orientada a la gestión y análisis de modelos de características, manteniendo la compatibilidad con las herramientas existentes del ecosistema.

No se ha realizado una comparación cuantitativa directa en términos de tiempo de generación o volumen máximo de modelos, debido a que las herramientas estudiadas no proporcionan escenarios de evaluación homogéneos que permitan establecer una comparación justa, por lo que la comparación se ha centrado en las capacidades funcionales y de expresividad.

En consecuencia, el trabajo desarrollado destaca por la generación de modelos con mayor expresividad y por su integración dentro del ecosistema UVLHub y Flamapy. No obstante, esta ventaja funcional no implica una superioridad global frente a las herramientas analizadas, ya que algunas de ellas presentan una mayor madurez,

una validación experimental más extensa o métricas comparativas que no se han reproducido en este trabajo.

8.4 Discusión

Los resultados obtenidos muestran que la solución desarrollada responde al problema planteado inicialmente: la necesidad de disponer de un generador capaz de producir modelos de características con una mayor expresividad que las aproximaciones tradicionales basadas principalmente en construcciones booleanas. La integración del generador dentro del ecosistema Flamapy y UVLHub permite disponer de una herramienta flexible para la generación de modelos UVL destinados a tareas posteriores de análisis y experimentación.

Durante el desarrollo se encontraron diferentes dificultades relacionadas con la integración y adaptación de los componentes existentes. Sin embargo, la evaluación confirmó el funcionamiento de las funcionalidades en los escenarios y combinaciones cubiertos por las pruebas, incluyendo los niveles de expresividad y los parámetros examinados.

La incorporación de construcciones avanzadas implica ciertos compromisos. Por un lado, aumentar la expresividad mediante atributos tipados, restricciones aritméticas, funciones agregadas o restricciones sobre cadenas incrementa la complejidad interna del generador. Por otro lado, ofrecer una configuración más flexible permite adaptar la generación a distintos escenarios, aunque aumenta el número de parámetros que el usuario debe comprender. Asimismo, una mayor riqueza en los modelos generados puede suponer un incremento del coste computacional en configuraciones más complejas.

Finalmente, existen algunas amenazas a la validez de los resultados. La evaluación confirma la compatibilidad sintáctica de los modelos generados con UVL y con las transformaciones empleadas, pero no determina si dichos modelos son representativos de dominios reales ni si resultan adecuados como conjunto de evaluación para motores de razonamiento. Asimismo, los resultados temporales se han obtenido con una única configuración de generación y en un entorno concreto, por lo que podrían variar al modificar el tamaño o la complejidad de los modelos. Tampoco se ha realizado una comparación cuantitativa directa con otras herramientas bajo condiciones experimentales equivalentes.

Este capítulo ha contrastado la evidencia disponible con los criterios de éxito y ha discutido el alcance de los resultados obtenidos. El Capítulo 9 sintetiza la aportación del trabajo, sus limitaciones y las líneas de desarrollo que se derivan de ellas.

9 CONCLUSIONES Y TRABAJO FUTURO

9.1 Conclusiones

La aportación principal de este trabajo es un generador de modelos de características expresados en UVL, integrado con Flamapy y accesible desde UVLHub. La solución permite configurar la estructura y el contenido de los modelos generados e incorporar construcciones de los niveles booleano, aritmético y de tipos.

La Sección 8.2 recoge la valoración individual de los objetivos a partir de sus respectivas evidencias. En conjunto, las pruebas de compatibilidad, la evaluación temporal y la ejecución de las suites automatizadas respaldan el funcionamiento de la solución en los escenarios estudiados. El alcance de esta conclusión depende de las configuraciones y condiciones de evaluación descritas en el capítulo anterior.

El desarrollo también ha puesto de manifiesto la importancia de trabajar con un entorno reproducible, incorporar pruebas durante la integración y planificar margen para las dificultades derivadas de adaptar componentes existentes. La solución constituye una base para ampliar la generación de modelos y estudiar su utilidad en la evaluación de motores de razonamiento.

9.2 Limitaciones

A pesar de los resultados obtenidos, existen ciertas limitaciones que deben tenerse en cuenta al interpretar el alcance del trabajo realizado. Estas limitaciones delimitan el alcance de las conclusiones obtenidas y los escenarios en los que se ha validado la solución.

Además, los objetivos formativos O6–O10 se han valorado mediante evidencias documentales de las actividades y decisiones realizadas durante el proyecto. Al no haberse definido métricas para medir la adquisición individual de esas competencias, su grado de adquisición queda sin cuantificar.

Desde un punto de vista técnico, la evaluación del generador se ha realizado sobre configuraciones representativas, pero no se ha estudiado su comportamiento en escenarios extremos con modelos de características de gran tamaño o con una complejidad significativamente superior. Esto puede afectar al coste computacional en configuraciones con un número elevado de características, atributos o restricciones. No obstante, la arquitectura desarrollada permite continuar optimizando y ampliando el generador para este tipo de escenarios.

Respecto a la validación, las pruebas realizadas permiten comprobar que los modelos generados se serializan correctamente en UVL, pueden volver a cargarse mediante **UVLReader** y, en las configuraciones booleanas, pueden transformarse a PySAT. Estas comprobaciones aportan evidencia sobre la validez sintáctica y la compatibilidad de los modelos con las herramientas del ecosistema. Sin embargo, no se ha realizado una evaluación específica de la calidad de los modelos generados en términos de consistencia semántica, representatividad respecto a dominios reales o utilidad práctica para evaluar motores de razonamiento. Tampoco se ha comparado el comportamiento de distintos razonadores utilizando los modelos producidos como conjunto de evaluación.

Finalmente, en cuanto al alcance del trabajo, la solución desarrollada se ha centrado en la generación automática de modelos UVL y su integración dentro del ecosistema Flamapy y UVLHub. No se han abordado otras líneas relacionadas, como la optimización avanzada del proceso de generación, nuevas estrategias de búsqueda o análisis posterior de los modelos generados, ya que quedan fuera de los objetivos definidos inicialmente.

9.3 Trabajo futuro

A partir de las limitaciones identificadas y de las posibilidades de extensión del sistema desarrollado, se plantean diferentes líneas de trabajo futuro orientadas a mejorar la reutilización de configuraciones, la integración con UVLHub y la capacidad del generador para adaptarse a nuevos escenarios de uso.

Exportación e importación de configuraciones

Una primera línea de trabajo consistiría en desarrollar una transformación de Flamapy que permita exportar la configuración seleccionada del generador en un formato estándar como JSON, permitiendo posteriormente importar dicha configuración desde UVLHub para reproducir una generación concreta.

Esta extensión mejoraría la reproducibilidad de los experimentos y facilitaría el intercambio de configuraciones entre usuarios, evitando la necesidad de definir manualmente conjuntos complejos de parámetros. Además, permitiría compartir escenarios de generación de forma sencilla dentro del ecosistema de herramientas de variabilidad.

La implementación de esta funcionalidad presenta una viabilidad elevada, ya que los parámetros del generador ya se encuentran estructurados internamente. Sería necesario únicamente definir un esquema estable de serialización y añadir los mecanismos de exportación e importación correspondientes.

Publicación directa de modelos en UVLHub

Otra posible extensión consistiría en ampliar la integración con UVLHub para permitir subir directamente los modelos generados a la plataforma, pudiendo establecer su visibilidad y compartirlos con otros usuarios.

Actualmente, el generador permite producir modelos UVL, pero la gestión posterior de estos modelos requiere pasos adicionales. La incorporación de esta funcionalidad permitiría centralizar el flujo completo, desde la generación hasta el almacenamiento y distribución de modelos de características.

Esta línea presenta una viabilidad media-alta, ya que requeriría ampliar la comunicación entre el generador y los servicios existentes de UVLHub, además de definir aspectos relacionados con permisos, almacenamiento y gestión de modelos públicos.

Validación de representatividad y evaluación con razonadores

Una línea de trabajo futuro consistiría en evaluar la calidad de los modelos generados tomando como referencia colecciones de modelos de características procedentes de dominios reales. El objetivo no sería utilizar estos modelos como entrada del generador, sino comparar sus propiedades estructurales y de expresividad con las de los modelos producidos, por ejemplo en número de características, profundidad, relaciones, atributos y restricciones.

Esta evaluación permitiría determinar hasta qué punto las configuraciones del generador producen modelos representativos de escenarios reales. Asimismo, permitiría definir perfiles de generación basados en características observadas en conjuntos de referencia, aumentando la utilidad de la herramienta para la experimentación.

Como paso posterior, los modelos generados podrían utilizarse para evaluar motores de razonamiento bajo condiciones controladas. Esto permitiría analizar el impacto de diferentes niveles de expresividad sobre operaciones como la comprobación de satisfacibilidad, el cálculo de configuraciones o las transformaciones entre metamodelos.

Optimización del proceso de generación

Una posible línea de trabajo futuro consistiría en optimizar el proceso interno de generación para mejorar el rendimiento en configuraciones con una mayor complejidad estructural o un volumen elevado de modelos generados.

Aunque los resultados obtenidos muestran un comportamiento adecuado en los escenarios evaluados, el coste computacional puede incrementarse al aumentar el número de características, atributos o restricciones. Por ello, sería interesante analizar los puntos críticos del proceso de generación y aplicar técnicas de optimización, como

mejoras en la gestión de estructuras internas, reducción de operaciones redundantes o paralelización de determinadas etapas.

Esta extensión presenta una viabilidad media-alta, ya que no requiere modificar la arquitectura general del sistema, sino realizar mejoras sobre los componentes existentes del motor de generación. La principal dificultad reside en identificar aquellas operaciones que tienen mayor impacto sobre el tiempo de ejecución y mantener la corrección de los modelos generados tras las optimizaciones.

Generación guiada mediante aprendizaje automático

Como línea de investigación más ambiciosa, se plantea la incorporación de técnicas de aprendizaje automático para aprender patrones presentes en modelos de características reales y utilizarlos durante el proceso de generación.

Esta aproximación permitiría evolucionar desde una generación basada únicamente en parámetros definidos manualmente hacia una generación capaz de producir modelos sintéticos adaptados a las características de dominios concretos. De esta forma, se podrían obtener modelos más realistas para tareas de evaluación y experimentación.

Su viabilidad es media, ya que requeriría disponer de colecciones de referencia adecuadas, definir métricas de comparación y establecer una metodología experimental para evaluar los razonadores en condiciones equivalentes. No obstante, constituye una posible evolución innovadora del generador desarrollado.

BIBLIOGRAFÍA

- [1] J. A. Zamudio. *Líneas de Producto Software para acelerar el desarrollo de aplicaciones relacionadas*. 2023. URL: <https://ingenieriadesoftware.es/lineas-producto-software/> (visitado 05-08-2026).
- [2] A. Felfernig, A. Falkner y D. Benavides. *Feature Models. AI-Driven Design, Analysis and Applications*. 1.^a ed. Springer, 2024. ISBN: 978-3-031-61874-1.
- [3] D. Benavides, C. Sundermann, K. Feichtinger, J. A. Galindo, R. Rabiser y T. Thüm. «UVL: Feature modelling with the Universal Variability Language». En: *Journal of Systems and Software* 225, 112326 (2025). DOI: **10.1016/j.jss.2024.112326**.
- [4] J. A. Galindo, J.-M. Horcas, A. Felfernig, D. Fernandez-Amoros y D. Benavides. «FLAMA: A Collaborative Effort to Build a New Framework for the Automated Analysis of Feature Models». En: *SPLC '23: Proceedings of the 27th ACM International Systems and Software Product Line Conference*. Vol. B2. Association for Computing Machinery, 2023, págs. 16-19. DOI: **10.1145/3579028.3609008**.
- [5] *TASOVA Plus: Red en nuevas Tendencias en Arquitectura Software y Variabilidad*. 2023. URL: <https://tasovaplus.github.io/> (visitado 05-08-2026).
- [6] D. Romero-Organvitez, J. A. Galindo, C. Sundermann, J.-M. Horcas y D. Benavides. «UVLHub: A feature model data repository using UVL and Open Science principles». En: *Journal of Systems and Software* 216 (2024), pág. 112150. DOI: **10.1016/j.jss.2024.112150**.
- [7] C. Sundermann, T. Heß, R. Sundermann, E. Kuitert, S. Krieter y T. Thüm. «Generating Feature Models with UVL's Full Expressiveness». En: *Proceedings of the 28th ACM International Systems and Software Product Line Conference. SPLC '24*. New York, NY, USA: Association for Computing Machinery, 2024, págs. 61-65. DOI: **10.1145/3646548.3676602**.
- [8] P. Clements y L. Northrop. *Software Product Lines: Practices and Patterns*. 2002. ISBN: 9780201703320.
- [9] J. van Gurp, J. Bosch y M. Svahnberg. «On the Notion of Variability in Software Product Lines». En: *Proceedings Working IEEE/IFIP Conference on Software Architecture*. IEEE Computer Society, 2001, págs. 45-54. DOI: **10.1109/WICSA.2001.948406**.

- [10] K. C. Kang, S. G. Cohen, J. A. Hess, W. E. Novak y A. S. Peterson. «Feature-Oriented Domain Analysis (FODA) Feasibility Study». En: *Software Engineering Institute, Carnegie Mellon University*, 1990.
- [11] D. Benavides, S. Segura y A. Ruiz-Cortés. «Automated Analysis of Feature Models 20 Years Later: A Literature Review». En: *Information Systems* 35.6 (2010), págs. 615-636.
- [12] R. E. Bryant. «Graph-Based Algorithms for Boolean Function Manipulation». En: *IEEE Transactions on Computers* C-35.8 (1986), págs. 677-691. DOI: **10.1109/TC.1986.1676819**.
- [13] C. Sundermann et al. «UVLParser: Extending UVL with Language Levels and Conversion Strategies». En: *Proceedings of the 27th ACM International Systems and Software Product Line Conference. SPLC '23*. Association for Computing Machinery, 2023, págs. 39-42. DOI: **10.1145/3579028.3609013**.
- [14] Flamapy. *Flamapy*. URL: <https://www.flamapy.org/> (visitado 22-09-2026).
- [15] WebAssembly Community Group. *WebAssembly*. URL: <https://webassembly.org/> (visitado 22-09-2026).
- [16] *UVLHub, a repository of feature models in UVL (Universal Variability Language) format, integrated with Zenodo and flamapy, and developed following Open Science*. 2023. URL: <https://docs.uvlhub.io/> (visitado 05-08-2026).
- [17] Wiki de EGC. *Práctica 1: Instalación del sistema base*. Material docente del curso 2024/2025. Diapositivas 15–16. Consultado el 24 de septiembre de 2026. 2024. URL: https://1984.lsi.us.es/wiki-egc/images/egc/c/ca/EGC_2024-25_P1.pdf.
- [18] S. Segura, R. M. Hierons, D. Benavides y A. Ruiz-Cortés. «Automated Metamorphic Testing on the Analyses of Feature Models». En: *Information and Software Technology* 53.3 (2011), págs. 245-258. DOI: **10.1016/j.infsof.2010.11.002**.
- [19] J. A. Galindo, A. J. Dominguez, J. White y D. Benavides. «Large Language Models to Generate Meaningful Feature Model Instances». En: *Proceedings of the 27th ACM International Systems and Software Product Line Conference*. ACM, 2023, págs. 15-26. DOI: **10.1145/3579027.3608973**.
- [20] M. Mendonça, M. Branco y D. Cowan. «S.P.L.O.T. - Software Product Lines Online Tools». En: *Proceedings of the 24th ACM SIGPLAN Conference Companion on Object Oriented Programming Systems Languages and Applications*. 2009, págs. 761-762.

- [21] T. Thüm, C. Kästner, F. Benduhn, J. Meinicke, G. Saake y T. Leich. «Featu-reIDE: An Extensible Framework for Feature-Oriented Software Development». En: *Science of Computer Programming* 79 (2014), págs. 70-85.
- [22] S. Segura, J. Galindo, D. Benavides, J. A. Parejo y A. Ruiz-Cortés. «BeTTy: Benchmarking and Testing on the Automated Analysis of Feature Models». En: *Proceedings of the 6th International Workshop on Variability Modelling of Software-intensive Systems (VaMoS)*. 2012, págs. 63-70.
- [23] S. Segura, J. A. Parejo, R. M. Hierons, D. Benavides y A. Ruiz-Cortés. «Au-tomated Generation of Computationally Hard Feature Models using Evo-lutionary Algorithms». En: *Expert Systems with Applications* 40.17 (2013), págs. 6935-6946.
- [24] Project Management Institute. *A Guide to the Project Management Body of Knowledge (PMBOK Guide) – Seventh Edition and The Standard for Project Management*. 7.^a ed. Project Management Institute, 2021. ISBN: 9781628256673.
- [25] Conventional Commits. *Conventional Commits*. URL: <https://www.conventionalcommits.org/en/v1.0.0/> (visitado 22-09-2026).
- [26] Universidad de Sevilla. *Grado en Ingeniería Informática – Ingeniería del Software*. <https://www.us.es/estudiar/que-estudiar/oferta-de-grados/grado-en-ingenieria-informatica-ingenieria-del-software>. Consultado el 28 de septiembre de 2026.
- [27] Gobierno de España. *Real Decreto 1125/2003, de 5 de septiembre*. <https://www.boe.es/buscar/act.php?id=B0E-A-2003-17643>. Artículo 4. 2003.
- [28] Pyodide developers. *Pyodide*. URL: <https://pyodide.org/en/0.27.7/> (visitado 22-09-2026).
- [29] Pallets Projects. *Flask Documentation*. URL: <https://flask.palletsprojects.com/en/stable/> (visitado 22-09-2026).
- [30] Bootstrap contributors. *Bootstrap Documentation*. URL: <https://getbootstrap.com/docs/5.3/> (visitado 22-09-2026).
- [31] Render. *Render Documentation*. URL: <https://render.com/docs> (visitado 22-09-2026).
- [32] Docker. *Docker Documentation*. URL: <https://docs.docker.com/> (visitado 22-09-2026).
- [33] UVLHub. *Rosemary CLI*. URL: <https://docs.uvlhub.io/rosemary> (visitado 22-09-2026).
- [34] Selenium contributors. *Selenium Documentation*. URL: <https://www.selenium.dev/documentation/> (visitado 22-09-2026).

A MODELOS UVL GENERADOS Y CONFIGURACIONES

Este apéndice recoge tres modelos UVL ilustrativos, uno por cada nivel de expresividad utilizado: booleano, aritmético y de tipos. Para cada ejemplo se reproduce el contenido del modelo y el objeto completo de parámetros registrado en su ejecución. Los parámetros desactivados también se conservan para documentar íntegramente cada configuración. Estos ejemplos proceden de ejecuciones independientes de la evaluación cuantitativa de O1 descrita en el Capítulo 8.

En las tres ejecuciones ilustrativas, **SEED** tiene el valor **4233** y **NUM_MODELS** tiene el valor **10**. Cada modelo mostrado procede de una tanda de diez modelos. Estos valores no corresponden a la semilla ni a los resultados cuantitativos de O1.

A.1 Modelos ilustrativos por nivel de expresividad

A.1.1 Nivel booleano y cardinalidad de grupo

En esta configuración se activa el nivel booleano y, como nivel menor, la cardinalidad de grupo (**GROUP_CARDINALITY**). Los niveles aritmético y de tipos permanecen desactivados. El Listado A.1 presenta uno de los modelos generados y el Listado A.2 recoge todos los parámetros de la tanda.

```
1 include
2   Boolean.group-cardinality
3 features
4   F0
5     mandatory
6       F2
7         mandatory
8           F3
9             mandatory
10              F6
11 mandatory
12   F1 {Attr0 true}
13     optional
14       F4
15         mandatory
16           F5
17             [2]
18               F8
19                 mandatory
20                   F10
21               F7
```

```

22 mandatory
23 F11
24 mandatory
25 F9
26 constraints
27 (((F8 | F3) => (! F1.Attr0 | ! F9)) | F6) | ! F5) & (F2 & ! F10)
28 (((F11 | ! F10) | ((! F1.Attr0 & ! F4) & F5)) | F7) & ! F9
29 (F6 | F9) & ! F10
30 ((F4 => F6) => ((! F10 & F11) & ! F8)) & F1.Attr0
31 ! F7 | F3

```

Código A.1: Modelo UVL generado con el nivel booleano y cardinalidad de grupo.

```

1 {
2   "AGGREGATE_FUNCTIONS": false,
3   "ARITHMETIC_LEVEL": false,
4   "ATTRIBUTES_LIST": [],
5   "ATTRIBUTE_ATTACH_PROBS": [],
6   "ATTRIBUTE_IN_CONSTRAINTS": [],
7   "ATTR_DIST_BOOLEAN": 1,
8   "ATTR_DIST_INTEGER": 0,
9   "ATTR_DIST_REAL": 0,
10  "ATTR_DIST_STRING": 0,
11  "BOOLEAN_LEVEL": true,
12  "CTC_DIST_BOOLEAN": 1,
13  "CTC_DIST_INTEGER": 0,
14  "CTC_DIST_REAL": 0,
15  "CTC_DIST_STRING": 0,
16  "DIST_ALTERNATIVE": 0.1,
17  "DIST_GROUP_CARDINALITY": 0.24,
18  "DIST_MANDATORY": 0.24,
19  "DIST_OPTIONAL": 0.24,
20  "DIST_OR": 0.18,
21  "ENSURE_SATISFIABLE": false,
22  "EXTRA_CONSTRAINT_REPRESENTATIVENESS": 1,
23  "FEATURE_CARDINALITY": false,
24  "FEATURE_DIST_BOOLEAN": 0.7,
25  "FEATURE_DIST_INTEGER": 0.1,
26  "FEATURE_DIST_REAL": 0.1,
27  "FEATURE_DIST_STRING": 0.1,
28  "GROUP_CARDINALITY": true,
29  "GROUP_CARDINALITY_MAX": 6,
30  "GROUP_CARDINALITY_MIN": 1,
31  "INCLUDE_CONSTRAINT_COUNT_SUFFIX": false,
32  "INCLUDE_FEATURE_COUNT_SUFFIX": false,
33  "MAX_ATTRIBUTES": 5,
34  "MAX_CONSTRAINTS": 8,
35  "MAX_FEATURES": 15,

```

```

36  "MAX_TREE_DEPTH": 5,
37  "MAX_VARS_PER_CONSTRAINT": 8,
38  "MIN_ATTRIBUTES": 1,
39  "MIN_CONSTRAINTS": 5,
40  "MIN_FEATURES": 10,
41  "MIN_VARS_PER_CONSTRAINT": 1,
42  "NAME_PREFIX": "",
43  "NUM_MODELS": 10,
44  "PROB_AND": 0.29,
45  "PROB_AVG_FUNCTION": 0,
46  "PROB_DIVIDE": 0,
47  "PROB_EQUALS": 0,
48  "PROB_EQUIVALENCE": 0.21,
49  "PROB_FEATURE_CARDINALITY": 0,
50  "PROB_GREATER": 0,
51  "PROB_GREATER_EQUALS": 0,
52  "PROB_IMPLICATION": 0.24,
53  "PROB_LEN_FUNCTION": 0,
54  "PROB_LESS": 0,
55  "PROB_LESS_EQUALS": 0,
56  "PROB_MULTIPLY": 0,
57  "PROB_NOT": 0.3,
58  "PROB_OR_CT": 0.26,
59  "PROB_SUBTRACT": 0,
60  "PROB_SUM": 0,
61  "PROB_SUM_FUNCTION": 0,
62  "RANDOM_ATTRIBUTES": true,
63  "SEED": 4233,
64  "STRING_CONSTRAINTS": false,
65  "TYPE_LEVEL": false
66  }

```

Código A.2: Parámetros de la tanda con nivel booleano y cardinalidad de grupo.

A.1.2 Nivel aritmético, cardinalidad de grupo, cardinalidad de característica y funciones de agregación

Esta configuración activa los niveles booleano y aritmético, así como la cardinalidad de grupo, la cardinalidad de característica (**FEATURE_CARDINALITY**) y las funciones de agregación (**AGGREGATE_FUNCTIONS**). El Listado A.3 contiene un modelo generado y el Listado A.4 muestra la configuración completa de su tanda.

```

1  include
2      Boolean.group-cardinality
3      Arithmetic.*
4  features
5      F0

```

```

6      mandatory
7          F2 cardinality [3..4]
8              mandatory
9                  F3 cardinality [4..4] {Attr2 26.38, Attr3 false}
10                     mandatory
11                         F6
12                             optional
13                                 F7
14                                     or
15                                         F11
16                                             F10 cardinality [5..5] {Attr1 53}
17                                                 F9 cardinality [3..3]
18
19      mandatory
20          F1 cardinality [4..4]
21              mandatory
22                  F4 cardinality [3..4]
23                      mandatory
24                          F5 {Attr0 45.82}
25                              optional
26                                  F8
27
28      constraints
29          ((F5.Attr0 > sum(F10.Attr1, F3.Attr2)) & F9) <=> ! F1
30          ! F5 & F6
31          ((F3.Attr2 < F10.Attr1) => ((F9 <=> F2) & F5)) <=> F1
32          F11 => F6
33          ! F5

```

Código A.3: Modelo UVL generado con nivel aritmético y sus niveles menores.

```

1  {
2      "AGGREGATE_FUNCTIONS": true,
3      "ARITHMETIC_LEVEL": true,
4      "ATTRIBUTES_LIST": [],
5      "ATTRIBUTE_ATTACH_PROBS": [],
6      "ATTRIBUTE_IN_CONSTRAINTS": [],
7      "ATTR_DIST_BOOLEAN": 0.27,
8      "ATTR_DIST_INTEGER": 0.26,
9      "ATTR_DIST_REAL": 0.47,
10     "ATTR_DIST_STRING": 0,
11     "BOOLEAN_LEVEL": true,
12     "CTC_DIST_BOOLEAN": 0.1,
13     "CTC_DIST_INTEGER": 0.45,
14     "CTC_DIST_REAL": 0.45,
15     "CTC_DIST_STRING": 0,
16     "DIST_ALTERNATIVE": 0.1,
17     "DIST_GROUP_CARDINALITY": 0.24,
18     "DIST_MANDATORY": 0.24,
19     "DIST_OPTIONAL": 0.24,

```

```
20 "DIST_OR": 0.18,
21 "ENSURE_SATISFIABLE": false,
22 "EXTRA_CONSTRAINT_REPRESENTATIVENESS": 1,
23 "FEATURE_CARDINALITY": true,
24 "FEATURE_DIST_BOOLEAN": 0.7,
25 "FEATURE_DIST_INTEGER": 0.1,
26 "FEATURE_DIST_REAL": 0.1,
27 "FEATURE_DIST_STRING": 0.1,
28 "GROUP_CARDINALITY": true,
29 "GROUP_CARDINALITY_MAX": 6,
30 "GROUP_CARDINALITY_MIN": 1,
31 "INCLUDE_CONSTRAINT_COUNT_SUFFIX": false,
32 "INCLUDE_FEATURE_COUNT_SUFFIX": false,
33 "MAX_ATTRIBUTES": 5,
34 "MAX_CONSTRAINTS": 8,
35 "MAX_FEATURES": 15,
36 "MAX_FEATURE_CARDINALITY": [
37     5
38 ],
39 "MAX_TREE_DEPTH": 5,
40 "MAX_VARS_PER_CONSTRAINT": 8,
41 "MIN_ATTRIBUTES": 1,
42 "MIN_CONSTRAINTS": 5,
43 "MIN_FEATURES": 10,
44 "MIN_FEATURE_CARDINALITY": [
45     2
46 ],
47 "MIN_VARS_PER_CONSTRAINT": 1,
48 "NAME_PREFIX": "",
49 "NUM_MODELS": 10,
50 "PROB_AND": 0.29,
51 "PROB_AVG_FUNCTION": 0.17,
52 "PROB_DIVIDE": 0.17,
53 "PROB_EQUALS": 0.2,
54 "PROB_EQUIVALENCE": 0.21,
55 "PROB_FEATURE_CARDINALITY": 0.7,
56 "PROB_GREATER": 0.2,
57 "PROB_GREATER_EQUALS": 0.2,
58 "PROB_IMPLICATION": 0.24,
59 "PROB_LEN_FUNCTION": 0,
60 "PROB_LESS": 0.2,
61 "PROB_LESS_EQUALS": 0.2,
62 "PROB_MULTIPLY": 0.17,
63 "PROB_NOT": 0.3,
64 "PROB_OR_CT": 0.26,
65 "PROB_SUBTRACT": 0.16,
66 "PROB_SUM": 0.17,
```

```

67 "PROB_SUM_FUNCTION": 0.16,
68 "RANDOM_ATTRIBUTES": true,
69 "SEED": 4233,
70 "STRING_CONSTRAINTS": false,
71 "TYPE_LEVEL": false
72 }

```

Código A.4: Parámetros de la tanda con nivel aritmético y sus niveles menores.

A.1.3 Nivel de tipos y restricciones sobre cadenas

La última configuración activa los niveles booleano, aritmético y de tipos, así como los cuatro niveles menores representados por **GROUP_CARDINALITY**, **FEATURE_CARDINALITY**, **AGGREGATE_FUNCTIONS** y **STRING_CONSTRAINTS**. El Listado A.5 muestra uno de los modelos generados y el Listado A.6 recoge sus parámetros.

```

1 include
2   Boolean.group-cardinality
3   Arithmetic.*
4   Type.string-constraints
5 features
6   F0
7     or
8       F2 cardinality [3..5] {Attr2 64.26}
9       optional
10        Integer F3
11        mandatory
12        Real F6 cardinality [4..4] {Attr4 '1
13        ad3pMLJJwXVPEkn\ZaDUfZZRpE92NiKFh6fzfFYfmik'}
14        mandatory
15        F7 cardinality [5..5] {Attr0 51, Attr1 false}
16        optional
17        F10 cardinality [2..5]
18        optional
19        F9 cardinality [3..5]
20   F1
21     mandatory
22     Integer F4 cardinality [3..4]
23     optional
24     Real F5 cardinality [3..3] {Attr3 '
25     z8K1qWctslCu01oz8hlVjtjltpagk'}
26     optional
27     Real F8 cardinality [2..2]
28 constraints
29   F5.Attr3 == F6.Attr4
30   F8 == (len(F6.Attr4) + F5)

```

```

29      F3 == F7.Attr0
30      ((F5 == F8) & (((F3 + F6) / F7.Attr0) <= F4)) | ! F9
31      F10

```

Código A.5: Modelo UVL generado con nivel de tipos y todos los niveles menores.

```

1  {
2    "AGGREGATE_FUNCTIONS": true,
3    "ARITHMETIC_LEVEL": true,
4    "ATTRIBUTES_LIST": [],
5    "ATTRIBUTE_ATTACH_PROBS": [],
6    "ATTRIBUTE_IN_CONSTRAINTS": [],
7    "ATTR_DIST_BOOLEAN": 0.27,
8    "ATTR_DIST_INTEGER": 0.18,
9    "ATTR_DIST_REAL": 0.26,
10   "ATTR_DIST_STRING": 0.29,
11   "BOOLEAN_LEVEL": true,
12   "CTC_DIST_BOOLEAN": 0.1,
13   "CTC_DIST_INTEGER": 0.29,
14   "CTC_DIST_REAL": 0.36,
15   "CTC_DIST_STRING": 0.25,
16   "DIST_ALTERNATIVE": 0.1,
17   "DIST_GROUP_CARDINALITY": 0.24,
18   "DIST_MANDATORY": 0.24,
19   "DIST_OPTIONAL": 0.24,
20   "DIST_OR": 0.18,
21   "ENSURE_SATISFIABLE": false,
22   "EXTRA_CONSTRAINT_REPRESENTATIVENESS": 1,
23   "FEATURE_CARDINALITY": true,
24   "FEATURE_DIST_BOOLEAN": 0.18,
25   "FEATURE_DIST_INTEGER": 0.27,
26   "FEATURE_DIST_REAL": 0.35,
27   "FEATURE_DIST_STRING": 0.2,
28   "GROUP_CARDINALITY": true,
29   "GROUP_CARDINALITY_MAX": 6,
30   "GROUP_CARDINALITY_MIN": 1,
31   "INCLUDE_CONSTRAINT_COUNT_SUFFIX": false,
32   "INCLUDE_FEATURE_COUNT_SUFFIX": false,
33   "MAX_ATTRIBUTES": 5,
34   "MAX_CONSTRAINTS": 8,
35   "MAX_FEATURES": 15,
36   "MAX_FEATURE_CARDINALITY": [
37     5
38   ],
39   "MAX_TREE_DEPTH": 5,
40   "MAX_VARS_PER_CONSTRAINT": 8,
41   "MIN_ATTRIBUTES": 1,
42   "MIN_CONSTRAINTS": 5,

```

```

43  "MIN_FEATURES": 10,
44  "MIN_FEATURE_CARDINALITY": [
45    2
46  ],
47  "MIN_VARS_PER_CONSTRAINT": 1,
48  "NAME_PREFIX": "",
49  "NUM_MODELS": 10,
50  "PROB_AND": 0.29,
51  "PROB_AVG_FUNCTION": 0.17,
52  "PROB_DIVIDE": 0.17,
53  "PROB_EQUALS": 0.2,
54  "PROB_EQUIVALENCE": 0.21,
55  "PROB_FEATURE_CARDINALITY": 0.7,
56  "PROB_GREATER": 0.2,
57  "PROB_GREATER_EQUALS": 0.2,
58  "PROB_IMPLICATION": 0.24,
59  "PROB_LEN_FUNCTION": 0.7,
60  "PROB_LESS": 0.2,
61  "PROB_LESS_EQUALS": 0.2,
62  "PROB_MULTIPLY": 0.17,
63  "PROB_NOT": 0.3,
64  "PROB_OR_CT": 0.26,
65  "PROB_SUBTRACT": 0.16,
66  "PROB_SUM": 0.17,
67  "PROB_SUM_FUNCTION": 0.16,
68  "RANDOM_ATTRIBUTES": true,
69  "SEED": 4233,
70  "STRING_CONSTRAINTS": true,
71  "TYPE_LEVEL": true
72  }

```

Código A.6: Parámetros de la tanda con nivel de tipos y todos los niveles menores.

B INTERFAZ DEL GENERADOR DE MODELOS UVL

Este apéndice reúne las capturas del asistente de configuración integrado en UVLHub y los bocetos iniciales elaborados durante el desarrollo. Su descripción general y la atribución del trabajo previo se encuentran en la Sección 5.4. Las capturas documentan la interfaz desde la que se utiliza el componente generador; la ampliación del asistente a seis pasos, el panel lateral y el selector visual de distribuciones proceden del trabajo previo de David Romero.

B.1 Recorrido por el asistente

Paso 1: Batch

El primer paso establece el número de modelos, la semilla y las opciones de nomenclatura. Véase la Figura B.1.

Home / Generate random models

1 Batch
Models, seed, name

2 Language levels
Pick the expressivity

3 Feature tree
Shape and relation mix

4 Constraints
Cross-tree constraints

5 Attributes
Attribute catalog

6 Output
Final options & download

YOUR SETUP

1 · Batch

Models 5

Seed 42

2 · Language levels

Majors Boolean, Arithmetic, Type

Minors Group card., String etc

3 · Feature tree

Batch
Models, seed, name

This first step is just the basics of the batch — how many models to produce, a seed for reproducibility, and a name prefix for the output files. Advanced options (language levels, tree shape, constraints, attributes, ensure-satisfiable, filename suffixes) come in later steps.

Number of models ⓘ
5

Seed ⓘ
42

Name prefix ⓘ
fm

Next >

Figura B.1: Interfaz del paso Batch. Captura realizada por el autor.

Paso 2: Language levels

El segundo paso permite activar los niveles principales y secundarios de expresividad UVL. Véase la Figura B.2.

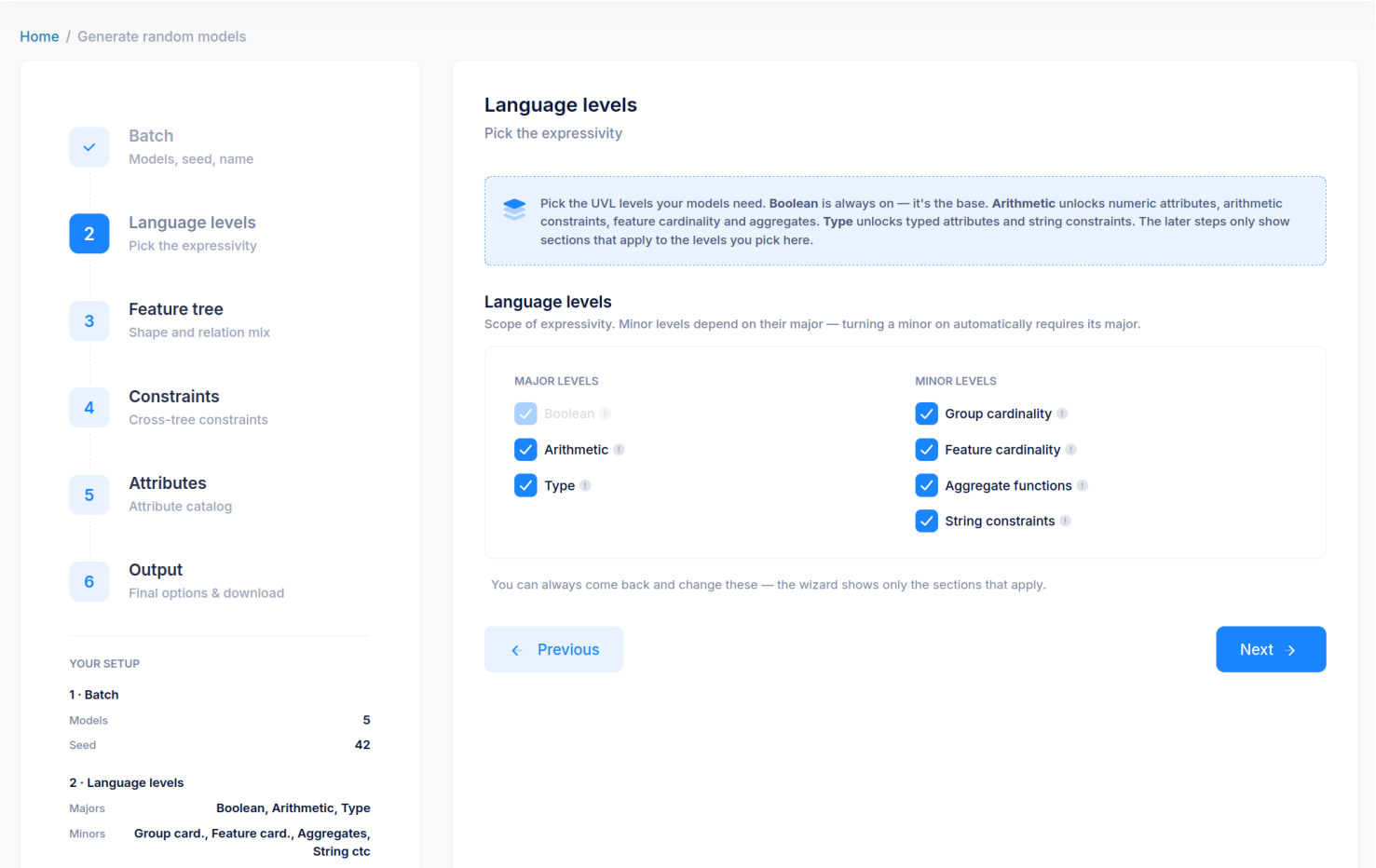


Figura B.2: Interfaz del paso Language levels. Captura realizada por el autor.

Paso 3: Feature tree

Este paso configura el número de características, la profundidad del árbol y la distribución de relaciones jerárquicas. Véanse las Figuras B.3 y B.4.

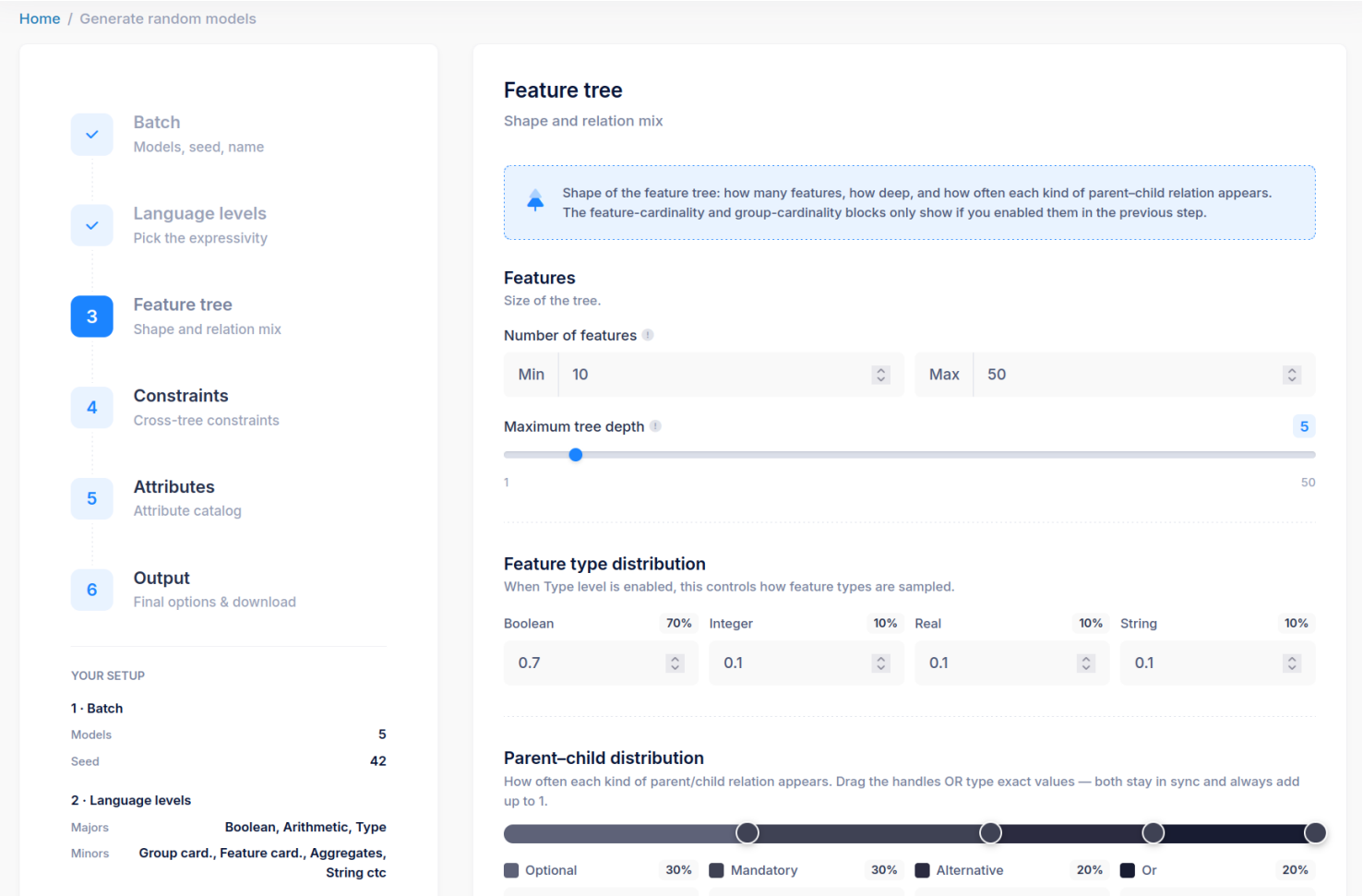


Figura B.3: Configuración del árbol de características, primera vista. Captura realizada por el autor.

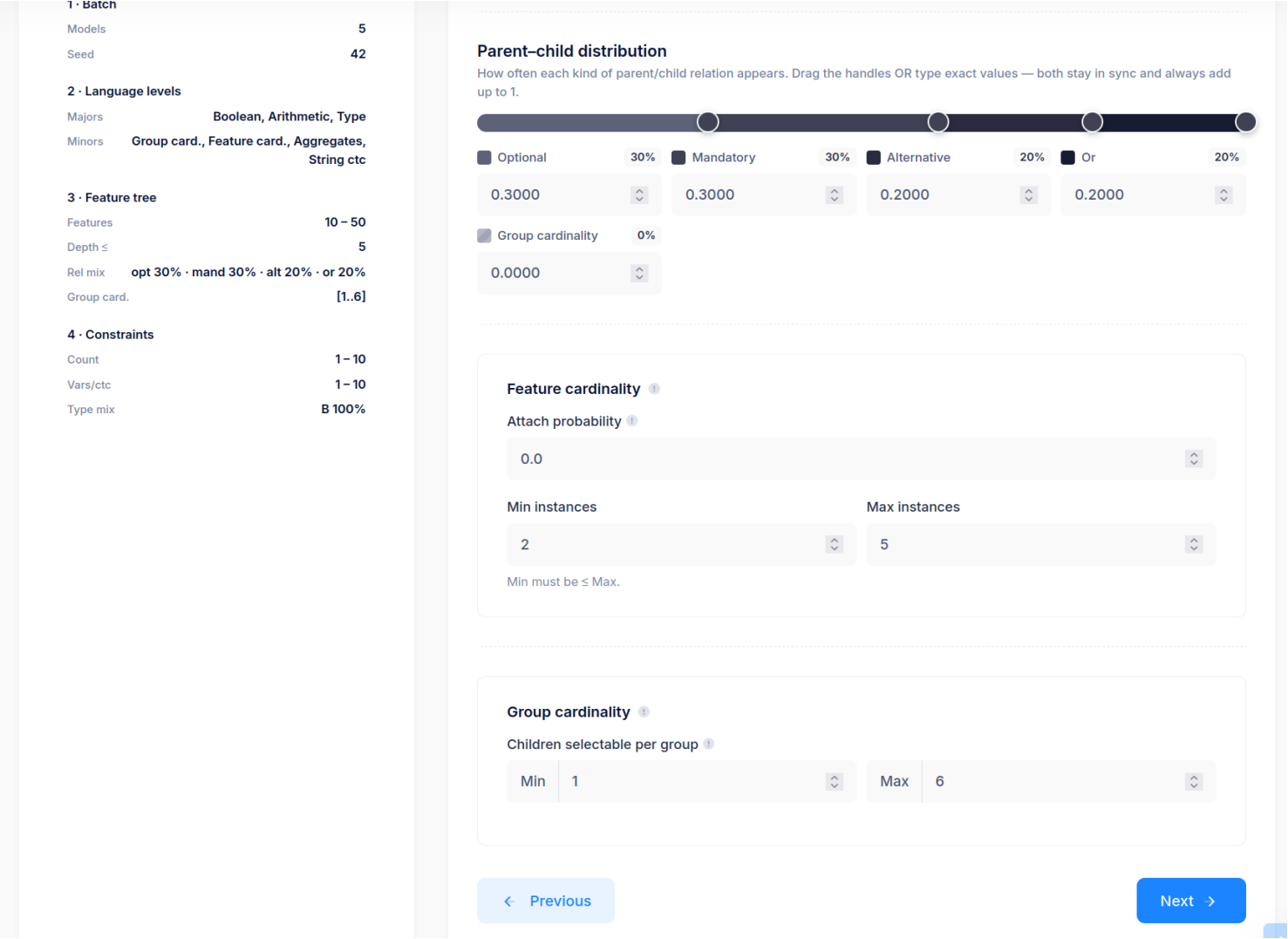


Figura B.4: Configuración del árbol de características, segunda vista. Captura realizada por el autor.

Paso 4: Constraints

Esta vista reúne la cantidad de restricciones, sus tipos y las distribuciones de operadores. Véanse las Figuras B.5 y B.6.

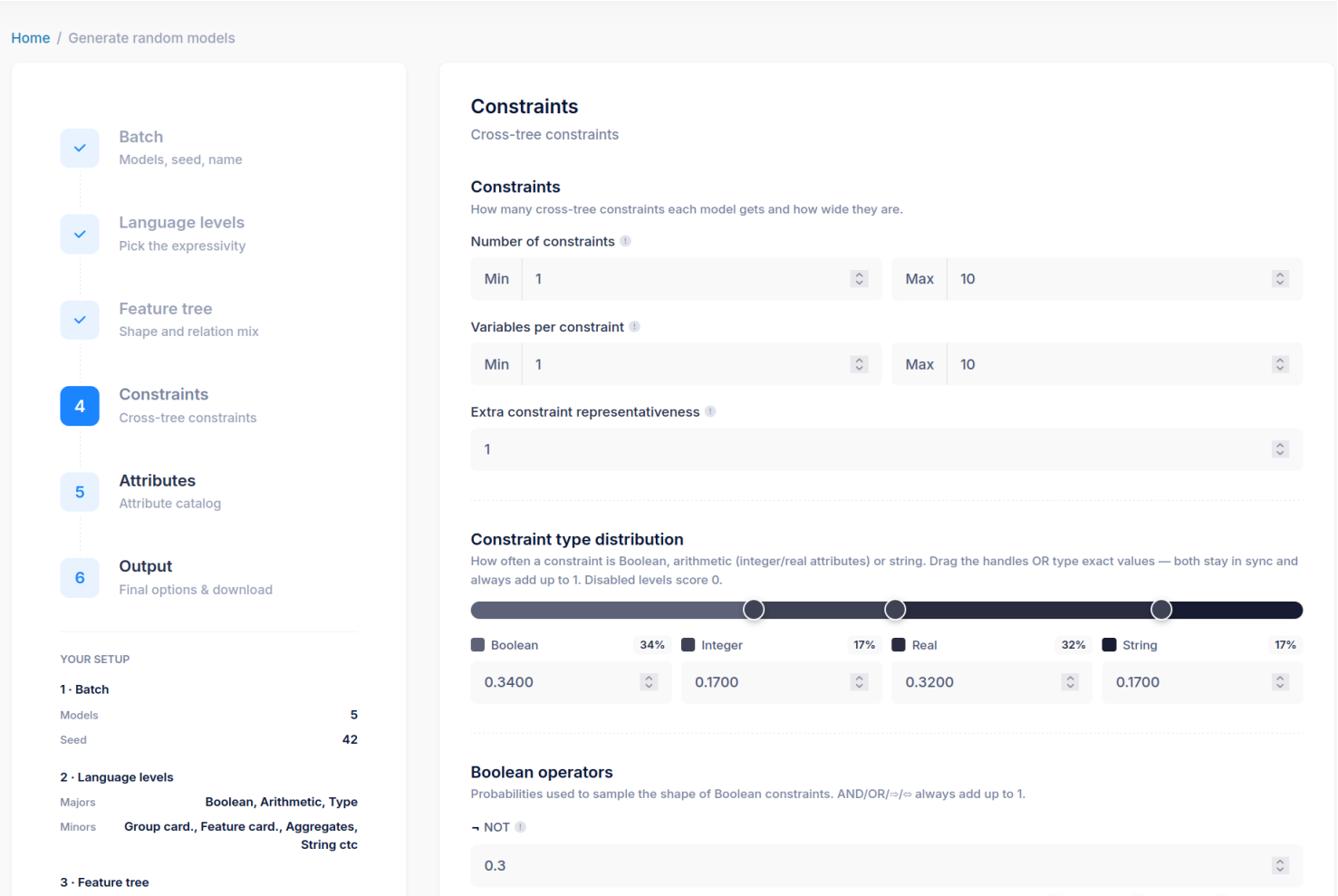


Figura B.5: Configuración de restricciones, primera vista. Captura realizada por el autor.

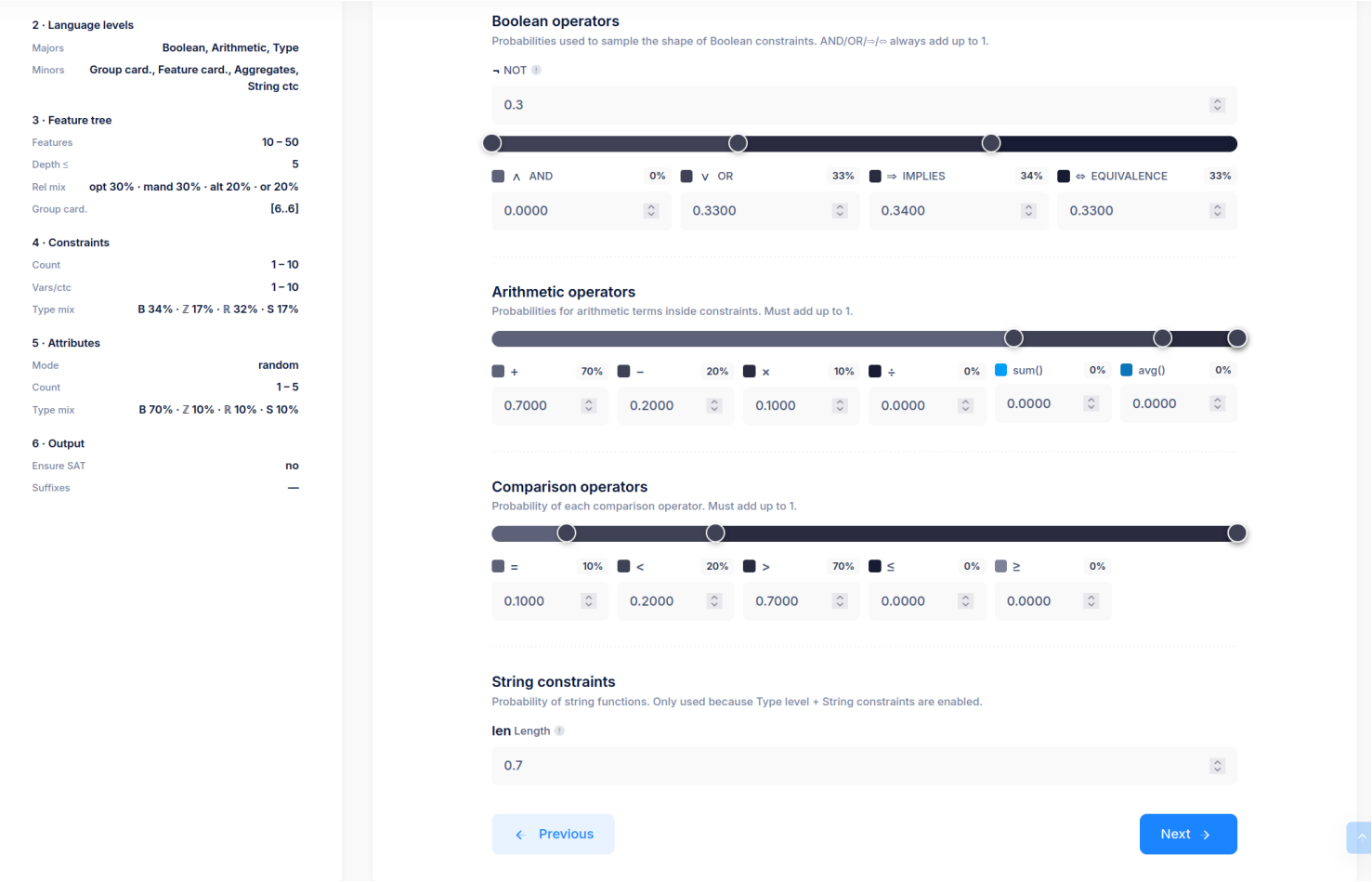


Figura B.6: Configuración de restricciones, segunda vista. Captura realizada por el autor.

Paso 5: Atributos

El quinto paso configura la generación de atributos y la distribución de sus tipos. Véanse las Figuras B.7 y B.8.

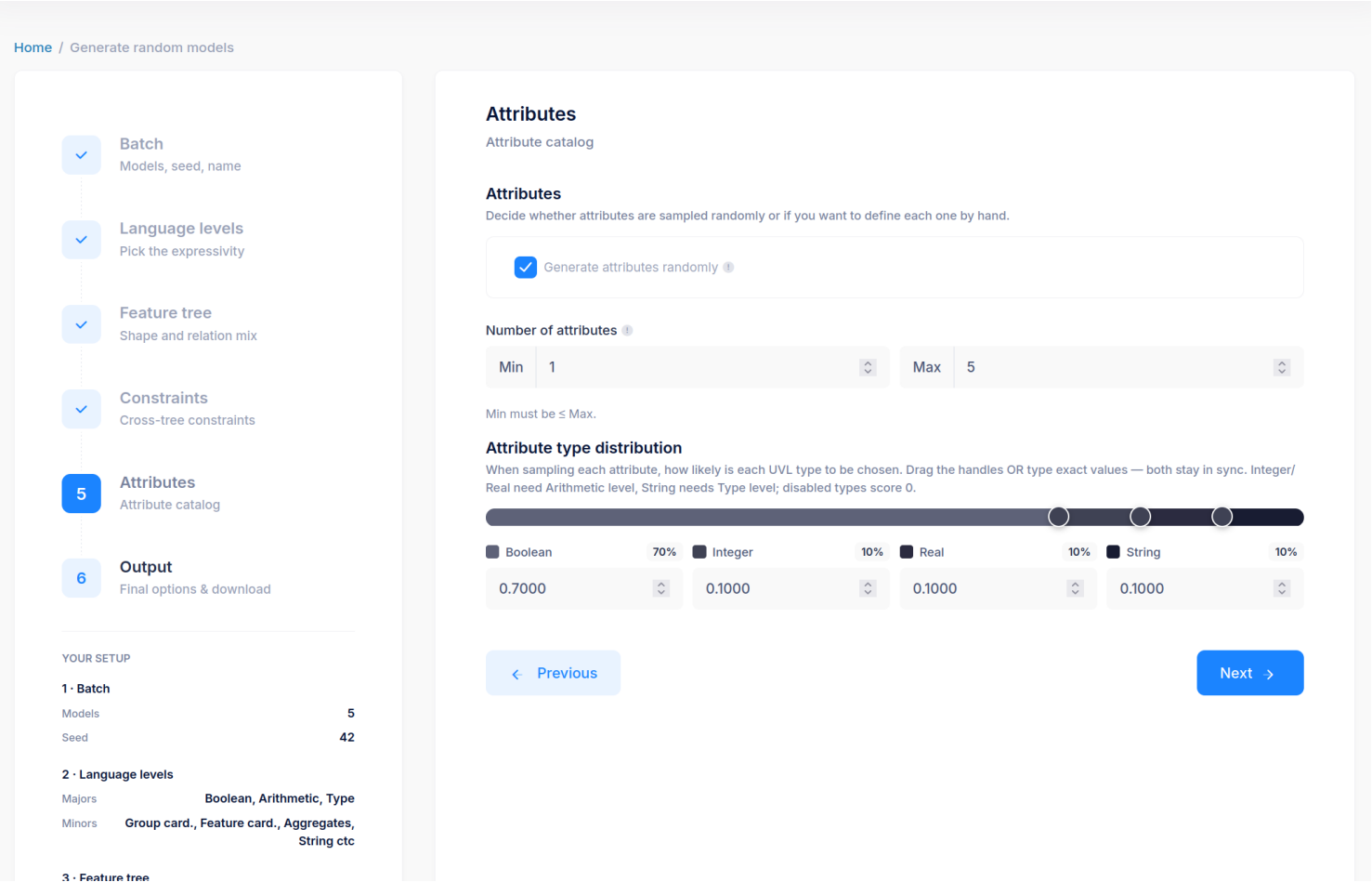


Figura B.7: Configuración de atributos, primera vista. Captura realizada por el autor.

Home / Generate random models

✓ Batch
Models, seed, name

✓ Language levels
Pick the expressivity

✓ Feature tree
Shape and relation mix

✓ Constraints
Cross-tree constraints

5 Attributes
Attribute catalog

6 Output
Final options & download

YOUR SETUP

1 · Batch

Models 5

Seed 42

2 · Language levels

Majors Boolean, Arithmetic, Type

Minors Group card., Feature card., Aggregates, String etc

3 · Feature tree

Features 10 – 50

Attributes

Attribute catalog

Attributes

Decide whether attributes are sampled randomly or if you want to define each one by hand.

☐ Generate attributes randomly ⓘ

There are 1 attribute card(s) with missing required fields. Complete them before continuing.

+ Add attribute

Attribute #1 Remove

Name Type

e.g. Seats Integer

Min Max

Attach probability

☒ Use in constraints

< Previous Next >

Figura B.8: Configuración de atributos, segunda vista. Captura realizada por el autor.

Paso 6: Output

El último paso permite activar, cuando corresponde, la opción de satisfacibilidad para modelos booleanos, seleccionar los sufijos de los archivos y generar los modelos. Los resultados pueden visualizarse o descargarse. Véanse las Figuras B.9, B.10 y B.11.

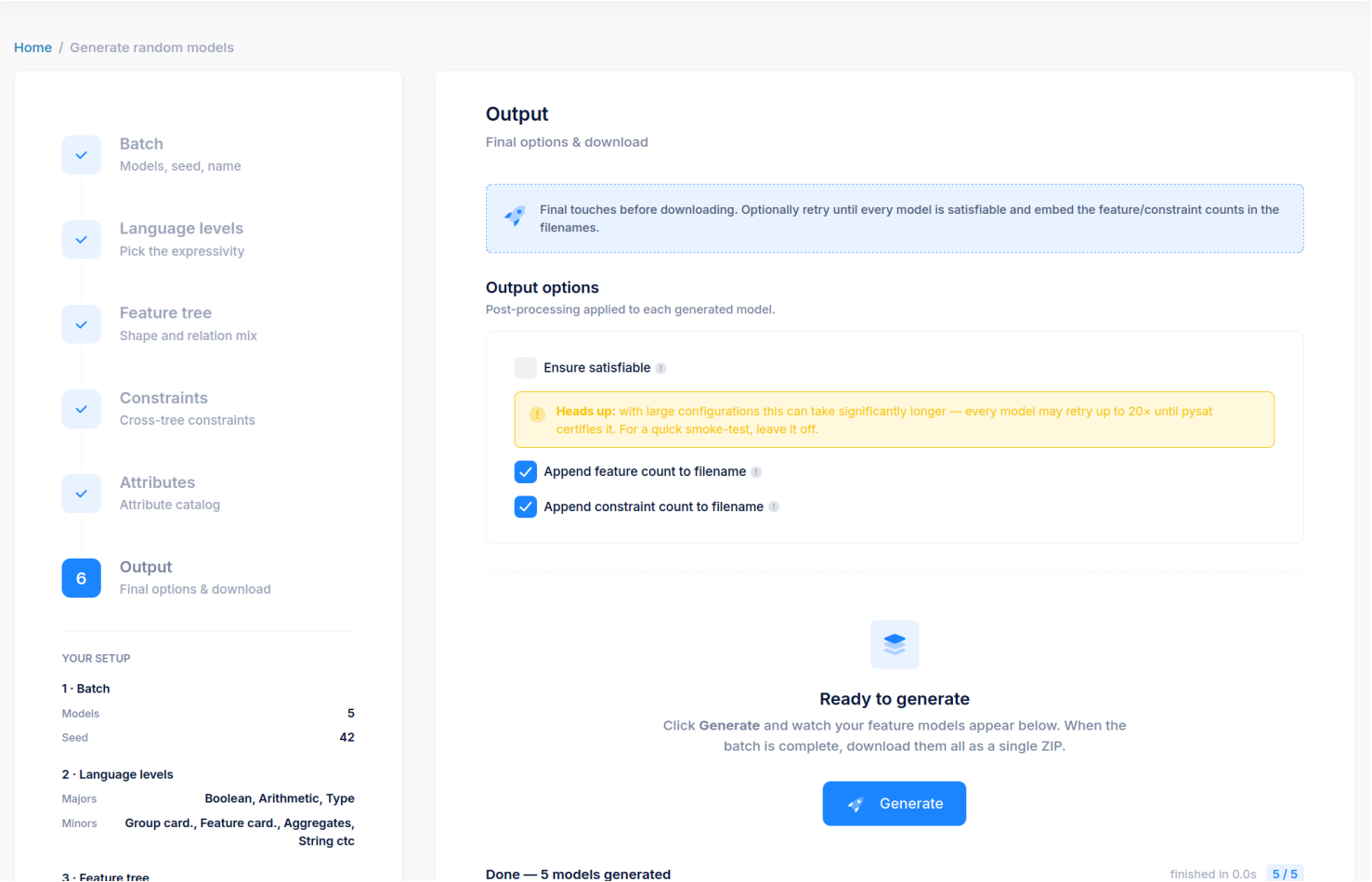


Figura B.9: Opciones de salida del generador. Captura realizada por el autor.

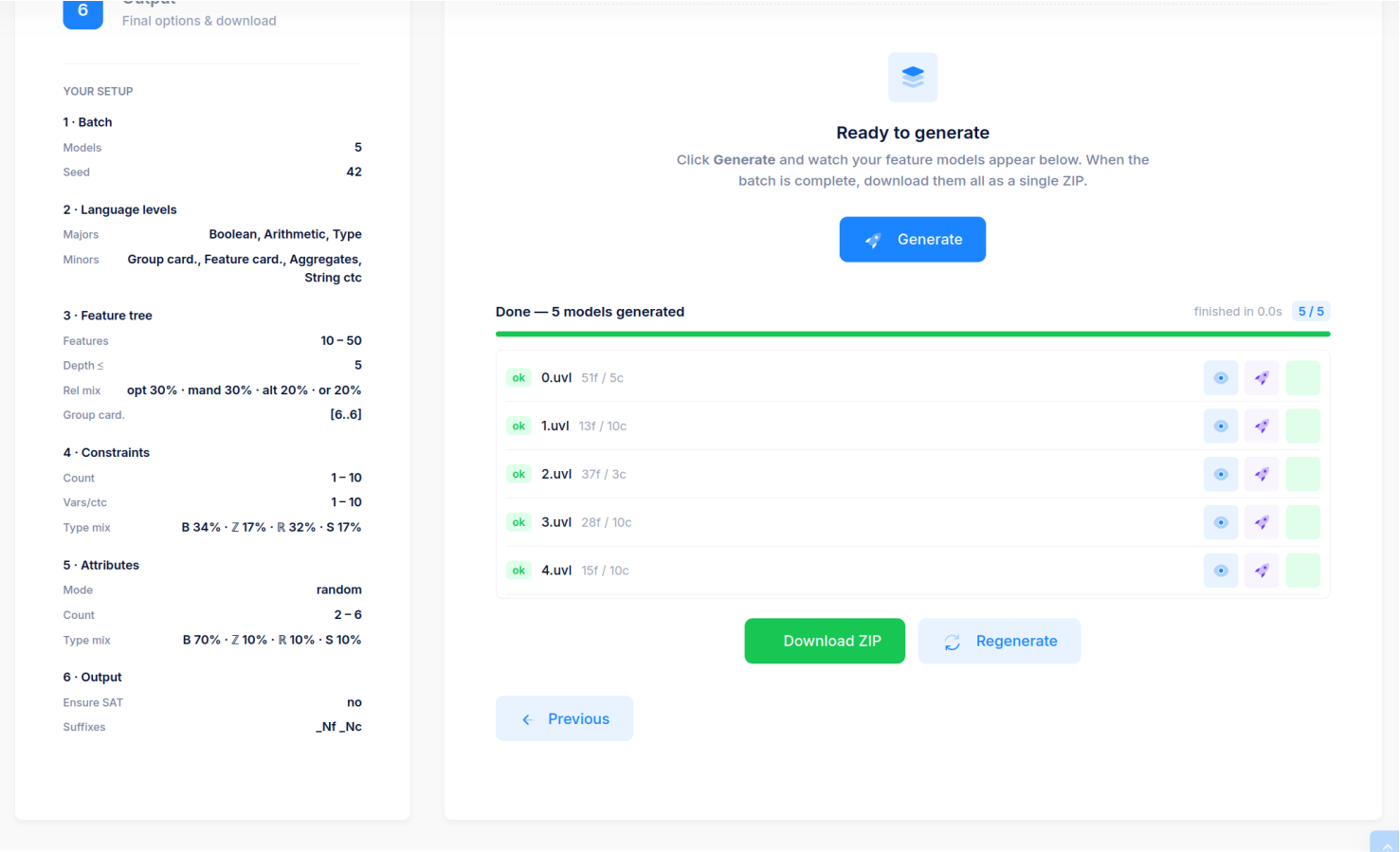


Figura B.10: Salida del generador, segunda vista. Captura realizada por el autor.

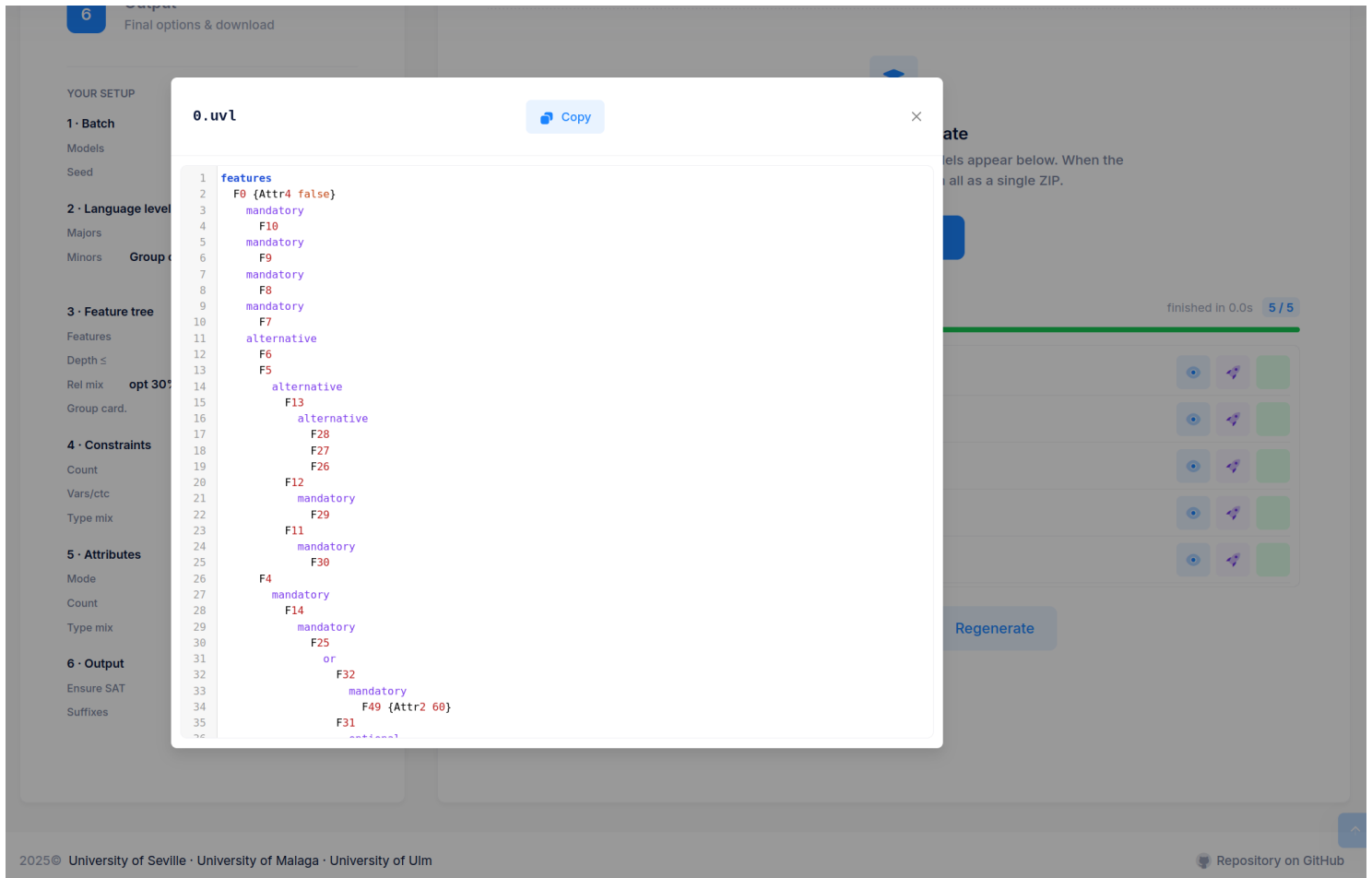


Figura B.11: Salida del generador, tercera vista. Captura realizada por el autor.

La Figura B.12 resume las vistas y el recorrido entre ellas.

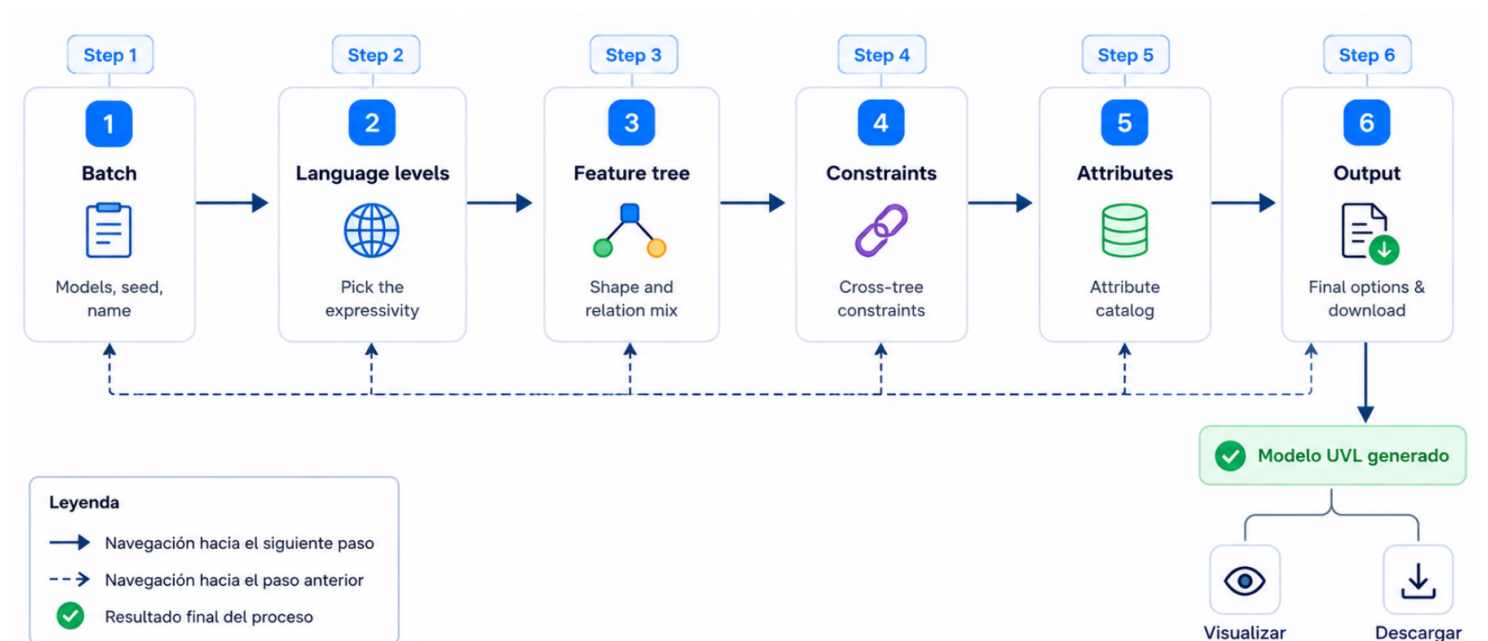


Figura B.12: Mapa de navegación del generador UVL. Elaboración propia.

B.2 Prototipos visuales iniciales

Los siguientes bocetos corresponden a una fase inicial del trabajo y no describen por completo la interfaz final. Se incluyen para mostrar la evolución de la propuesta visual. Las Figuras B.13, B.14, B.15, B.16 y B.17 presentan los cinco bocetos iniciales.

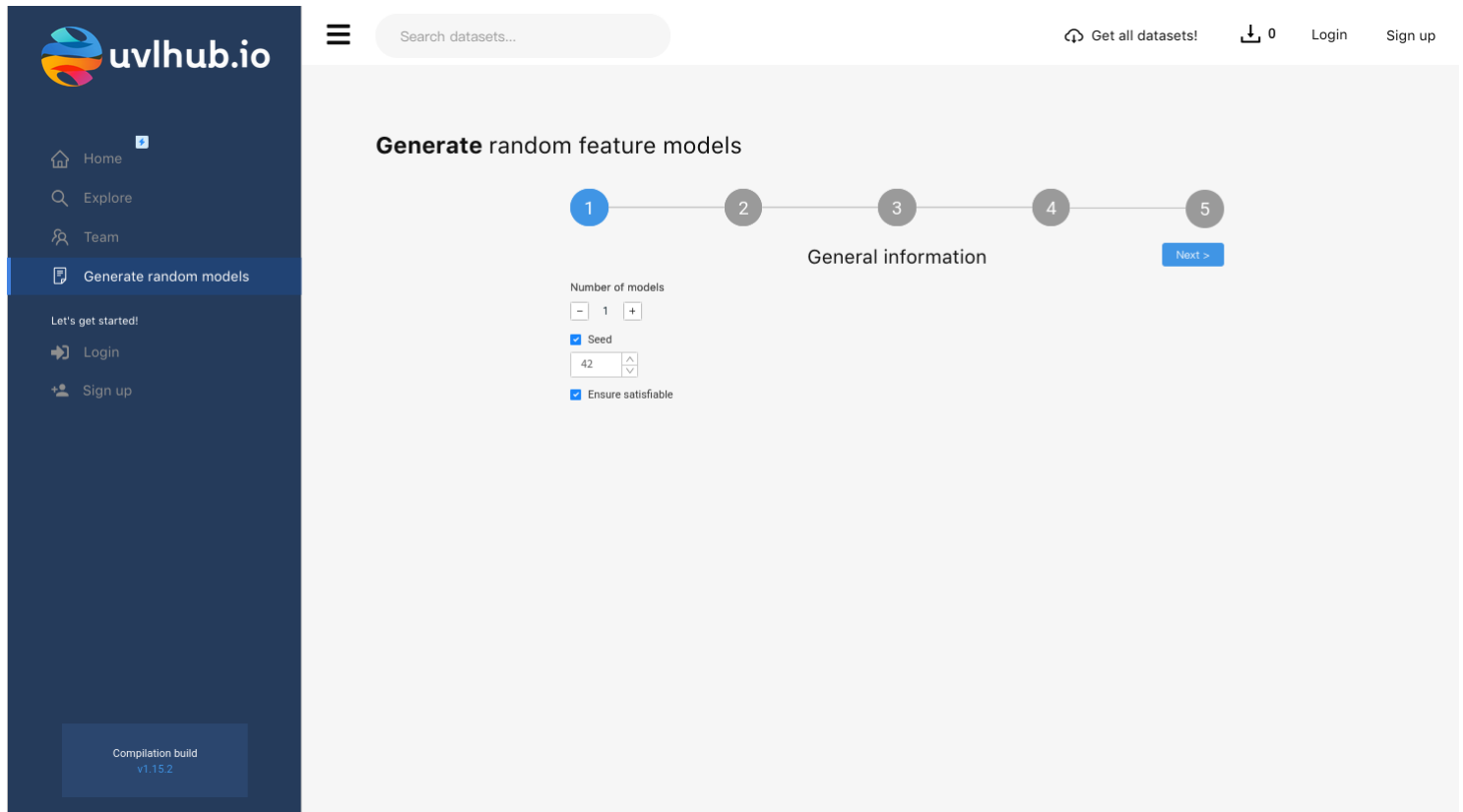


Figura B.13: Boceto inicial del paso 1. Elaboración propia.

uvlhub.io

Home Explore Team Generate random models

Let's get started! Login Sign up

Search datasets...

Get all datasets! 0 Login Sign up

Generate random feature models

1 2 3 4 5

< Previous Tree hierarchy Next >

Features

Number of features

Select one

- 1 +

Distribution (as a per unit value)

Boolean: 0.7 Integer: 0.2 Real: 0.7 String: 0.0

Language level. Boolean level is selected by default.

☒ Arithmetic level
☐ Type level

Note: Arithmetic level includes Boolean level, and Type level include all of them.

☒ Feature cardinality (only if Arithmetic level is selected)

Min: Select one

Min: - 1 + Max: - 3 +

Max: Select one

Min: - 1 + Max: - 3 +

Attach probability: 0.05

Maximum tree depth: 5

Groups

Optional: 0.7 Mandatory: 0.2 Alternative: 0.7 Or: 0.0

Figura B.14: Boceto inicial del paso 2. Elaboración propia.

The screenshot shows the 'Generate random feature models' interface on the UVLhub.io website. The interface is divided into a left sidebar and a main content area. The sidebar contains the UVLhub.io logo, navigation links (Home, Explore, Team, Generate random models), and a 'Let's get started!' section with 'Login' and 'Sign up' buttons. The main content area is titled 'Generate random feature models' and shows a progress bar with five steps. Step 3, 'Constraints', is the active step. The 'Constraints' section includes several input fields and buttons for configuring the model generation process. The 'Number of models' section has a 'Select one' dropdown, a 'Min' value of 1, and a 'Max' value of 4. The 'Extra constraint representativeness' is set to 0.7. The 'Variables per constraint' section has a 'Select one' dropdown, a 'Min' value of 1, and a 'Max' value of 3. The 'Distribution' section has input fields for 'Boolean' (0.7), 'Integer' (0.2), 'Real' (0.7), and 'String' (0.0). The 'Boolean level' section has input fields for '!' (0.7), '&' (0.2), '|' (0.7), '=>' (0.0), and '<=>' (0.0). The 'Arithmetic level constraints (only if Arithmetic level is selected)' section has input fields for '+', '-', '*', '/', '=', '<', '>', '<=', '>=', 'sum()', and 'avg()'. The 'Type level constraints (only if Type level is selected)' section has an input field for 'len()'.

uvlhub.io

Home Explore Team **Generate random models**

Let's get started!
Login Sign up

Search datasets...

Get all datasets! 0 Login Sign up

Generate random feature models

1 2 **3** 4 5

< Previo Next >

Constraints

Number of models
Select one

Min: 1 Max: 4

Extra constraint representativeness: 0.7

Variables per constraint
Select one

Min: 1 Max: 3

Distribution

Boolean: 0.7 Integer: 0.2 Real: 0.7 String: 0.0

Boolean level

! 0.7 & 0.2 | 0.7 => 0.0 <=> 0.0

Arithmetic level constraints (only if Arithmetic level is selected)

+ 0.7 - 0.2 * 0.7 / 0.0 = 0.0 < 0.2
> 0.7 <= 0.0 >= 0.0 sum() 0.0 avg() 0.0

Type level constraints (only if Type level is selected)

len() 0.7

Compilation build v1.15.2

Figura B.15: Boceto inicial del paso 3. Elaboración propia.

uvlhub.io

Search datasets...

Get all datasets! 0 Login Sign up

Generate random feature models

1 2 3 4 5

< Previo **Attributes** Finish

Add Delete last one

Name: Seats

Type: Integer

Value: Select one

Min: 1 Max: 3

Attach probability: 0.7

☒ Use in constraints

Name: License Plate

Type: String

Length: Select one

Min: 1 Max: 3

Attach probability: 0.7

Figura B.16: Boceto inicial del paso 4. Elaboración propia.

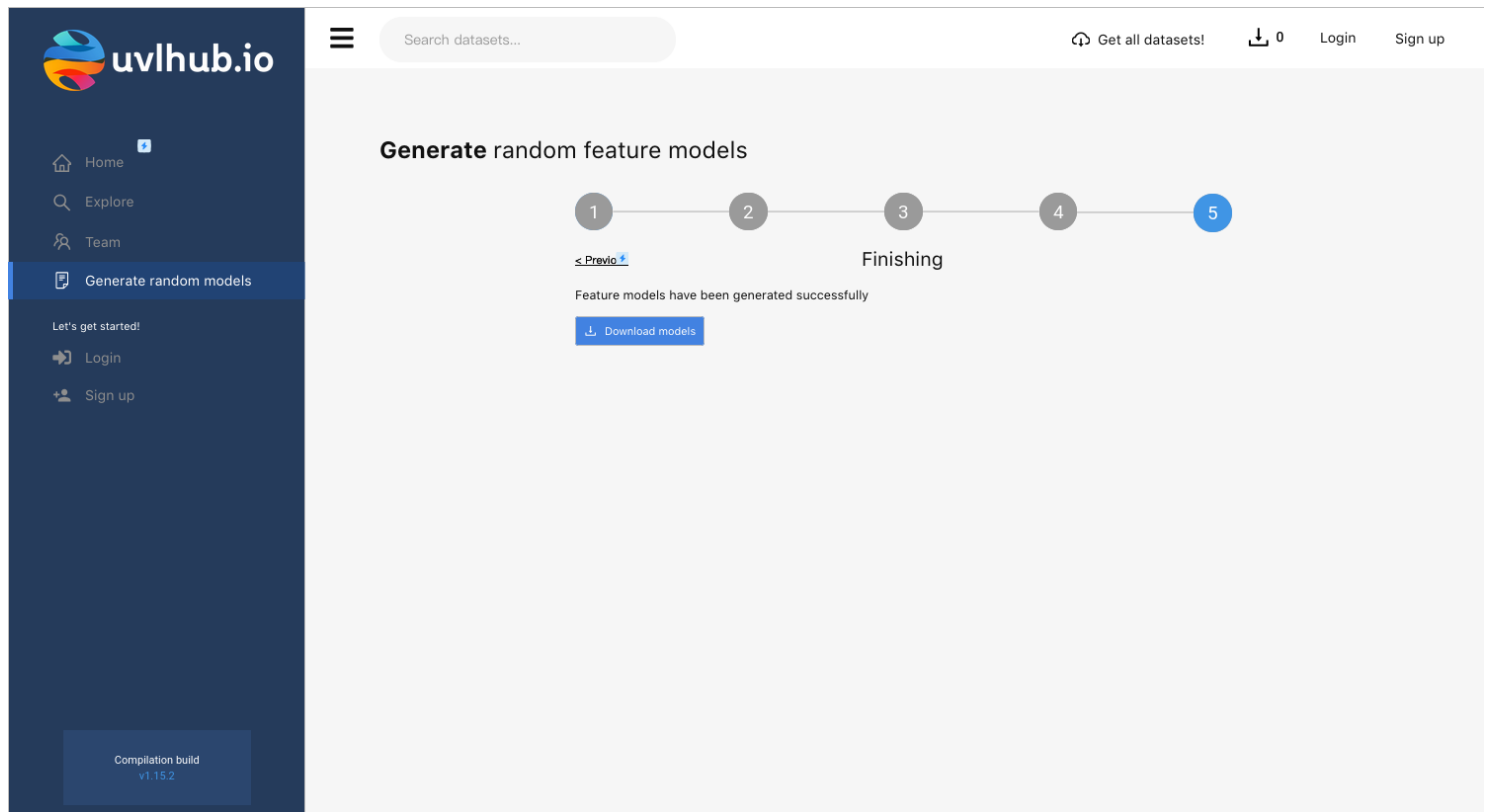


Figura B.17: Boceto inicial del paso 5. Elaboración propia.

B.3 Selector visual de distribuciones

El selector representa gráficamente las probabilidades con las que se generan determinados elementos. La Figura B.18 recoge su aspecto en la interfaz integrada.

Constraint type distribution

How often a constraint is Boolean, arithmetic (integer/real attributes) or string. Drag the handles OR type exact values — both stay in sync and always add up to 1. Disabled levels score 0.

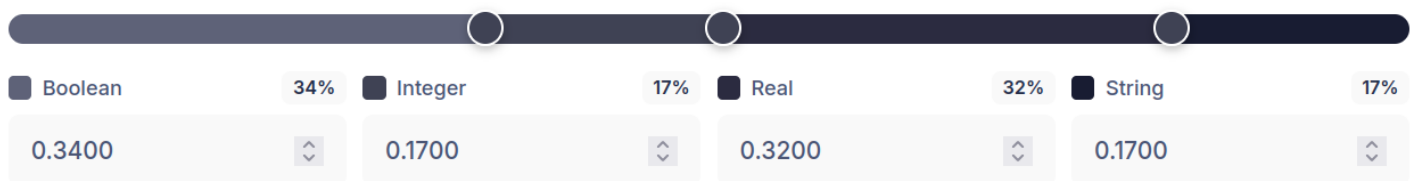


Figura B.18: Selector visual de distribuciones. Captura realizada por el autor.

C MANUAL DE INSTALACIÓN Y DESPLIEGUE

C.1 Requisitos previos

- **Sistema operativo:** el entorno utilizado durante el desarrollo y validación del sistema ha sido Ubuntu 24.04 LTS. También es posible utilizar otros sistemas compatibles con Docker.
- **Lenguaje y runtime:** no es necesario instalar manualmente Python ni sus dependencias en el sistema anfitrión, ya que el entorno de ejecución se encuentra definido dentro de los contenedores del proyecto.
- **Gestor de paquetes:** las dependencias necesarias para ejecutar la aplicación y el generador se instalan automáticamente durante la construcción de las imágenes Docker.
- **Servicios externos:** no es necesario instalar manualmente servicios adicionales. La configuración Docker del proyecto incluye los contenedores necesarios para los servicios utilizados por UVLHub, entre ellos la base de datos MariaDB, Redis, Elasticsearch, MailHog y Selenium.
- **Otras herramientas:** es necesario disponer de Git para obtener el código fuente y Docker junto con Docker Compose para construir y ejecutar el entorno de desarrollo.

C.2 Disponibilidad del código

El código fuente relacionado con este trabajo está disponible públicamente en GitHub. La Tabla C.1 recoge la dirección de cada repositorio, el commit de referencia considerado en esta memoria y la licencia declarada en cada caso.

Repositorio	URL	Commit de referencia	Licencia
uvlhub	https://github.com/antjimort/uvlhub	b4ca8d81f5b03c688a1468289a74d144ce5d081a	MIT
uvlhub_v2	https://github.com/antjimort/uvlhub_v2	45ee7a8c8f0ed0b800ee7a4646a561ae0145995f	No declarada
fm_generator	https://github.com/antjimort/fm_generator	5b0a32f7d728defcfcaf736e6c1f88d3d5e3c171	No declarada

Tabla C.1: Repositorios, commits de referencia y licencias declaradas.

El repositorio **uvlhub** contiene la aplicación web y su integración con el generador. En su directorio raíz se ha añadido un archivo **LICENSE** que declara la licencia MIT. Se mantienen los avisos de licencia específicos incluidos en determinados archivos del proyecto. La licencia del repositorio **uvlhub** no se extiende automáticamente a los repositorios **uvlhub_v2** y **fm_generator**, que no tienen una licencia declarada.

La aplicación está desplegada en Render y puede consultarse en <https://uvlhub-antjimort-8e4j.onrender.com>. La versión disponible es la **v1.0.0**, cuyo código ejecutable corresponde al commit **b4ca8d81f5b03c688a1468289a74d144ce5d081a** de **uvlhub**. El commit posterior que añade el archivo de licencia no modifica la aplicación. El generador se ejecuta como parte de UVLHub y no dispone de un despliegue público independiente. Los repositorios **uvlhub_v2** y **fm_generator** tampoco cuentan con despliegues separados.

Además del código disponible en los repositorios indicados, se ha depositado en Zenodo una copia archivada de *UVLHub with fm_generator integration*. El archivo corresponde a UVLHub 1.0.0 e incluye la versión 0.0.1 de **fm_generator**. Esta versión puede citarse mediante el DOI 10.5281/zenodo.22963634; su registro está disponible en <https://zenodo.org/records/22963634>.

C.3 Instalación de dependencias y ejecución

Las dependencias del proyecto se gestionan mediante Docker, por lo que no es necesaria una instalación manual de paquetes. La construcción de los contenedores instala automáticamente las dependencias definidas en la configuración del proyecto.

El siguiente comando construye las imágenes necesarias y levanta el entorno de desarrollo definido para UVLHub. Durante este proceso se instala también el paquete **fm_generator** y se construyen los recursos necesarios para su ejecución desde el navegador mediante Pyodide.

```
1 docker compose -f docker/docker-compose.dev.yml up --build
```

Para detener y eliminar los contenedores se usa el siguiente comando:

```
1 docker compose -f docker/docker-compose.dev.yml down
```

C.4 Despliegue en producción

La herramienta utilizada para el despliegue de la app principal ha sido Render, pero ha sido usada únicamente para las revisiones de los tutores. También se ha usado Redis, utilizado como almacenamiento externo para la gestión de las sesiones de Flask para conservar el estado del proceso de configuración del generador.

No obstante, la plataforma utilizada no constituye un requisito del sistema, ya que la aplicación puede ser desplegada mediante otras alternativas compatibles con contenedores.

El despliegue oficial de UVLHub es gestionado por DiversoLab.

C.5 Verificación de la instalación

Una vez levantados los contenedores mediante el comando indicado en la Sección C.3, se puede comprobar que la instalación se ha realizado correctamente accediendo a la dirección local proporcionada por la aplicación durante el arranque.

La dirección de acceso local depende de los puertos publicados por la configuración Docker. Puede comprobarse mediante el siguiente comando:

```
1 docker compose -f docker/docker-compose.dev.yml ps
```

El servicio debe abrirse en el navegador utilizando el puerto expuesto en el equipo anfitrión, normalmente mediante una dirección de la forma `http://localhost:<PUERTO>`. La salida del arranque permite comprobar que el contenedor web se ha iniciado correctamente; véase la Figura C.1.

```

web_app_container | [DEBUG] Indexing document with ID 'hubfile-12' in 'search_index'
web_app_container | [SUCCESS] Document 'hubfile-12' indexed successfully.
web_app_container | [REINDEX] Reindexing completed.
web_app_container | [✓] Reindexing completed successfully!
mariadb_container | 2026-09-07 11:15:13 9 [Warning] Aborted connection 9 to db: 'uVL
web_app_container | Looking for hubfiles without FactLabel...
web_app_container | [✓] All hubfiles already have FactLabels!
mariadb_container | 2026-09-07 11:15:14 10 [Warning] Aborted connection 10 to db: 'u
web_app_container | * Serving Flask app 'app'
web_app_container | * Debug mode: on
web_app_container | WARNING: This is a development server. Do not use it in a product
web_app_container | * Running on all addresses (0.0.0.0)
web_app_container | * Running on http://127.0.0.1:5000
web_app_container | * Running on http://172.18.0.10:5000
web_app_container | Press CTRL+C to quit
web_app_container | * Restarting with stat
web_app_container | * Debugger is active!
web_app_container | * Debugger PIN: 114-430-192

```

Figura C.1: Salida de consola tras el arranque correcto del contenedor web.
Elaboración propia.

C.5.1 Ejecución de las pruebas automatizadas

Una vez iniciado el entorno Docker, la suite de pruebas integrada del módulo *generator* puede ejecutarse mediante el siguiente comando:

```

1 docker compose -f docker/docker-compose.dev.yml exec web rosemary test
  ↪ generator --all

```

La ejecución correcta debe finalizar sin errores. Esta suite valida el flujo del asistente, la generación de modelos UVL, la persistencia de parámetros y las opciones de salida integradas en UVLHub.

Las pruebas del paquete independiente *fm_generator* se ejecutan desde la raíz del repositorio mediante:

```

1 pytest fm_generator/tests -v

```

C.5.2 Medición de cobertura

La cobertura del código del módulo *generator* integrado en UVLHub puede obtenerse ejecutando la suite mediante Rosemary dentro del contenedor web:

```

1 docker compose -f docker/docker-compose.dev.yml exec web rosemary
  ↪ coverage generator --all

```

El informe generado incluye tanto el código de producción como los ficheros de prueba. Para los resultados recogidos en esta memoria se considera únicamente la cobertura de los ficheros de producción, tal y como se explica en la Sección 7.5.

La cobertura del paquete independiente **fm_generator** puede obtenerse mediante el siguiente comando:

```
1  pytest fm_generator/tests -v --cov=fm_generator
   ↪  --cov-report=term-missing --cov-report=html
```

El informe textual muestra las líneas no cubiertas, mientras que el informe HTML se genera en el directorio **htmlcov**. Los resultados de cobertura utilizados en esta memoria se describen en la Sección 7.5.

D MANUAL DE USUARIO

D.1 Acceso al sistema

La versión del generador descrita en este manual puede utilizarse ejecutando la aplicación en un entorno local, siguiendo los pasos indicados en la Sección C.5 del manual de instalación y despliegue.

UVLHub dispone de una instancia pública en <https://www.uvlhub.io/>. No obstante, esta instancia puede evolucionar de forma independiente al código descrito en esta memoria. Por este motivo, la ejecución local mediante Docker constituye la forma de acceso reproducible a la versión evaluada en el presente trabajo.

La generación de modelos no requiere que el usuario disponga de una cuenta ni que realice un proceso de autenticación previo.

La pantalla inicial de UVLHub se muestra en la Figura D.1.

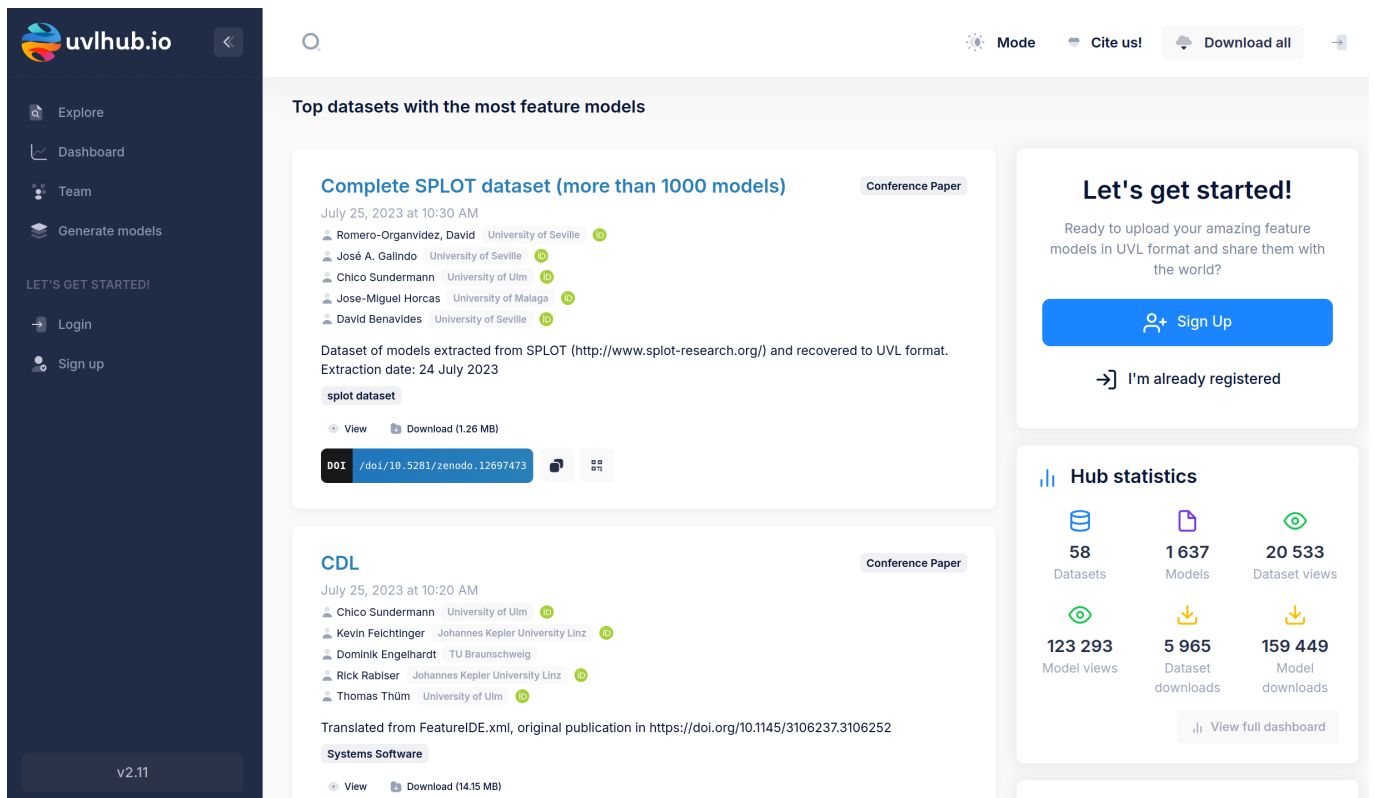


Figura D.1: Pantalla inicial de UVLHub. Captura realizada por el autor.

D.2 Funcionalidad principal

La funcionalidad principal del sistema consiste en la generación automática de modelos de características mediante un asistente de configuración. La estructura completa del flujo de generación y las pantallas correspondientes a cada etapa pueden consultarse en la Sección 5.4.1.

El funcionamiento del asistente consiste en configurar los diferentes parámetros de generación según las necesidades del usuario. En cada etapa del flujo se pueden modificar las opciones disponibles y avanzar al siguiente paso mediante el botón *Next*, situado en la parte inferior de la pantalla. En caso de querer modificar una configuración anterior, es posible volver a los pasos previos utilizando el botón *Previous*.

Una vez completada la configuración, en el sexto y último paso del asistente se generan los modelos de características solicitados. Desde esta pantalla el usuario dispone de diferentes opciones sobre los modelos generados:

- Descargar el conjunto de modelos generado mediante el botón *Download*.
- Visualizar un modelo directamente desde la aplicación mediante el botón con el icono del ojo.
- Descargar individualmente cada modelo mediante el botón verde disponible junto a cada resultado.
- Abrir un modelo mediante el botón morado con el icono del cohete en FlamapyIDE, un entorno externo que permite visualizar, editar y realizar tareas de análisis automático sobre modelos de características (<https://github.com/flamapy/flamapy-ide>).

D.3 Mensajes de error y resolución de incidencias

Durante la configuración, el asistente valida los valores introducidos en cada etapa antes de permitir que el usuario continúe al siguiente paso. Cuando una opción no cumple las reglas de negocio definidas en la Sección 4.5, se muestra un mensaje asociado al campo correspondiente indicando la condición que debe corregirse.

Estas validaciones incluyen, entre otros casos, límites numéricos incorrectos, distribuciones de probabilidades que no suman uno, dependencias entre niveles de expresividad y rangos inválidos para cardinalidades, restricciones o atributos. Una vez corregidos los valores señalados, el usuario puede continuar con el proceso de configuración y generar los modelos solicitados.

Cuando se activa la opción *Ensure satisfiable*, la generación puede detenerse si el sistema no encuentra un modelo que el comprobador SAT confirme como satisfacible

tras 20 intentos para ese modelo. En ese caso, el asistente muestra un mensaje de error y detiene el lote. Este resultado no demuestra que la configuración sea insatisfacible; indica que no se ha obtenido un modelo confirmado dentro del límite de intentos establecido.

Como indica el mensaje de la Figura D.2, se puede simplificar la configuración, reducir el número de restricciones o desactivar *Ensure satisfiable* y volver a generar. Si se desactiva esta opción, el sistema no realiza la comprobación SAT de los modelos generados.

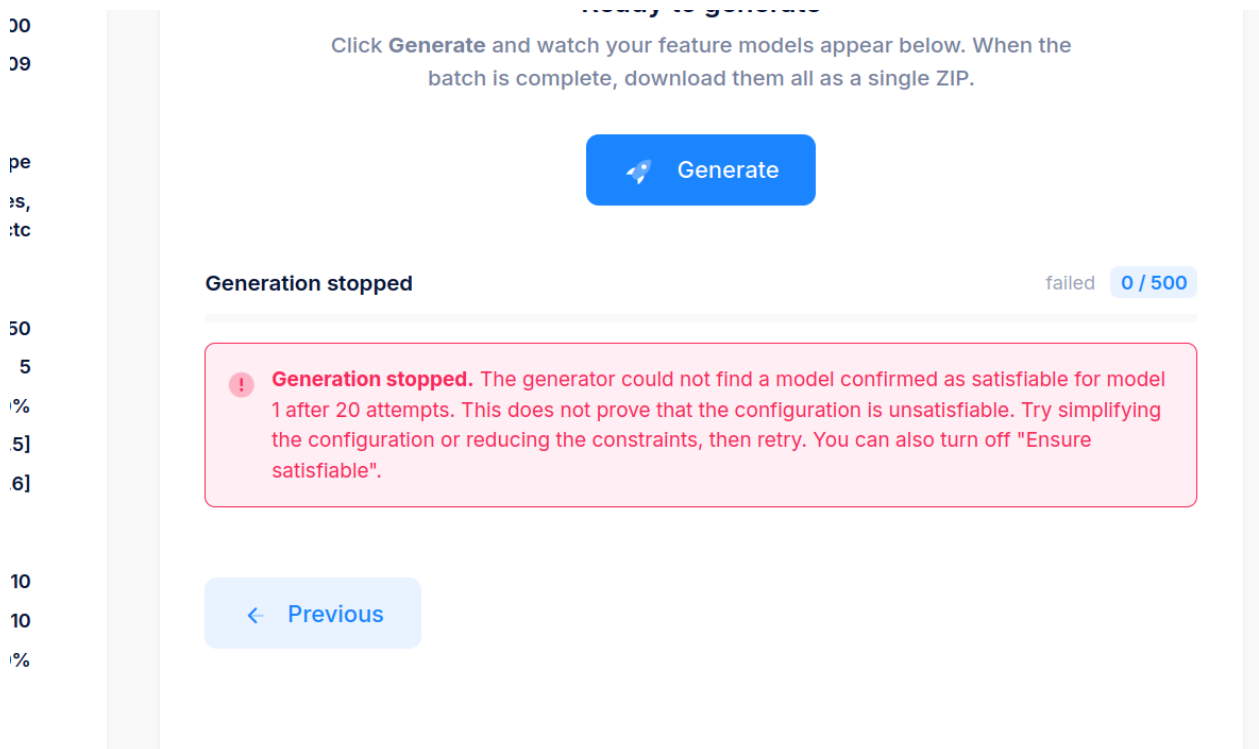


Figura D.2: Mensaje mostrado cuando no se obtiene un modelo confirmado como satisfacible tras 20 intentos. Captura realizada por el autor.

D.4 Contacto y soporte

Para cualquier duda relacionada con el funcionamiento del sistema o para reportar errores no contemplados en este manual, se puede contactar con el desarrollador mediante las siguientes direcciones de correo electrónico:

- antjimort@alum.us.es
- ajimenez34@us.es